

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase — cuyo aporte metodológico central es un procedimiento reproducible para etiquetar como *compute-bound* o *memory-bound* cada ventana de ejecución de CPU o corrida completa de GPU, mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación — junto con un primer entrenamiento y análisis del clasificador de GPU, posible ya dentro de esta fase por disponer de su conjunto de entrenamiento definido.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy-Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	12
Planteamiento y justificación del problema	13
Objetivos	14
1. Marco de referencia	16
1.1. Marco Conceptual	16
1.1.1. Potencia CMOS y fundamento de DVFS	16
1.1.2. Modelo Roofline y regímenes de ejecución	17
1.1.3. Telemetría microarquitectónica con PMU	19
1.1.4. Espacios de ejecución <i>user-space</i> y <i>kernel-space</i>	21
1.1.5. Interfaces de observabilidad	21
1.1.6. DVFS, P-states y mecanismos de actuación	22
1.1.7. Cargas de calibración y modelos de clasificación ligera	23
1.1.8. Hiperparámetros y Optuna	24
1.1.9. Producto Energía–Retardo (EDP)	24
1.2. Estado del Arte	25
2. Metodología	28
2.1. Enfoque y diseño	28
2.1.1. Población, muestra y criterios de inclusión	29
2.1.2. Plataforma experimental	29
2.1.3. Instrumento de medición	30
2.1.4. Modelo Roofline y etiquetado	31
2.2. Fase 1: recolección y caracterización	32
2.2.1. Verificación previa y catálogo	32
2.2.2. Matriz experimental y control de sesgos	33
2.2.3. Calidad de los datos y granularidad	34
2.2.4. Reproducibilidad	34
2.3. Fase 2: modelos de clasificación y política de frecuencia	34

2.3.1.	Formulación, variables y validación	34
2.3.2.	Decisión selectiva y calibración	36
2.3.3.	Comparación y selección de modelos	36
2.3.4.	Invarianza de las entradas frente a la acción del agente	37
2.3.5.	Derivación de la política de frecuencia	37
2.4.	Fase 3: agente de control DVFS	37
2.4.1.	Arquitectura	38
2.4.2.	Detección de fase, decisión y parámetros	39
2.4.3.	Política, actuación y coordinación	40
2.4.4.	Aplicaciones compuestas y verificación previa	41
2.5.	Fase 4: validación experimental	42
2.5.1.	Diseño de la matriz	42
2.5.2.	Escenarios adicionales	43
2.5.3.	Medición y análisis	43
3.	Resultados	45
3.1.	Resultados de CPU	45
3.1.1.	Composición de fase por kernel	45
3.1.2.	Variación de la composición con la frecuencia	47
3.1.3.	Techos Roofline y <i>ridge</i>	47
3.1.4.	Clasificador de fase en CPU	48
3.1.5.	Invarianza del clasificador de CPU frente a la acción del agente	57
3.1.6.	Política de frecuencia por clase en CPU	58
3.2.	Resultados de GPU	60
3.2.1.	Validación de control y etiquetado	60
3.2.2.	Conjunto de datos y clasificador de fase en GPU	61
3.2.3.	Candidato operativo de GPU: invarianza frente a la acción del agente	62
3.2.4.	Política de frecuencia por clase en GPU	65
3.3.	Resultados del agente de control	68
3.3.1.	Candado de reloj de GPU y premisa física de la política	69
3.3.2.	Latencia, restauración y detección del agente	69
3.4.	Evaluación del agente frente a la base	70
3.4.1.	Aplicaciones compuestas	70
3.4.2.	Clasificación de fase con el agente en el lazo	71
3.4.3.	Tiempo, energía, EDP y costo del agente frente a la base	72
3.4.4.	Potencial de la GPU	73
3.4.5.	Escenario E: GPU dominada por fases de memoria	74
3.4.6.	Escenario D: LAMMPS como aplicación de terceros	77
3.4.7.	CloverLeaf CUDA Fortran como aplicación continua	77
4.	Discusión	79
4.1.	Validez de la medición	79
4.2.	Generalización de los clasificadores	79
4.3.	Alcance de la ganancia energética	80

5. Conclusiones	81
5.1. Limitaciones	82
5.2. Trabajo futuro	82
A. Detalle técnico de la metodología	84
A.1. Diseño de la medición	84
A.1.1. Muestreo y dominio <i>uncore</i>	84
A.1.2. Detección del transitorio de arranque	84
A.2. Fase 1. Recolección y caracterización	85
A.2.1. Catálogo	85
A.2.2. Conteo de FLOPs e intensidad de GPU	87
A.3. Fase 2. Desarrollo y validación de los modelos	87
A.3.1. Variables y prevención de fuga	87
A.3.2. Ponderación y diagnóstico del límite	87
A.3.3. Procedimiento estadístico de la política	88
A.4. Señales y unidad de observación	88
A.5. Amenazas a la validez	89
A.6. Protocolos de la Fase 4	89
B. Resultados complementarios	91
B.1. Conjunto de datos y clasificador de CPU	91
B.1.1. Tamaño de la muestra	92
B.1.2. Vector de entrada y protocolo de partición	93
B.1.3. Métricas de los modelos finales	94
B.1.4. Auditoría de frecuencia de la campaña CPU	94
B.2. Agente y Fase 4	95
Repositorio y recursos complementarios	103

Índice de tablas

2.1. Plataforma experimental.	30
2.2. Rejilla fina de frecuencia de la campaña definitiva	33
2.3. Brazos del agente.	39
2.4. Parámetros del agente.	40
2.5. Acciones del agente	41
2.6. Aplicaciones compuestas con verdad de fase.	42
2.7. Matriz experimental de la Fase 4.	42
2.8. Mapa de los escenarios de la Fase 4	43
3.1. Configuración final del clasificador CPU.	52
3.2. Síntesis de la política GPU	66
3.3. Composición del escenario E	75
A.1. Verificaciones previas agrupadas por familia.	85
A.2. Catálogo de kernels y procedencia de las suites.	85
A.3. Señales de telemetría del instrumento.	88
A.4. Unidad de observación del conjunto de entrenamiento.	88
A.6. Amenazas a la validez y medidas de control.	89
A.7. Protocolos complementarios ejecutados en la Fase 4.	90
B.1. Flujo de intervalos de la campaña CPU	92
B.2. Sensibilidad al tope de intervalos por celda	92
B.3. Variantes del vector de entrada	93
B.4. Efecto del esquema de partición	93
B.5. Valores de todas las métricas de los modelos finales	94
B.6. Restauración bajo terminación anómala y coordinación entre procesos.	95

Índice de figuras

1.1. Modelo Roofline conceptual	17
1.2. Unidad de Monitorización de Rendimiento (PMU)	19
1.3. Contadores de núcleo y contadores de <i>uncore</i>	20
1.4. Espacios de usuario y de kernel	21
1.5. P-states, C-states y jerarquía DVFS	22
2.1. Fases de la metodología	29
2.2. Recorrido de una campaña a través de las dos capas del instrumento	31
2.3. Obtención de la etiqueta en CPU y en GPU	32
2.4. Validación anidada por familia	35
2.5. Arquitectura del agente	38
3.1. Composición de fase por familia en CPU	46
3.2. Composición de fase por nivel de frecuencia en CPU	47
3.3. <i>Ridge</i> Roofline según el nivel de frecuencia CPU	48
3.4. Muestreo estratificado del conjunto CPU	49
3.5. Correlación entre las variables candidatas del clasificador CPU	50
3.6. Comparación de modelos del clasificador CPU	51
3.7. Matrices de confusión del clasificador CPU	52
3.8. Resultado del clasificador CPU por familia	53
3.9. Exactitud y cobertura del clasificador CPU por nivel de frecuencia	54
3.10. Calibración y compromiso de la abstención en CPU	55
3.11. Perfiles de celdas confusables entre clases opuestas	56
3.12. Exactitud según la distancia al <i>ridge</i>	56
3.13. Curva de aprendizaje por número de familias	57
3.14. Ganancia del EDP por nivel y clase, en forma de <i>forest plot</i>	58
3.15. Energía, tiempo y producto energía-retardo por nivel de frecuencia	59
3.16. Potencia de paquete y DRAM frente a la frecuencia	59
3.17. Comparación de modelos candidatos GPU	62
3.18. Invarianza de los candidatos GPU frente al reloj	63
3.19. Resultado del <i>random forest</i> de GPU por familia	63
3.20. Calibración y abstención del <i>random forest</i> de GPU	64
3.21. Matrices de confusión del <i>random forest</i> de GPU	65
3.22. Importancia de las entradas del <i>random forest</i> de GPU	65
3.23. Ganancia del EDP GPU por nivel y clase, en forma de <i>forest plot</i>	67

3.24. Tiempo, energía y EDP relativos por nivel de frecuencia GPU	68
3.25. Potencia de GPU frente al reloj SM y piso estático	68
3.26. Rendimiento de GPU con el reloj fijado	69
3.27. Ciclo de cada aplicación compuesta	71
3.28. EDP del nodo relativo a la base en la matriz inicial	73
3.29. Potencial de la GPU en fases <i>memory_bound</i>	74
3.30. Escenario E: EDP del nodo y energía de GPU	76
3.31. EDP mediano de LAMMPS por entrada y brazo	77
3.32. Resultados de CloverLeaf frente a la base	78
A.1. Ruta de muestreo con estructura circular de un productor y un consumidor .	84
B.1. Composición de fase por configuración de kernel en CPU	91

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como `ondemand` o `schedutil`, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente en

espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como compute-bound o memory-bound a partir de telemetría microarquitectónica, y de ajustar dinámicamente la frecuencia de CPU y GPU para optimizar el Producto Energía–Retardo (EDP). Se plantea demostrar que este enfoque mejora la eficiencia energética frente a los gobernadores nativos de Linux, manteniendo una degradación controlada del rendimiento y una sobrecarga de inferencia mínima y cuantificada mediante métricas experimentales.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [1] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [2].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-bound*, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [3, 4].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar

frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [5], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [1, 2, 3, 4, 5]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS)

en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El marco conceptual parte de la relación entre el ajuste de la frecuencia y del voltaje (DVFS), la potencia requerida por el hardware y el tiempo de ejecución, cuyo comportamiento depende del régimen que limita cada carga. Régimen que puede estar condicionado por capacidad de cómputo (compute-bound) o de memoria (memory-bound), dos límites que se pueden distinguir mediante el modelo Roofline y algunas señales del sistema asociadas con este comportamiento durante la ejecución. Señales que funcionan como vector de entrada a un clasificador ligero que intenta inferir el régimen, permitiendo así tomar decisiones de ajuste de frecuencia, teniendo en cuenta el Producto Energía–Retardo (EDP), el cual permite relacionar el consumo energético con el tiempo de ejecución para valorar el efecto de dichas decisiones.

1.1.1. Potencia CMOS y fundamento de DVFS

La potencia dinámica de un circuito CMOS depende del factor de actividad, la capacitancia efectiva, el voltaje de alimentación y la frecuencia de operación. Esta relación es la base física de DVFS y se expresa como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α representa el factor de actividad, C la capacitancia efectiva conmutada, V el voltaje de alimentación y f la frecuencia de operación [6]. El voltaje y la frecuencia están relacionados dentro de los estados de rendimiento, ya que para trabajar a frecuencias más

altas normalmente se necesita un mayor voltaje. Por esto, al reducir la frecuencia también puede disminuir el voltaje, lo que afecta directamente la potencia dinámica debido a la relación cuadrática de V en la ecuación.

Sin embargo, esta reducción de potencia por sí sola no asegura un menor consumo de energía, porque también interviene el tiempo de ejecución. Si la aplicación tarda más en finalizar al trabajar a una frecuencia menor, parte del ahorro obtenido puede perderse por ese tiempo adicional. Además, durante toda la ejecución sigue existiendo consumo asociado a la potencia estática. Por eso, el efecto energético de cambiar la frecuencia también depende de cómo responda la carga de trabajo.

1.1.2. Modelo Roofline y regímenes de ejecución

El modelo Roofline relaciona el rendimiento alcanzable de una carga con la capacidad de cómputo y el ancho de banda de memoria de la plataforma. La intensidad operacional determina cuál de estos dos límites condiciona el rendimiento y se incorpora al modelo como:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} representa el rendimiento aritmético pico, BW_{pico} el ancho de banda pico de memoria e I la intensidad operacional [7]. Cuando el rendimiento queda limitado por $I \cdot BW_{\text{pico}}$, la carga se encuentra en régimen *memory-bound*. Si el límite corresponde a P_{pico} , la carga se encuentra en régimen *compute-bound*. La Figura 1.1 muestra esta separación entre ambos regímenes.

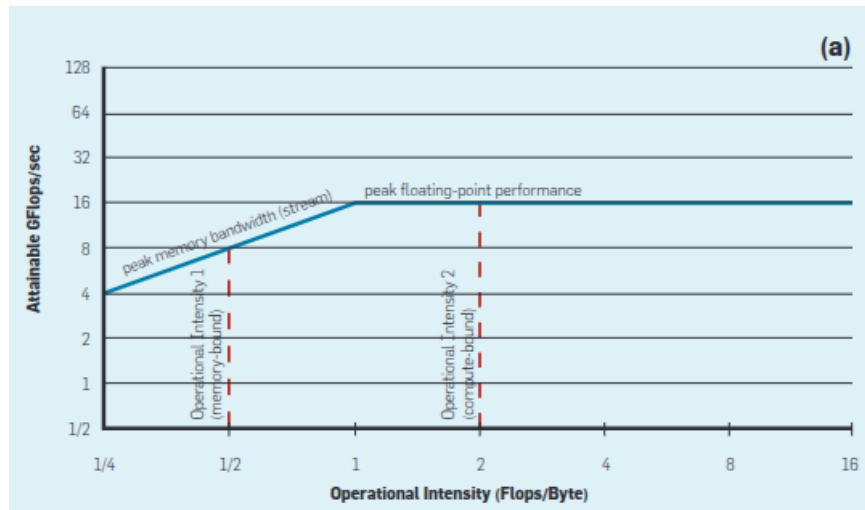


Figura 1.1: Modelo Roofline conceptual con el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Nota. Tomado de Williams et al. [7, Fig. 1(a)].

La intensidad operacional expresa la cantidad de operaciones realizadas por cada byte transferido desde el nivel de memoria considerado [7, p. 66]. Por ello, no depende únicamente del tamaño del conjunto de datos ni de si este cabe o no en caché. La reutilización de datos puede reducir el tráfico hacia memoria y aumentar la intensidad operacional, incluso cuando el conjunto de trabajo supera la capacidad de la caché [8, Secs. 3–4]. El modelo establece así un límite de rendimiento para cada intensidad, aunque una carga puede permanecer por debajo de ese límite por otros efectos de la ejecución.

La separación entre los dos regímenes ocurre en el punto de inflexión, también denominado *ridge point*. En ese punto, el techo de cómputo y el límite impuesto por el ancho de banda tienen el mismo valor, por lo que el umbral se obtiene como:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

Una intensidad inferior a I_{ridge} corresponde al régimen *memory-bound*, mientras que una intensidad superior corresponde al régimen *compute-bound* [7]. El *ridge point* funciona como frontera entre ambos regímenes y no como una frecuencia óptima de ejecución. Si alguno de los techos cambia, también puede cambiar la posición de esta frontera.

El techo de cómputo depende de la capacidad aritmética disponible y puede variar con la precisión de las operaciones y con la frecuencia de las unidades de cómputo [9]. El ancho de banda de memoria pertenece a otro dominio de la arquitectura y no tiene que cambiar en la misma proporción cuando se modifica la frecuencia [10, 5]. Por ello, el valor de I_{ridge} utilizado para clasificar una carga debe corresponder a la precisión y al punto de operación bajo el cual se realiza la observación. Los valores nominales sirven como referencia, pero la clasificación requiere techos representativos de las condiciones de ejecución consideradas [11].

La intensidad operacional observada se obtiene a partir del trabajo aritmético y del tráfico hacia memoria medidos durante el mismo intervalo. Para un intervalo i se calcula como:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

donde FLOPs_i representa las operaciones realizadas y B_i^{DRAM} los bytes transferidos hacia o desde memoria principal durante ese intervalo [7]. Los eventos utilizados para obtener los FLOPs dependen de la microarquitectura y deben interpretarse de acuerdo con la semántica de la PMU [12]. El tráfico de DRAM se obtiene desde contadores asociados al controlador

de memoria, cuyo ámbito puede ser diferente al de los contadores de núcleo. Ambas mediciones deben corresponder al mismo intervalo para que la intensidad operacional conserve su significado físico.

1.1.3. Telemetría microarquitectónica con PMU

La clasificación en CPU requiere señales que describan el comportamiento de la ejecución sin reconstruir en línea la intensidad operacional. Las Unidades de Monitorización de Rendimiento (PMU), proporcionan esa observabilidad mediante contadores de hardware que acumulan eventos como instrucciones retiradas, ciclos y actividad de la jerarquía de memoria. Estos contadores pueden ser fijos, asociados a eventos arquitectónicos predefinidos, o programables, configurados para registrar eventos específicos de la arquitectura [13, 14]. La siguiente figura resume esta organización:

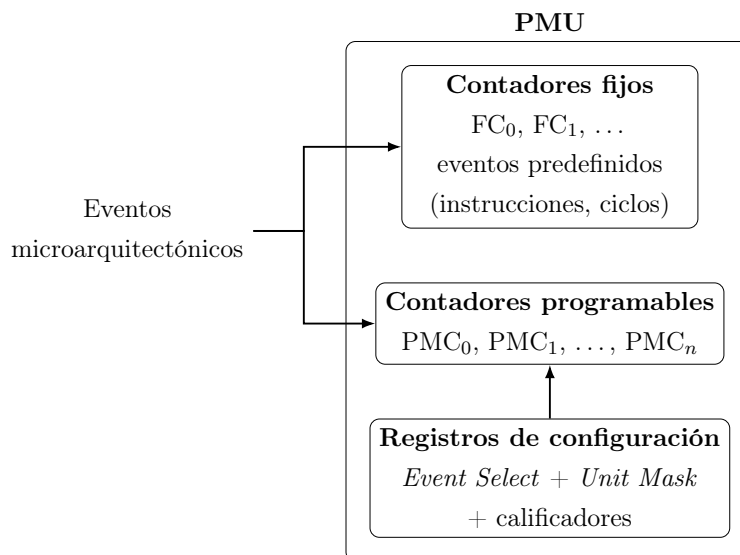


Figura 1.2: Organización conceptual de una Unidad de Monitorización de Rendimiento (PMU) con contadores fijos y programables. Fuente: elaboración propia.

Sin embargo, PMU no interpreta el comportamiento de la aplicación, sino que entrega conteos cuyo valor depende del evento y del dominio observado. Por ello, el mismo valor numérico solo adquiere significado después de considerar qué recurso cuenta, durante qué intervalo y con qué alcance.

Un problema que presentan estos contadores es la multiplexación, que ocurre cuando se solicitan más eventos de los que el hardware puede medir al mismo tiempo. En estas condiciones, el sistema alterna distintos grupos de eventos y cada contador permanece activo solo durante una parte de la ventana de medición, por lo que el valor registrado se ajusta

teniendo en cuenta el tiempo durante el cual estuvo realmente activo [15, 16]. Aunque este mecanismo permite trabajar con más eventos de los disponibles físicamente, la estimación puede perder precisión si la aplicación cambia de fase durante los periodos en los que un evento no está siendo medido, lo que hace conveniente limitar la cantidad de señales observadas de forma simultánea.

Otro detalle que también es importante tener en cuenta es el dominio de los contadores, puesto que existen contadores de núcleo y contadores de *uncore* que difieren en su nivel de acceso y alcance. Los primeros observan recursos asociados a un núcleo y pueden vincularse con la ejecución de una tarea, mientras que los de *uncore* cubren componentes compartidos del zócalo, entre ellos el controlador de memoria integrado y partes de la interconexión, desde donde se puede observar el movimiento efectivo de datos hacia memoria principal. Sin embargo, al tratarse de medidas con alcance compartido, el tráfico registrado por los contadores de *uncore* no puede atribuirse directamente a un proceso individual sin condiciones adecuadas de aislamiento. Esta diferencia entre una medida local de núcleo y una medida compartida también hace que ambas fuentes puedan trabajar con granularidades temporales distintas, por lo que no deben combinarse fila a fila sin establecer antes una correspondencia temporal. La siguiente figura muestra esta diferencia estructural.

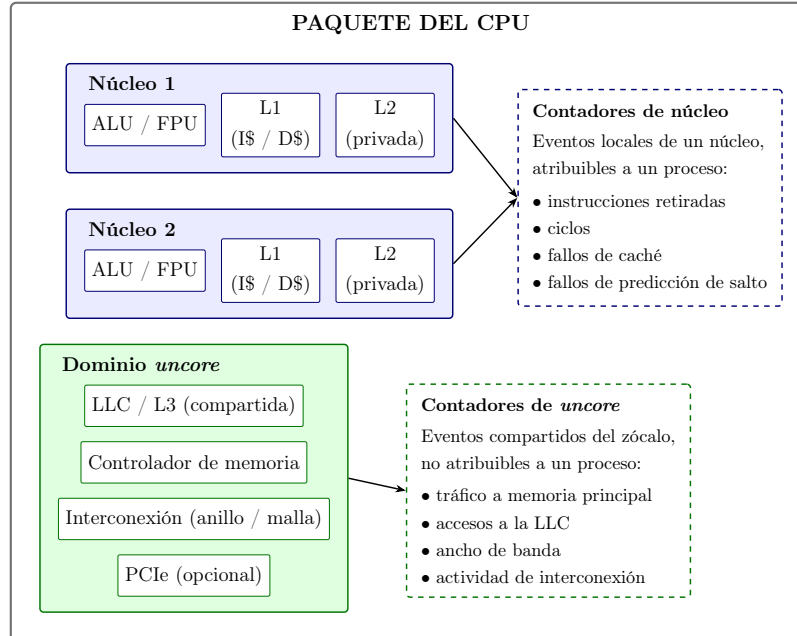


Figura 1.3: Ámbito de atribución de los contadores de núcleo y de *uncore* dentro del paquete del procesador. Fuente: Elaboración propia.

1.1.4. Espacios de ejecución *user-space* y *kernel-space*

Los sistemas operativos modernos emplean distintos niveles de privilegio para regular el acceso a los recursos del sistema, lo que permite distinguir entre los procesos que se ejecutan en *user-space* y aquellos que operan en *kernel-space*. Los procesos en *user-space* trabajan con acceso restringido, mientras que el *kernel-space* concentra las operaciones que requieren acceso directo a recursos protegidos, como la gestión de memoria, la entrada/salida o ciertas funciones del hardware [17, ch. 2]. Debido a esta separación, una aplicación en espacio de usuario no accede directamente a estos recursos y depende de las interfaces que el sistema operativo expone para realizar operaciones controladas [18], y es esta condición la cual también influye en la ubicación de programas de monitoreo y control, ya que determina la forma en que pueden consultar contadores, leer información de energía o solicitar cambios de frecuencia sin modificar estructuras internas del kernel. La siguiente figura detalla más la estructura de estos espacios y la relación conceptual entre ellos y el hardware físico.



Figura 1.4: Relación conceptual entre *user-space*, *kernel-space* y el hardware físico. Fuente: Elaboración propia.

1.1.5. Interfaces de observabilidad

Algunos sistemas operativos proporcionan interfaces que permiten a los programas en espacio de usuario consultar información del hardware sin acceder directamente a sus estructuras internas. En Linux, los eventos de la PMU pueden obtenerse mediante `perf_event_open`, una interfaz que permite configurar los contadores disponibles y consultar sus valores durante la ejecución [16], aunque su disponibilidad depende del soporte del procesador y de los permisos definidos en el sistema.

Estos conteos permiten observar la actividad del procesador, pero no cubren por sí solos la información energética. Para ello puede utilizarse Intel RAPL (*Running Average Power Limit*), un mecanismo incorporado en procesadores Intel que expone contadores de energía para distintos dominios, entre ellos el paquete del procesador y, cuando está disponible, la memoria DRAM [19, 20]. Sus valores representan energía acumulada durante la ejecución, por lo que pueden utilizarse para conocer el consumo asociado a un intervalo y posteriormente relacionarlo con métricas como el EDP.

1.1.6. DVFS, P-states y mecanismos de actuación

El escalado dinámico de voltaje y frecuencia (*Dynamic Voltage and Frequency Scaling*, DVFS) permite modificar el punto operativo del procesador para adaptar su capacidad de cómputo y su consumo energético a las necesidades de la carga. Estos cambios se apoyan en mecanismos de gestión de potencia que reciben una solicitud de rendimiento y la materializan sobre el hardware [21].

En CPU, los *Performance States* o P-states representan distintos niveles de rendimiento mientras el procesador permanece activo en C0. P0 corresponde al mayor nivel de rendimiento y los estados siguientes reducen progresivamente la capacidad disponible. Los C-states, en cambio, representan estados de inactividad con distintos compromisos entre ahorro energético y latencia de reactivación. Sobre los P-states operan los *governors*, políticas software que deciden qué nivel de rendimiento solicitar según el estado del sistema. En Linux, **performance** y **powersave** favorecen extremos distintos del rango disponible, mientras que **ondemand**, **conservative** y **schedutil** ajustan la solicitud de forma dinámica; **userspace**, por su parte, permite que una aplicación solicite explícitamente el nivel deseado [21, 22].

La Figura 1.5 reúne estos dos niveles de funcionamiento. En la parte (a) se representa la relación entre los estados de rendimiento y de inactividad del procesador, mientras que la parte (b) muestra el recorrido de una solicitud desde la interfaz de espacio de usuario, pasando por las políticas y los controladores del sistema, hasta los mecanismos que actúan sobre el hardware. Esta separación permite entender también por qué una decisión de rendimiento no se traduce necesariamente de forma inmediata en la frecuencia observada.

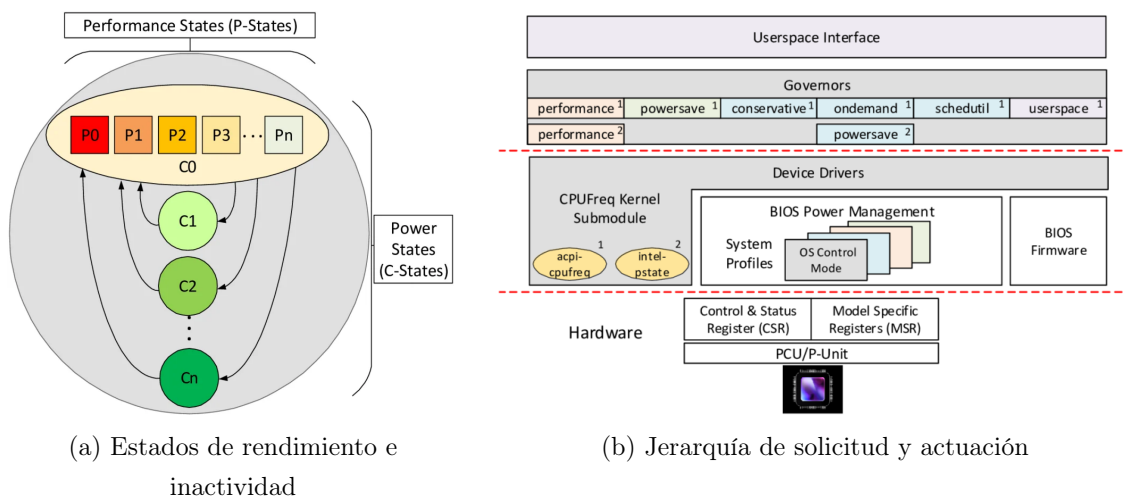


Figura 1.5: Relación entre los estados de rendimiento e inactividad del procesador y la jerarquía mediante la cual una política de rendimiento solicita cambios que posteriormente son materializados por el hardware. Fuente: Tomado de Hebbar y Milenković [21].

En GPU, DVFS mantiene el mismo principio general de ajustar el punto de operación del dispositivo, aunque la actuación suele realizarse sobre dominios de reloj distintos, principalmente el reloj de cómputo y el de memoria. Estos cambios dependen de los mecanismos de gestión propios de la arquitectura y del controlador, que determinan los niveles de frecuencia disponibles y la forma en que una solicitud se aplica sobre el hardware [10, 5]. Al igual que en CPU, la transición entre niveles requiere un tiempo de actuación, por lo que el cambio de frecuencia no es instantáneo [23].

1.1.7. Cargas de calibración y modelos de clasificación ligera

Las aplicaciones de cómputo pueden combinar fases con demandas distintas sobre las unidades de procesamiento y la jerarquía de memoria, por lo que su comportamiento puede observarse mediante cargas o unidades de ejecución más acotadas. Entre ellas se encuentran los *kernels*, que corresponden a operaciones computacionales bien delimitadas, como una multiplicación de matrices, una transformada de Fourier o un stencil, y permiten observar con mayor claridad cómo una operación utiliza los recursos del sistema [7]. También se encuentran los *benchmarks*, que proporcionan cargas utilizadas para medir o comparar el comportamiento de una plataforma bajo condiciones reproducibles, y los *microbenchmarks*, que se construyen para aislar propiedades más específicas, como el ancho de banda de memoria, la latencia de acceso o la capacidad de ejecución aritmética [24].

Las observaciones obtenidas durante estas ejecuciones pueden representarse mediante un conjunto de características y una clase asociada, relación que puede aprenderse mediante clasificación supervisada para después inferir la clase de nuevas observaciones. Cuando este proceso debe realizarse con un costo reducido, conviene trabajar con modelos de clasificación ligera, entendidos como modelos cuya complejidad permite mantener un tiempo de inferencia bajo sobre un número reducido de características.

Existen varios modelos ligeros y no todos representan de la misma forma la relación entre las características. Por ejemplo, la regresión logística establece una frontera lineal y de bajo costo, por lo que sirve como referencia cuando la separación entre clases puede describirse mediante relaciones relativamente simples [25, Ch. 4]. Un árbol de decisión, en cambio, permite introducir relaciones no lineales mediante divisiones sucesivas de las características y conserva una estructura fácil de interpretar, aunque su complejidad aumenta a medida que crece la profundidad del árbol [25, Ch. 8].

A partir de los árboles también pueden construirse modelos de ensamble, como Random Forest, que combina varios árboles generados mediante muestreo y selección aleatoria de

características, reduciendo la dependencia de una única partición y permitiendo representar interacciones no lineales entre las variables [26]. Extra-Trees parte de una estructura similar, pero introduce mayor aleatoriedad en la selección de los puntos de corte [27], por lo que ambos modelos pueden trabajar con relaciones más complejas que las representadas por una frontera lineal.

XGBoost también utiliza árboles de decisión, pero los construye de manera secuencial, de modo que cada nuevo árbol busca corregir parte de los errores acumulados por los anteriores [28]. Esto le permite representar relaciones más complejas, aunque incorpora un mayor número de parámetros y su costo depende de la configuración del ensamble. En conjunto, estas familias cubren distintos niveles de complejidad, desde una frontera lineal hasta conjuntos de árboles capaces de representar interacciones no lineales, manteniendo como consideración común el costo necesario para realizar la inferencia.

1.1.8. Hiperparámetros y Optuna

Los hiperparámetros controlan aspectos de la estructura y del proceso de entrenamiento de un modelo, por lo que su elección puede modificar tanto la capacidad de ajuste como su complejidad. En modelos como Random Forest, Extra-Trees o XGBoost, estos valores pueden incluir cantidades de árboles, profundidades, tasas de aprendizaje u otros parámetros de configuración, lo que genera espacios de búsqueda con combinaciones muy diferentes entre sí. Explorar todas las posibilidades mediante una rejilla puede resultar costoso, de modo que suelen emplearse métodos de optimización que orientan la búsqueda hacia configuraciones más prometedoras.

Optuna es un marco de optimización de hiperparámetros que realiza esta exploración de forma secuencial [29]. Entre sus estrategias se encuentra el estimador de Parzen estructurado (TPE), que utiliza los resultados obtenidos en evaluaciones anteriores para proponer nuevas configuraciones con mayor probabilidad de mejorar el objetivo [30]. De esta manera, la búsqueda no necesita recorrer todas las combinaciones posibles y puede adaptarse a espacios que mezclan variables enteras, continuas y categóricas.

1.1.9. Producto Energía–Retardo (EDP)

Los cambios de frecuencia mediante DVFS afectan tanto el consumo de energía como el tiempo de ejecución, por lo que reducir una de estas magnitudes no significa necesariamente que el resultado sea mejor. Es por esto que el Producto Energía–Retardo (*Energy–Delay Product*, EDP) relaciona ambas en una sola expresión:

$$EDP = E \cdot T$$

donde E representa la energía consumida y T el tiempo de ejecución [31]. Según el valor obtenido, un EDP menor representa una relación más favorable entre energía y tiempo. Sin embargo, ahorrar energía no siempre resulta conveniente si para lograrlo la ejecución tarda mucho más, ya que durante ese tiempo adicional el sistema continúa consumiendo energía, incluida la asociada a la potencia estática.

El EDP reúne la energía y el tiempo en un solo valor, aunque ambas magnitudes todavía deben revisarse por separado, porque dos configuraciones pueden obtener un EDP parecido aun cuando una consuma menos energía y la otra tenga un menor tiempo de ejecución. Por eso, la métrica permite comparar distintos puntos de operación teniendo en cuenta ambos efectos al mismo tiempo.

1.2. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. Dentro de este proceso, el escalado dinámico de voltaje y frecuencia (DVFS) se ha consolidado como uno de los mecanismos más estudiados para modificar el punto operativo de procesadores y aceleradores. Su aplicación, sin embargo, no conduce necesariamente al mismo resultado para todas las cargas de trabajo, ya que una reducción de frecuencia puede disminuir la potencia instantánea, pero también prolongar el tiempo de ejecución y anular el beneficio energético esperado. Esta dependencia ha llevado a que buena parte de la investigación pase de aplicar DVFS de forma uniforme a buscar mecanismos capaces de reconocer el comportamiento de la carga y ajustar la frecuencia de acuerdo con sus características.

Las primeras aproximaciones que analizaron DVFS sobre regiones concretas de una aplicación ya mostraban que mantener una misma frecuencia durante toda la ejecución podía desaprovechar oportunidades de ahorro. Calore et al. [4], al estudiar distintos kernels y una aplicación HPC completa sobre una CPU Intel Haswell y una GPU NVIDIA K80, observaron que el efecto energético variaba entre regiones y arquitecturas. En lugar de mantener un único estado operativo, evaluaron cambios de frecuencia a nivel de función, mostrando que la conveniencia de cada configuración dependía del comportamiento de la región ejecutada y de la plataforma sobre la cual se realizaba el cálculo.

Esta dependencia de la carga llevó a buscar formas de anticipar su respuesta sin recorrer todas las frecuencias disponibles. Guerreiro et al. [10] estudiaron aplicaciones GPGPU a partir de métricas recolectadas en una configuración inicial y utilizaron esa caracterización para estimar cómo responderían ante cambios en los dominios de cómputo y memoria. Con ello redujeron la necesidad de explorar exhaustivamente el espacio DVFS y mostraron que la información de rendimiento podía utilizarse para agrupar aplicaciones según su sensibilidad a la frecuencia y orientar la selección hacia configuraciones energéticamente favorables.

Sobre una idea similar, Ali et al. [5] propusieron un método automatizado y portable para seleccionar frecuencias de GPU a partir de métricas obtenidas inicialmente en la configuración predeterminada del dispositivo. Con esta caracterización construyeron modelos de potencia y tiempo de ejecución para estimar el comportamiento de la aplicación en otros estados DVFS sin ejecutarla en cada uno de ellos. La selección consideró de forma conjunta el consumo energético y la degradación del desempeño, evitando interpretar una reducción de potencia como una mejora cuando venía acompañada de un incremento importante en el tiempo de ejecución. En las plataformas evaluadas, la propuesta reportó reducciones promedio de energía de 29.6 % con una pérdida de rendimiento de 5.2 % sobre NVIDIA GA100 y de 22.6 % con una pérdida de 4.7 % sobre NVIDIA GV100 [5].

Aunque estos enfoques permiten reducir el espacio de búsqueda, parten principalmente de una caracterización con la que se anticipa el comportamiento posterior de la aplicación. Cuando la carga cambia durante la ejecución aparece otra necesidad, reconocer el régimen que domina en cada momento. Antici et al. [3], mediante MCBound, abordaron esta identificación como un problema de clasificación entre cargas *memory-bound* y *compute-bound*, utilizando información de ejecuciones HPC para caracterizar automáticamente nuevos trabajos. Su evaluación sobre datos del supercomputador Fugaku mostró que esta separación puede realizarse con una puntuación F1-macro de al menos 0.89 y con un costo reducido para el funcionamiento del sistema [3]. Este resultado aporta evidencia de que la naturaleza computacional de una carga puede inferirse automáticamente sin depender únicamente de una clasificación manual basada en el conocimiento previo de la aplicación.

La clasificación de MCBound resulta especialmente cercana al problema de identificación del régimen, aunque su objetivo no es gobernar directamente los estados DVFS de CPU o GPU. Aun así, permite separar dos partes que aparecen de manera recurrente en los trabajos revisados, primero reconocer el comportamiento de la carga y luego decidir cómo utilizar esa información para modificar el punto de operación. La segunda parte ha sido explorada desde distintas aproximaciones orientadas a adaptar la frecuencia durante la propia ejecución.

Simsek et al. [1], por ejemplo, estudiaron esta actuación dinámica en simulaciones astrofísicas ejecutadas sobre GPU. Los autores instrumentaron SPH-EXA para medir el comportamiento energético de diferentes regiones y modificar la frecuencia del acelerador en puntos determinados de la ejecución. Frente a una política de frecuencia uniforme, esta adaptación permitió reducir el consumo energético por GPU hasta en 7.82 %, con una degradación del rendimiento de aproximadamente 2.95 %. El trabajo muestra que existe un beneficio potencial al cambiar la frecuencia durante la ejecución, aunque la decisión permanece asociada a regiones previamente conocidas e instrumentadas dentro de la aplicación.

Un paso más hacia un control *online* completo aparece en DRLCAP, un marco de *frequency capping* para GPU basado en aprendizaje por refuerzo profundo. DRLCAP monitoriza información a nivel de sistema, detecta cambios de fase y ajusta el límite de frecuencia sin intervención manual, logrando una reducción promedio de 22 % en la energía de GPU con menos de 3 % de degradación de rendimiento en arquitecturas NVIDIA [32]. Este resultado muestra que el control de frecuencia durante la ejecución puede automatizarse a partir de señales observadas en línea, aunque mediante un enfoque de aprendizaje por refuerzo profundo y concentrado exclusivamente en GPU.

Los trabajos revisados muestran una transición desde la aplicación de configuraciones DVFS sobre regiones conocidas hacia mecanismos que utilizan información de ejecución para anticipar o adaptar la frecuencia. En este recorrido, la caracterización de la carga ha permitido reducir búsquedas exhaustivas, mientras que enfoques como MCBound muestran que el régimen computacional también puede inferirse mediante clasificación. De forma paralela, los trabajos de Simsek et al. [1] y Wang et al. [32] evidencian que la frecuencia puede ajustarse durante la propia ejecución, aunque mediante estrategias ligadas a regiones previamente instrumentadas o a modelos de aprendizaje más complejos.

Aun con estos avances, gran parte de los enfoques recientes de predicción y control dinámico se concentra en GPU, mientras que CPU y GPU presentan diferencias tanto en sus mecanismos de frecuencia como en las señales disponibles para observar la ejecución. Calore et al. [4] consideran ambas arquitecturas, pero Guerreiro et al. [10], Ali et al. [5], Simsek et al. [1] y Wang et al. [32] profundizan principalmente en aceleradores gráficos. Esta diferencia deja espacio para estudiar estrategias que integren ambas plataformas y utilicen una caracterización de bajo costo como apoyo para decisiones DVFS durante la ejecución, sin depender de una instrumentación específica de cada aplicación.

Capítulo 2

Metodología

2.1. Enfoque y diseño

La investigación es cuantitativa, experimental y aplicada. Es experimental porque manipula de forma deliberada el estado de frecuencia de CPU y GPU sobre un conjunto controlado de cargas y observa su efecto sobre el tiempo, la energía y el Producto Energía–Retardo. Es aplicada porque su producto es un agente de software funcional cuya utilidad se evalúa empíricamente frente a gobernadores nativos del sistema operativo y del controlador de la GPU. El estado del dispositivo sin agente es el **escenario base** (en adelante, la base), que la rejilla de frecuencia rotula **REF**; ese rótulo se conserva solo en la rejilla, en las tablas por nivel y en las figuras, y en el texto se usa base. La base tiene el turbo desactivado: en la Fase 4, que se remidió por completo así, la CPU queda con el gobernador *performance* y frecuencia fija de 3.2 GHz en los núcleos delegados, y la GPU con la gestión automática del controlador. El trabajo se organiza en cuatro fases encadenadas (Figura 2.1): la Fase 1 produce el conjunto de datos etiquetado; la Fase 2 entrena y valida los clasificadores, los exporta a ONNX (un formato abierto para modelos entrenados, ejecutable desde C++) y deriva las tablas de política; la Fase 3 implementa el agente; la Fase 4 lo evalúa. Los esquemas y cuadros de verificación complementarios se reúnen en el Anexo A.

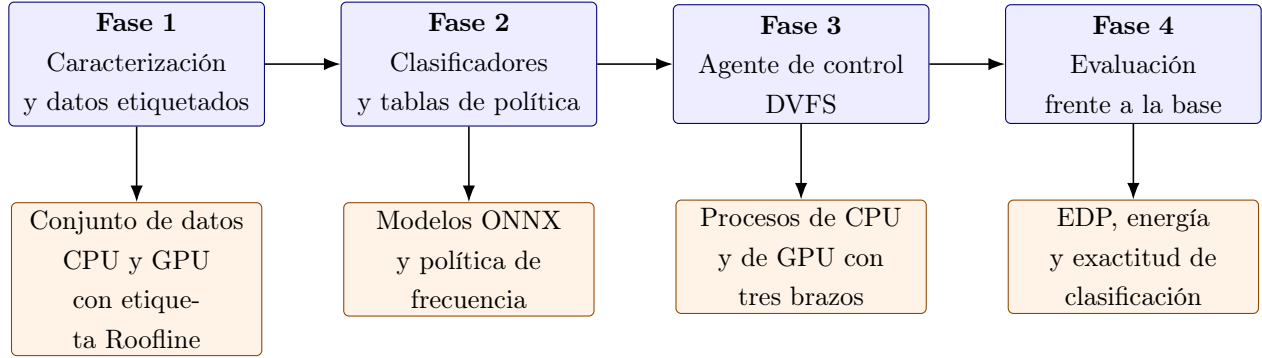


Figura 2.1: Fases de la metodología y producto de cada una. Fuente: elaboración propia.

2.1.1. Población, muestra y criterios de inclusión

La población son las aplicaciones para sistemas heterogéneos CPU–GPU; se estudió una muestra intencional de cargas reproducibles con regímenes contrastantes. Cada carga debía verificar su resultado, exponer trabajo de punto flotante medible por el instrumento y admitir una etiqueta de intensidad operacional. La unidad de diversidad y de validación fue la familia algorítmica: variantes de tamaño del mismo algoritmo no se trataron como familias independientes. El catálogo y su cribado se detallan en el Anexo A.

2.1.2. Plataforma experimental

Todos los experimentos se ejecutan en un único nodo, con asignación exclusiva del gestor de recursos para que ninguna carga ajena comparta el procesador durante las mediciones (Tabla 2.1). La plataforma se eligió porque expone la energía por interfaz interna sin privilegios administrativos (RAPL) y un acelerador instrumentable con NVML [33]. Las actuaciones de frecuencia de CPU se restringen a los procesadores lógicos delegados, los núcleos 0 a 5 de un zócalo y sus hilos hermanos, porque el dominio de frecuencia es el núcleo físico: en `intel_pstate`, la solicitud de un hilo hermano puede elevar el reloj efectivo del núcleo por encima del límite escrito en el otro [34], de modo que escribir un solo hilo no fija el reloj. Su efecto sobre el rendimiento se comprueba en el Anexo B, sección B.1.4.

Tabla 2.1: Plataforma experimental.

Componente	Descripción
CPU	2 × Intel Xeon Gold 5315Y (Ice Lake-SP), 8 núcleos físicos por zócalo, 2 hilos por núcleo, 2 nodos NUMA
Frecuencia	<code>intel_pstate</code> , 0.8 a 3.2 GHz (3.6 GHz con <i>turbo</i> activo), dominio por núcleo físico
Energía de CPU	Intel RAPL, dominios de paquete y de DRAM por zócalo
GPU	NVIDIA A100 PCIe 40 GB, telemetría por NVML
Ejecución	Nodo en asignación exclusiva (Slurm)

2.1.3. Instrumento de medición

El instrumento propio tiene dos capas (Figura 2.2). La capa inferior, en C++, lanza la carga como proceso hijo detenido antes de su primera instrucción, abre los contadores de hardware sobre ese proceso con herencia y solo entonces lo reanuda, de modo que ninguna instrucción queda fuera de la observación y la medida abarca todo el árbol de procesos e hilos de la carga. Un hilo productor lee los contadores a intervalos regulares y deposita las muestras en una estructura circular de un productor y un consumidor; un hilo consumidor, en otro procesador lógico, las persiste (Figura A.1, anexo A). La capa superior, en Python, ejecuta la lógica experimental: interpreta el manifiesto de la campaña, verifica la plataforma, calibra, planifica cada combinación, aplica el estado de frecuencia, valida cada corrida y construye el conjunto de datos.

La Tabla A.3 resume las señales. Los contadores de CPU se limitan a nueve eventos simultáneos, dimensionados contra el presupuesto de contadores físicos del nodo, verificado empíricamente, para evitar la multiplexación y su extrapolación. Una **ventana de muestreo** es el intervalo entre dos lecturas consecutivas; su contenido es la diferencia de las lecturas acumuladas. El tráfico a DRAM se mide con los contadores del controlador de memoria (*uncore*), cuyo intervalo práctico es de 10 ms o más, ya que el manual de perf advierte de overhead crítico y desestabilidad sub-10 ms [35]. En GPU, el recolector solicita lecturas NVML cada 5 ms; sin embargo, la actualización física de los sensores del controlador tiene una cadencia efectiva aproximada de 100–120 ms. Por ello, las muestras NVML se usan como telemetría agregada y no como ventanas GPU con resolución de 5 ms.

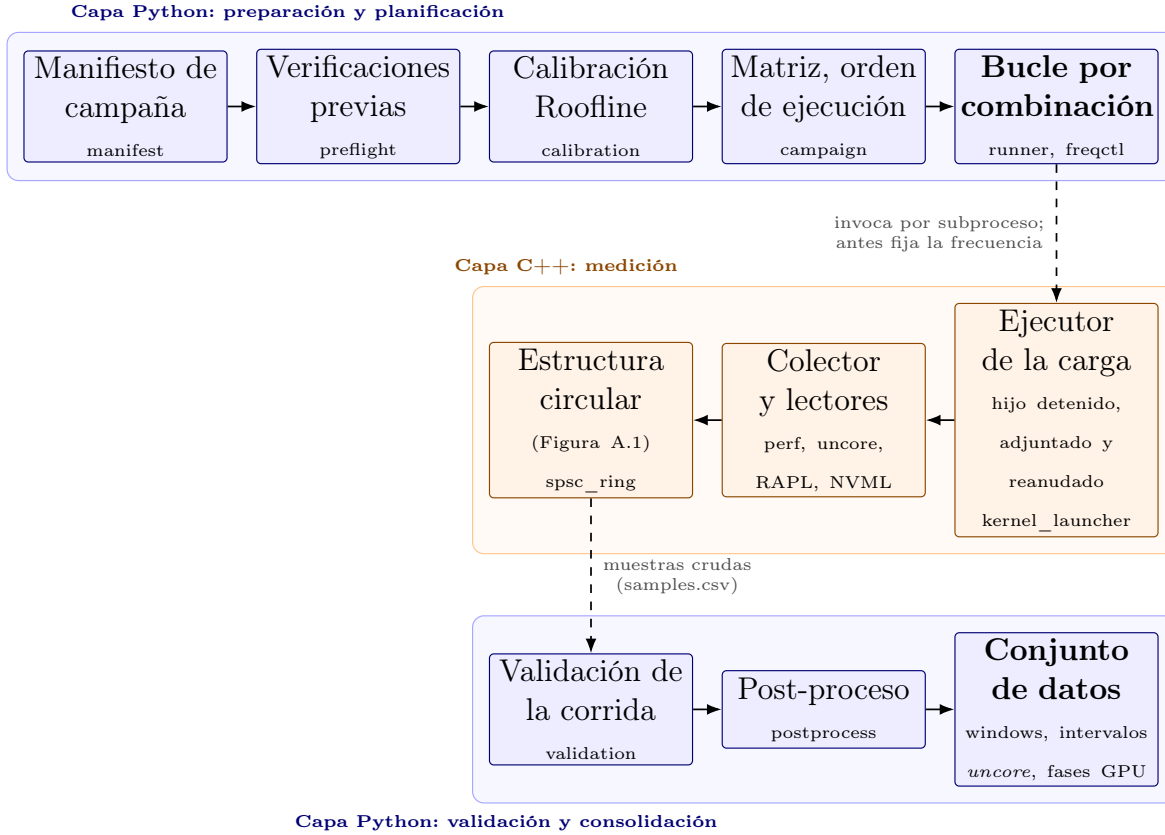


Figura 2.2: Recorrido de una campaña a través de las dos capas del instrumento. La capa Python prepara la campaña (banda superior), invoca por subprocesso el ejecutor de la capa C++ una vez por combinación de carga y frecuencia (banda intermedia), y consolida las muestras crudas que este produce en el conjunto de datos (banda inferior). La capa C++ concentra la medición sensible a la latencia; la capa Python, la lógica experimental, donde priman la claridad y la trazabilidad. Fuente: elaboración propia.

2.1.4. Modelo Roofline y etiquetado

La etiqueta de cada observación (*compute-bound* o *memory-bound*) resulta de comparar su intensidad operacional I con el punto de inflexión del modelo Roofline (ecuación (1.3)), calibrado en la plataforma por precisión aritmética y, en las campañas con barrido, por nivel de frecuencia. Los techos se calibran al inicio de cada campaña, con la misma asignación de núcleos que las corridas, mediante microbenchmarks de ancho de banda y de cómputo, y se contrastan con una referencia declarada. La etiqueta se deriva sin juicio manual, y la expectativa cualitativa de régimen que declara cada carga se usa solo como contraste.

Los dos dispositivos obtienen I de forma distinta (Figura 2.3). En CPU, los FLOPs se miden por ventana con los sub-eventos de `FP_ARITH_INST_RETIRED` [36] ponderados por el ancho vectorial (ecuación (A.1)), y los bytes con los comandos CAS del controlador de

memoria; I se calcula por intervalo, en vivo. En las campañas de GPU empleadas en este trabajo, NVML no proporciona una medición de FLOPs ni de bytes por ventana o por lanzamiento. Por ello, la intensidad operacional se caracteriza fuera de línea con Nsight Compute [37] (ecuación (A.2)) y se asocia a cada corrida. Una etiqueta con granularidad de fase requeriría combinar el perfilado de contadores con una traza temporal de lanzamientos mediante CUPTI [38], mecanismo que no forma parte de las observaciones aceptadas para este conjunto. Para un kernel determinista, con tamaño, precisión, entradas y configuración de lanzamiento fijos, esta caracterización se trata como una propiedad operacional estable tras verificar la convergencia entre los dos puntos de mayor trabajo, cuyo cambio relativo debe ser inferior al 1 %. Esta condición no afirma que el tráfico observado sea invariante ante cualquier configuración de ejecución. La caracterización distingue la precisión efectivamente ejercitada y excluye las cargas de aritmética entera o de precisión mixta no resoluble.

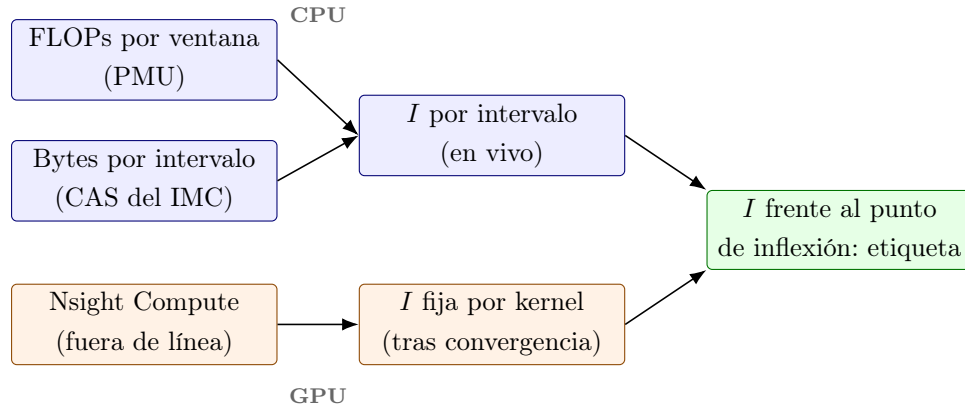


Figura 2.3: Obtención de la etiqueta Roofline. En CPU la intensidad se mide en vivo por intervalo del controlador de memoria; en GPU se caracteriza una vez por kernel, fuera de línea. Fuente: elaboración propia.

2.2. Fase 1: recolección y caracterización

2.2.1. Verificación previa y catálogo

Antes de medir, una batería de verificaciones automáticas, parte del comando que lanza la campaña, confirma las capacidades del entorno, la integridad del catálogo y la disponibilidad de la instrumentación (Tabla A.1, anexo A). Las verificaciones bloqueantes detienen la campaña antes de tocar el sistema: se prefiere no producir datos a producir datos cuya validez no puede sostenerse. El catálogo definitivo es una lista versionada y congelada de familias elegibles, tras un cribado con una rejilla de frecuencia reducida de tres niveles que descarta candidatos, comprueba la utilidad de su señal y estima su cobertura del plano Roofline. La

Tabla A.2 identifica las familias, las variantes representativas, la suite u origen y la referencia correspondiente. Cada clase debe quedar representada por al menos cinco o seis familias distintas por dispositivo. Con esto en mente, el conjunto de CPU reúne 30 familias, y el de GPU 16.

2.2.2. Matriz experimental y control de sesgos

La matriz resulta de combinar las cargas del catálogo, los estados de frecuencia (Tabla 2.2) y las repeticiones independientes del binario. Los niveles se verifican contra los relojes soportados por el nodo. En CPU, fijar un nivel requiere acotar el rango por núcleo y desactivar el *turbo* global para evitar ajustes autónomos del gestor de energía. Cada ventana se clasifica como válida, no confiable, no verificable (margen tras transición) o no aplicable (nivel nativo).

Tabla 2.2: Rejilla fina de frecuencia de la campaña definitiva, con pasos uniformes de 300 MHz en CPU y de 150 MHz en GPU.

Dispositivo	Niveles de reloj en MHz
CPU	REF, F0 (3200), F1 (2900), F2 (2600), F3 (2300), F4 (2000), F5 (1700), F6 (1400), F7 (1100) y F8 (800).
GPU	REF, F0 (1410), F1 (1260), F2 (1110), F3 (960), F4 (810), F5 (660), F6 (510), F7 (360) y F8 (210).

Para mitigar la latencia de asentamiento del reloj, la cual fue medida bajo carga en 10–11 ms para CPU y 70–80 ms para GPU, se verificó cada nivel dentro de una tolerancia de $\pm 5\%$ y se aplicó un margen de guarda de 0.15 s al inicio y final de la región medible.

Las medidas contra el sesgo son cinco. Cada combinación se repite tres veces (validado empíricamente como suficiente). El orden de ejecución se aleatoriza con una semilla registrada en el manifiesto [39]. Se incluyen repeticiones de una carga de referencia, cuya dispersión relativa señala inestabilidad del entorno. El eje de frecuencia de un dispositivo se mide con el otro dispositivo en su *governor* nativo, porque fijar la CPU en su nivel mínimo mientras se barre la GPU alarga las cargas de GPU y contamina la comparación. Y toda campaña reanudable conserva una huella del protocolo, de modo que dos manifiestos con protocolos distintos no se mezclan.

2.2.3. Calidad de los datos y granularidad

Ninguna observación se descarta en silencio. Cada ventana conserva una anotación de calidad (válida, sin diferencia previa, excluida por calentamiento, contadores degradados, intensidad indefinida, energía inválida o sin lectura de frecuencia) y el filtrado es una decisión del análisis. El transitorio de arranque de cada carga se identifica en dos etapas sobre una señal representativa del estado de carga (IPC en CPU, utilización en GPU): el primer instante desde el cual el coeficiente de variación se mantiene en 5 % o menos en dos ventanas móviles consecutivas y, como respaldo, un método de detección de puntos de cambio. Ese intervalo solo actúa como filtro posterior y nunca condiciona la adquisición. Cada corrida completa se valida además por integridad global: sin pérdidas en la estructura de muestreo, suma de verificación correcta y criterio de éxito propio cumplido. Los parámetros de la detección del transitorio están en el anexo (sección A.1.2).

La unidad de observación del entrenamiento se declara por dispositivo (Tabla A.4), porque en ambos casos la ventana del instrumento no coincide con la unidad que posee una etiqueta con respaldo físico. En CPU es un **intervalo del controlador de memoria**, con las variables recalculadas sobre los deltas agregados del intervalo y no promediando cocientes. En GPU es una **corrida completa**, con agregados robustos (mediana, dispersión) de las muestras válidas posteriores al calentamiento. Las lecturas repetidas de una misma corrida no se tratan como ejemplos independientes.

2.2.4. Reproducibilidad

Cada campaña queda definida por un manifiesto declarativo bajo control de versiones (cargas, estados de frecuencia, repeticiones, período de muestreo, asignación de núcleos, semilla y referencias de calibración). Junto con los datos se persisten el perfil de capacidades del nodo, la calibración y los metadatos de cada corrida. Una campaña interrumpida se reanuda saltando las corridas ya aceptadas, protegida por la huella del protocolo.

2.3. Fase 2: modelos de clasificación y política de frecuencia

2.3.1. Formulación, variables y validación

Se formula una clasificación binaria supervisada por dispositivo; la salida es la probabilidad de la clase *memory-bound*. En CPU, las seis variables son cinco tasas derivadas de los

contadores del intervalo (IPC, MPKI, tasa de fallos de caché, fracción de ciclos detenidos por memoria e instrucciones por segundo) y la frecuencia observada, conservada como covariable porque la misma carga puede cambiar de posición respecto del punto de inflexión cuando cambia el techo de cómputo. Antes de entrenar, se revisan las correlaciones entre las columnas candidatas. Los pares con correlación absoluta superior a 0.85 (común en la literatura), y la colinealidad restante, evaluada mediante el factor de inflación de la varianza (con alerta a partir de 10), orientan el descarte de variables redundantes. El conjunto final se registra en un contrato versionado por dispositivo (anexo, sección A.3.1).

La validación es *leave-one-familia-out* (LOFO): en cada pliegue se retira por completo una familia, que no participa en el ajuste, la selección de representación ni la elección del umbral (Figura 2.4) [40, 41]. Las celdas de familia y clase se limitan a 1000 intervalos con una semilla fija, y cada observación se pondera de forma inversa al tamaño de su celda (peso medio uno), de modo que ninguna celda domina por volumen. La métrica principal es la exactitud balanceada por celda de familia y clase [42],

$$EB = \frac{1}{|\mathcal{C}|} \sum_{(f,c) \in \mathcal{C}} \frac{a_{fc}}{n_{fc}}, \quad (2.1)$$

donde $\mathcal{C}$ es el conjunto de celdas observadas (cada combinación de familia f y clase c), $|\mathcal{C}|$ es el número de celdas, a_{fc} son los aciertos en cada celda y n_{fc} las observaciones totales de esa celda. Está definida aunque una familia tenga una sola clase y no depende de la prevalencia. Como complemento se reporta la puntuación F_1 macro agrupada (promedio de la F_1 de cada clase sobre todos los intervalos), que no debe confundirse con el nivel de frecuencia $F1$. La incertidumbre se reporta por dos vías: cinco semillas de submuestreo, y remuestreo con reemplazo de las familias (2000 réplicas) [43] para los intervalos de confianza del 95 % y las comparaciones pareadas.

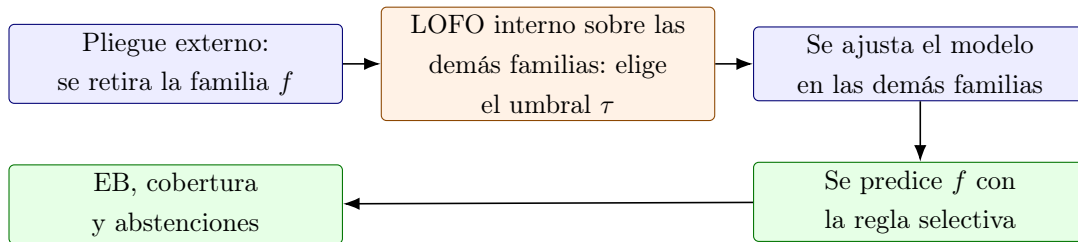


Figura 2.4: Validación anidada LOFO. La familia retirada no interviene en ninguna decisión de ajuste. Fuente: elaboración propia.

2.3.2. Decisión selectiva y calibración

Con p_i la probabilidad de *memory-bound* y $q_i = \max(p_i, 1 - p_i)$ la confianza, la regla selectiva entrega la clase solo si $q_i \geq \tau$ y en caso contrario emite *revisar*:

$$\hat{y}_i = \begin{cases} \text{memory-bound}, & p_i \geq 0,5 \wedge q_i \geq \tau, \\ \text{compute-bound}, & p_i < 0,5 \wedge q_i \geq \tau, \\ \text{revisar}, & q_i < \tau. \end{cases} \quad (2.2)$$

El umbral τ se elige sobre una rejilla mediante un LOFO interno, maximizando la exactitud balanceada por celda calculada únicamente sobre las observaciones aceptadas, con una cobertura media por celda mínima de 0.60. Las salidas *revisar* se consideran abstenciones y se excluyen del cálculo de las métricas de clasificación sobre las predicciones aceptadas. Su efecto se cuantifica mediante la cobertura, definida como la proporción del total de observaciones para las cuales el modelo emite una clase. La confianza se evalúa además mediante el error de calibración esperado, que divide los valores de confianza en diez rangos de igual amplitud y compara, en cada rango, la confianza media con la proporción real de aciertos,

$$\text{ECE} = \sum_{b=1}^{10} \frac{n_b}{N} |\text{conf}_b - \text{acc}_b|, \quad (2.3)$$

donde N es el número total de observaciones evaluadas, n_b es el número de observaciones del rango b , conf_b es la confianza media de ese rango y acc_b es su proporción de aciertos.

2.3.3. Comparación y selección de modelos

Se compararon bajo el mismo protocolo una regla mayoritaria, un árbol de profundidad uno, regresión logística, *random forest*, *extra trees* y XGBoost (seis modelos en CPU; en GPU se añadió un árbol de profundidad seis, siete en total), con hiperparámetros fijos [44]. La búsqueda de hiperparámetros con Optuna se evaluó anidada en cada pliegue externo y se descartó: con un número limitado de familias, el criterio interno que maximiza no discrimina de forma confiable la configuración que generaliza mejor [45]. Para explorar si el desempeño observado estaba limitado principalmente por la capacidad del modelo o por la información contenida en las variables se aplicaron cuatro diagnósticos complementarios. Entre ellos se evaluó un clasificador de vecinos más cercanos sin ajuste, utilizado para examinar la separabilidad local de las clases en el espacio de variables (anexo, sección A.3.2). La selección del modelo final se realizó después de comparar el desempeño bajo el protocolo LOFO y de verificar la invarianza de las variables frente a la acción del agente. Los modelos y umbrales

seleccionados se reportan en el capítulo de resultados.

2.3.4. Invarianza de las entradas frente a la acción del agente

La comparación por exactitud identifica candidatos que generalizan fuera de familia, pero un clasificador integrado en un controlador debe usar entradas que no cambien como consecuencia de su propia acción. En GPU se audita esa condición con dos pruebas sobre los datos ya disponibles. La prueba contrafactual sustituye el valor de una entrada potencialmente afectada por la actuación y cuantifica los cambios de predicción. La prueba de exclusión repite la validación LOFO sin dicha entrada. Toda variable que no supera ambas pruebas se excluye del contrato operativo.

Los modelos se vuelven a evaluar con el contrato de entradas resultante. La selección exige exactitud balanceada, calibración y cobertura suficientes bajo LOFO; la latencia de inferencia se informa como propiedad de implementación, mientras que la latencia completa desde la detección hasta la actuación se mide separadamente durante la evaluación del agente. En CPU se aplica la misma auditoría a la frecuencia observada, mediante una sustitución contrafactual coherente con el cambio de frecuencia que puede ejecutar el agente. Los resultados de ambas auditorías se presentan en las secciones 3.2.3 y 3.1.5.

2.3.5. Derivación de la política de frecuencia

La política de frecuencia se deriva de la campaña de la Fase 1, por separado del clasificador. Cada nivel se compara con la base sobre el mismo kernel y la ganancia agregada se estima por *bootstrap* de kernels y un nivel solo se adopta si su ganancia es significativa y supera un efecto mínimo relevante de 1 %. La regla de elección se valida dejando una familia fuera; cualquier refinamiento se congela antes de probarse en familias externas. En GPU, los kernels próximos al punto de inflexión se marcan como ambiguos sin excluirlos. El emparejamiento, las pruebas y los criterios de refinamiento se especifican en el Anexo A.

2.4. Fase 3: agente de control DVFS

La Fase 3 no vuelve a entrenar clasificadores ni vuelve a escoger frecuencias. Recibe como entradas congeladas tres artefactos de la Fase 2: el modelo de cada dispositivo en formato ONNX, su umbral de abstención y la tabla de política. El modelo y el umbral proceden de la validación LOFO y de la auditoría de invarianza. La tabla de política procede, de forma independiente, de la comparación pareada de EDP entre niveles de frecuencia. Esta separación

evita que el desempeño del clasificador se interprete como evidencia de que una frecuencia determinada mejora el EDP, o que la política de frecuencia determine la selección del modelo. Los modelos se entrenan en Python, pero el agente decide en C++ cada pocas decenas de milisegundos, y un intérprete de Python en ese camino añadiría latencia y consumo. ONNX (*Open Neural Network Exchange*) resuelve esto: representa el modelo ya entrenado de forma independiente de la biblioteca que lo produjo y lo ejecuta ONNX Runtime desde C++, sin reentrenar ni reimplementar el modelo. El *random forest* de GPU se convirtió con `skl2onnx` y el XGBoost de CPU con el convertidor de `onnxmltools` registrado en `skl2onnx`. Cada exportación se verifica fila a fila contra el modelo original sobre todas las observaciones elegibles (intervalos en CPU, corridas en GPU), y se rechaza si alguna probabilidad difiere en más de 10^{-4} o si alguna etiqueta cambia. La Fase 3 implementa detección, decisión, actuación, coordinación y restauración con esos artefactos sin modificarlos; la Fase 4 evalúa el conjunto ya congelado.

2.4.1. Arquitectura

El agente son dos procesos independientes en C++, uno por dispositivo, coordinados por una señal de un solo bit (Figura 2.5). Se separaron porque los bucles operan a escalas distintas (ventanas de milisegundos en CPU, fases de segundos en GPU), porque un fallo en uno no impide que el otro restaure el dispositivo que le corresponde, y porque los experimentos de un solo dispositivo no cargan la biblioteca de gestión del otro. Ambos leen los modelos, umbrales y tabla de política congelados al inicio de la ejecución; el lanzador los traduce a las banderas de cada proceso.

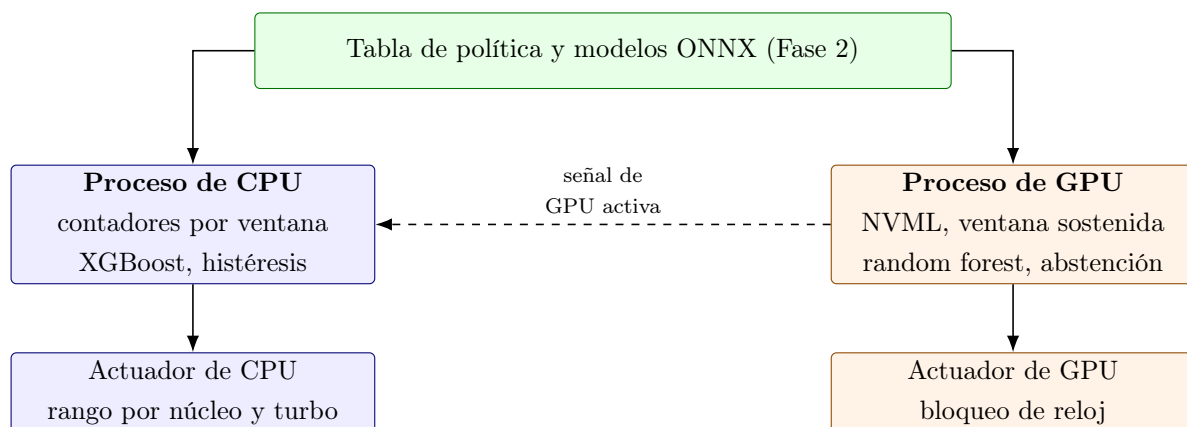


Figura 2.5: Arquitectura del agente: dos procesos, una tabla de política compartida y una señal de coordinación. Fuente: elaboración propia.

Cada proceso admite los tres brazos de la Tabla 2.3. El brazo *base* es la referencia contra

la que se compara todo, y el brazo *sombra* mide la sobrecarga del agente sin su efecto sobre el hardware.

Tabla 2.3: Brazos del agente.

Brazo	Agente	Comportamiento
Base	No corre	En CPU, gobernador <i>performance</i> con turbo desactivado y frecuencia fija de 3.2 GHz; en GPU, gestión automática del controlador
Sombra	Muestrea, clasifica y decide	Registra cada decisión y nunca escribe sobre el hardware
Activo	Muestrea, clasifica, decide y escribe	Aplica la frecuencia decidida y restaura el estado al terminar

2.4.2. Detección de fase, decisión y parámetros

El agente de GPU delimita fases con la utilización NVML, reúne 3 s de actividad y decide una vez por fase. Si se abstiene, libera el reloj. El agente de CPU decide por ventana y exige 50 propuestas consecutivas antes de cambiar de nivel. Los umbrales y periodos se reúnen en la Tabla 2.4; la máquina de decisión se verificó por separado de la biblioteca de inferencia.

Tabla 2.4: Parámetros del agente.

Proceso Parámetro		Valor y razón
GPU	Umbral de actividad	5 % de utilización
GPU	Ventana sostenida	3 s; una decisión por fase
GPU	Cadencia de muestreo	50 ms
GPU	Umbral de abstención	0.90 (cobertura media por celda ≥ 0.60); seleccionado en la Fase 2 (sección 3.2.3)
GPU	Permanencia mínima entre cambios	3.7 s, 10 veces la latencia de un cambio completo de reloj (hasta 0.37 s) medida en la Fase 3 (sección 3.3.2)
CPU	Umbral de decisión	0.85; congelado en la Fase 2 (sección 3.1.4)
CPU	Histéresis	50 ventanas consecutivas
CPU	Piso con GPU activa	3.2 GHz; verificado en la Fase 3 (sección 3.3.2)

2.4.3. Política, actuación y coordinación

La Tabla 2.5 separa la política operativa derivada en la Fase 2 de las variantes experimentales de CPU. La salida operativa de la Fase 2 para CPU es *no actuar* en ambas clases, por lo que no se presenta F0 ni F1 como recomendación de despliegue. Para comprobar que esa decisión no oculta un efecto al cerrar el lazo de control, la Fase 4 incorpora un brazo de CPU experimental que fija F0 en cómputo y F1 en memoria, y una variante *activo-F0* que mantiene F0 en ambas. Estas variantes no actualizan la tabla derivada ni alteran el modelo; se comparan contra el *governor* nativo.

Tabla 2.5: Acciones del agente. Las filas de GPU provienen de la política derivada en la Fase 2; las filas de CPU son variantes experimentales evaluadas en la Fase 4.

Dispositivo	Clase	Acción
CPU	<i>compute_bound</i>	Fija F0 (3.2 GHz, turbo desactivado); experimental
CPU	<i>memory_bound</i>	Baja a F1 (2.9 GHz) con confianza mínima de 0.85; experimental
GPU	<i>memory_bound</i>	Fija el reloj en 1260 MHz (F1)
GPU	<i>compute_bound</i>	No actúa: libera el reloj al <i>governor</i> nativo
GPU	<i>revisar</i> (abstención)	No actúa: libera el reloj al <i>governor</i> nativo

El actuador de CPU reutiliza las reglas de restricción de rango por núcleo del módulo de la Fase 1 (delegados y hermanos de *SMT*), desactiva el turbo global con el envoltorio administrativo del nodo y escribe los núcleos en paralelo. Registra el estado previo, verifica cada escritura por relectura, falla cerrado ante cualquier inconsistencia y restaura el estado inicial al terminar, incluso ante **SIGTERM** o **SIGINT**; su equivalencia con el módulo de la Fase 1 se comprobó con una prueba de paridad. El actuador de GPU fija y libera el reloj con la herramienta de administración del controlador, con la misma disciplina de restauración.

La coordinación entre procesos es una regla de protección distinta de la tabla de política. Cuando la señal de actividad GPU está presente, impone temporalmente un piso de CPU de 3.2 GHz y no espera la histéresis. No reclasifica la fase de CPU ni convierte F0 en una acción recomendada por la Fase 2. Se adopta para evitar que las señales de una CPU que espera a la GPU desencadenen una reducción de frecuencia que prolongue la fase GPU. La Fase 3 verifica que el piso se aplique y se restaure; su efecto sobre EDP se mide por separado en la Fase 4 con el brazo sin piso.

2.4.4. Aplicaciones compuestas y verificación previa

Las aplicaciones compuestas registran con el reloj monotónico las fronteras reales de fase para puntuar al agente (Tabla 2.6). A usa familias vistas y B familias inéditas; una tercera aplicación alterna CPU y GPU. Cada fase supera los 3 s de observación y se separa de la siguiente por 1 s de reposo. Los kernels inéditos de GPU usan parámetros ampliados del mismo binario verificado para sostener la actividad.

Tabla 2.6: Aplicaciones compuestas con verdad de fase.

Alcance	Aplicación	Fases (clase)
CPU	A (vistos)	dgemm_n2048 (<i>compute</i>), npb_cg (<i>memory</i>)
CPU	B (inéditos)	cpu_xsbench_omp (<i>memory</i>), cpu_rsbench_omp (<i>compute</i>)
GPU	A (vistos)	gpu_cutlass_simt_dgemm_n4096 (<i>compute</i>), gpu_rajaperf_stream_triad (<i>memory</i>)
GPU	B (inéditos)	BabelStream con -numtimes 1000 (<i>memory</i>), rodinia_lavamd con -boxes1d 100 (<i>compute</i>)

Antes de la evaluación se verificó el agente por dispositivo y con ambos a la vez: latencia de conmutación, sobrecarga, restauración bajo terminación anómala inducida y cumplimiento del piso de coordinación (sección 3.3).

2.5. Fase 4: validación experimental

2.5.1. Diseño de la matriz

La matriz inicial compara el agente contra el *governor* nativo en tres ejes: el alcance (solo CPU, solo GPU o ambos a la vez sobre una aplicación que alterna fases), el tipo de kernel (vistos frente a inéditos, que mide la brecha de generalización con las mismas celdas) y el brazo (Tabla 2.7). El orden de las celdas se aleatoriza con semilla fija y cada una ejecuta la aplicación dos ciclos completos. El brazo *sombra* aísla la sobrecarga del agente; *activo-F0* y *activo sin piso* se incluyen como brazos experimentales porque la política de CPU resultó *no actuar* en la Fase 2 y se quiere medir si esa conclusión se sostiene con el agente en el lazo. La matriz inicial usó cinco repeticiones en el alcance conjunto y tres en los demás casos. Con tres observaciones por brazo, el análisis es descriptivo mediante razones de medianas y valores individuales.

Tabla 2.7: Matriz experimental de la Fase 4.

Alcance	Brazos	Vistos Inéditos		Rep.
CPU	base, sombra, activo, activo-F0	A	B	3
GPU	base, sombra, activo	A	B	3
Conjunto	base, sombra, activo con piso, activo sin piso	A	B	3 (5 en la matriz inicial)

2.5.2. Escenarios adicionales

Además de la matriz inicial, que se reporta completa y es la referencia, se ejecutaron los protocolos complementarios de la Tabla 2.8; sus condiciones completas se conservan en la Tabla A.7 del Anexo A. Cada protocolo se fijó antes de sus mediciones y todos los ejecutados se reportan, cualquiera que sea su resultado. El escenario D emplea LAMMPS, un simulador paralelo de dinámica molecular [46]. El confirmatorio adicional emplea CloverLeaf, una miniaplicación hidrodinámica explícita [47]. El escenario E se definió con el criterio de selección de sus familias (ganancia de EDP con F1 de al menos 10 % en la Fase 2, más una variante con familias no vistas) y se verificó con una corrida previa de una repetición por brazo antes de la matriz completa.

En las etapas C y E la base se midió una segunda vez (*base sin turbo*), sin daemon y con el turbo de CPU verificado desactivado, como control de repetibilidad. Es el mismo estado que la base, con tres repeticiones para cada combinación de aplicación y alcance incluida, y conserva el reloj nativo de la GPU, por lo que no representa una frecuencia fija de GPU.

Tabla 2.8: Mapa de los escenarios de la Fase 4. El protocolo completo se conserva en el Anexo A.

Escenario	Contraste principal
Matriz inicial	Por dispositivo, familias vistas e inéditas y brazo
C	Repetición independiente de CPU y conjunto
E	GPU con fases largas de memoria, familias vistas (E-A) e inéditas (E-B)
base sin turbo	Segunda medición de la base, como control de repetibilidad
D	LAMMPS con cuatro entradas y fases naturales
F	Premedición diagnóstica de la solicitud de F1 fijo
Confirmatorio E-A	Cinco bloques con la base, sombra, agente y solicitud de F1 fijo
E-B ampliado	Repetición de E-B con ocho bloques y corrida de diagnóstico de la variabilidad de <i>myocyte</i> , con telemetría de GPU y CPU
CloverLeaf	Cinco bloques con la base, sombra y agente en aplicación continua

2.5.3. Medición y análisis

En todos los escenarios, cada celda registra duración, energía de CPU (RAPL de ambos paquetes) y energía de GPU (contador NVML) por separado. Así, el EDP del nodo, $EDP =$

$(E_{\text{CPU}} + E_{\text{GPU}})T$, se calcula y se desagrega después; también se calcula $E_{\text{GPU}}T$. La ventana energética abarca desde el arranque de la aplicación hasta su fin, con los procesos del agente ya activos, por lo que incluye su sobrecarga. Cada celda verifica la finalización de la aplicación y de los procesos del agente que correspondan, y restaura los estados de frecuencia que modificó.

En la matriz y en el escenario E, las aplicaciones compuestas registran las fronteras de fase conocidas y las decisiones de cada proceso (instante, clase, confianza y escritura), y el reloj de GPU se libera entre celdas. LAMMPS se ejecuta con fases naturales que no se registran. CloverLeaf se ejecuta como un proceso CUDA Fortran continuo con ocho CPU asignadas; no se le impone ni se le atribuye una frontera de fase. Para CloverLeaf se conserva además la salida numérica, muestras NVML cada 250 ms y el estado de reloj antes y después de cada celda.

Para cada combinación de alcance, kernels y brazo se calcula la mediana de duración, energías y EDP y su razón frente al brazo *base* (menor es mejor). En la matriz inicial conjunta, que tiene cinco observaciones no apareadas por brazo, el EDP se contrasta con Mann–Whitney bilateral [48] (con cinco contra cinco, el mínimo alcanzable es $p = 0,0079$). Los experimentos confirmatorios E-A y CloverLeaf se analizan por bloques pareados, con una prueba de signos exacta bilateral [48] sobre el EDP (con cinco bloques todos favorables, $p = 0,0625$, el mínimo posible). Para cada bloque se calcula la razón entre el brazo experimental y el brazo base en duración, energía y EDP, donde valores inferiores a uno indican una reducción respecto al caso base. Se reportan las razones individuales y su comportamiento agregado, preservando la variabilidad entre bloques.

La clasificación se puntúa contra las fronteras reales solo en las aplicaciones compuestas: cada decisión se atribuye a la fase que contiene su instante y se reportan correctas, erróneas y abstenciones por alcance, tipo de kernel y dispositivo, junto con la cobertura. La diferencia de exactitud entre A y B mide la generalización dentro de ese protocolo. En CPU las decisiones son por ventana, así que la exactitud se pondera por ventanas y no por fases. En LAMMPS y CloverLeaf se reportan las decisiones y la actuación observada, pero no una exactitud de clasificación, pues no existe una verdad de fase independiente con esa granularidad.

La Tabla A.6 resume las principales amenazas a la validez y la medida que la controla.

Capítulo 3

Resultados

Este capítulo presenta los resultados de las campañas de CPU y GPU, del agente de control y de su evaluación frente a la referencia en aplicaciones compuestas. Las tablas y figuras de sensibilidad, desglose y trazabilidad se incluyen en el Anexo B; el cuerpo principal reúne la evidencia necesaria para sustentar cada conclusión.

3.1. Resultados de CPU

La campaña CPU cubrió 49 configuraciones de kernel en el nivel nativo y nueve niveles fijos de frecuencia, con tres repeticiones por combinación. Sus 1470 corridas finalizaron aceptadas. Tras el reprocesamiento con los calentamientos calibrados, los transitorios fueron inferiores a 2% de la duración de corrida en la mayoría de los kernels y de 4.8% como máximo.

La Tabla B.1 resume la cobertura tras el filtrado de calidad: de 1 424 531 intervalos procesados, 1 309 755 (91.9%) cumplieron simultáneamente los criterios de telemetría y de frecuencia definidos en Metodología (incluye el desglose de rechazos). Los restantes se conservaron para auditoría, pero no recibieron una etiqueta de entrenamiento.

3.1.1. Composición de fase por kernel

Las familias de cargas, sus variantes representativas y la suite de procedencia se enumeran en la Tabla A.2 del Anexo A. La Figura 3.1 resume la composición por familia; el desglose de cada configuración de kernel se conserva en la Figura B.1 del Anexo B. El conjunto cubre ambos regímenes y conserva configuraciones próximas al *ridge*.

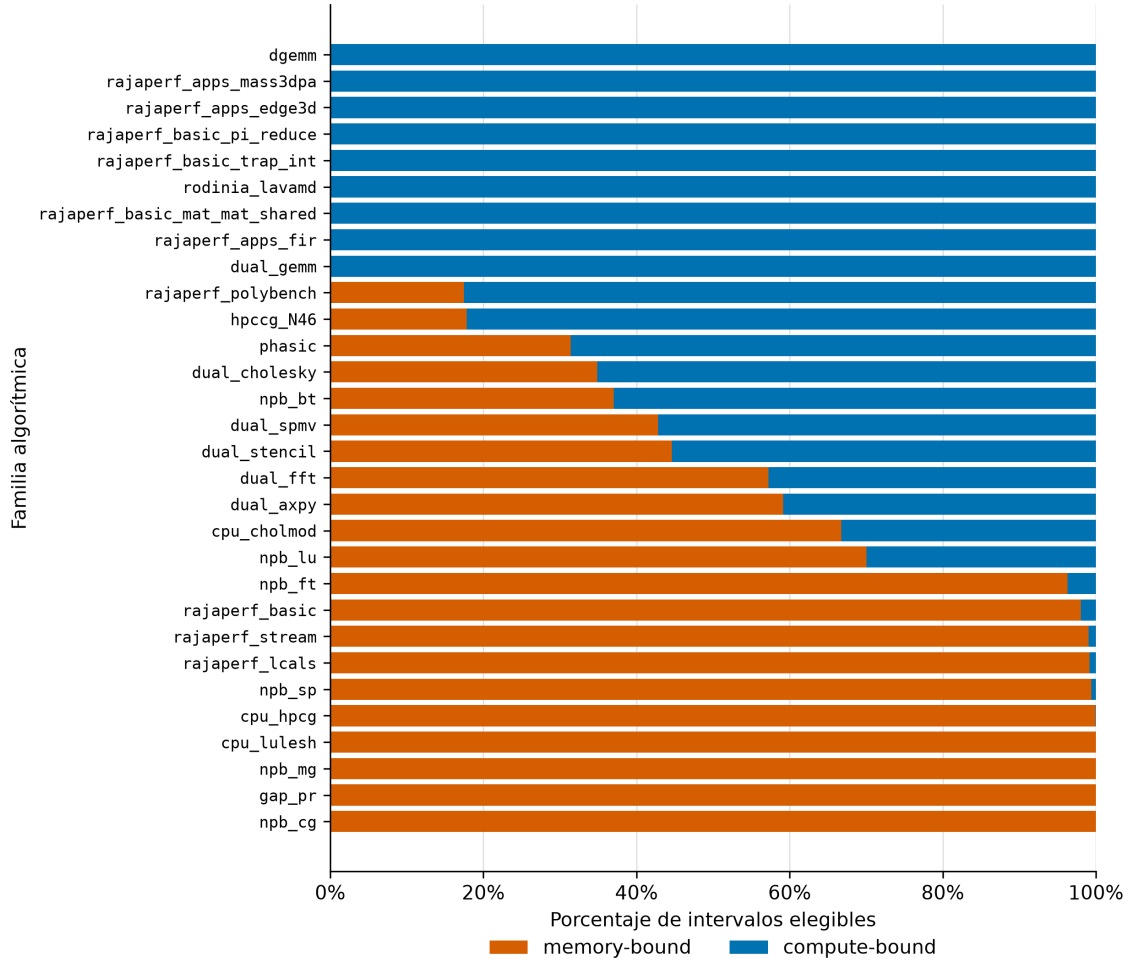


Figura 3.1: Porcentaje de intervalos elegibles clasificados *compute_bound* y *memory_bound* por familia algorítmica. Las variantes de tamaño se agregan dentro de cada familia. Fuente: elaboración propia.

La composición intermedia no se concentra en un solo algoritmo. *npb_bt*, *npb_lu*, *cpu_cholmod*, *hpccg_cpu_N46*, las variantes *phasic* y los tamaños dirigidos de FFT y Cholesky aportan intervalos de ambos regímenes. Estas configuraciones evitan que el conjunto quede limitado a ejemplos alejados del punto de inflexión.

Tres kernels de patrón de acceso irregular de Rodinia (*kmeans*, *srad* y *particlefilter*) se midieron por separado y resultaron casi puramente *memory-bound*; como no aportaban ejemplos *compute-bound* ni mixtos, quedaron fuera del conjunto de entrenamiento por no añadir más que redundancia *memory-bound*.

3.1.2. Variación de la composición con la frecuencia

La proporción global de intervalos *compute_bound* aumentó desde 29.0% en el nivel nativo hasta 46.4% en F8, el nivel fijo más bajo. La Figura 3.2 presenta esta variación. El resultado es consistente con un *ridge* que disminuye cuando baja el techo de cómputo, de modo que una parte de los intervalos cercanos a la frontera cambia de etiqueta. La figura agrega todas las configuraciones y debe leerse junto con la composición por kernel, porque las cargas no aportan el mismo número de intervalos.

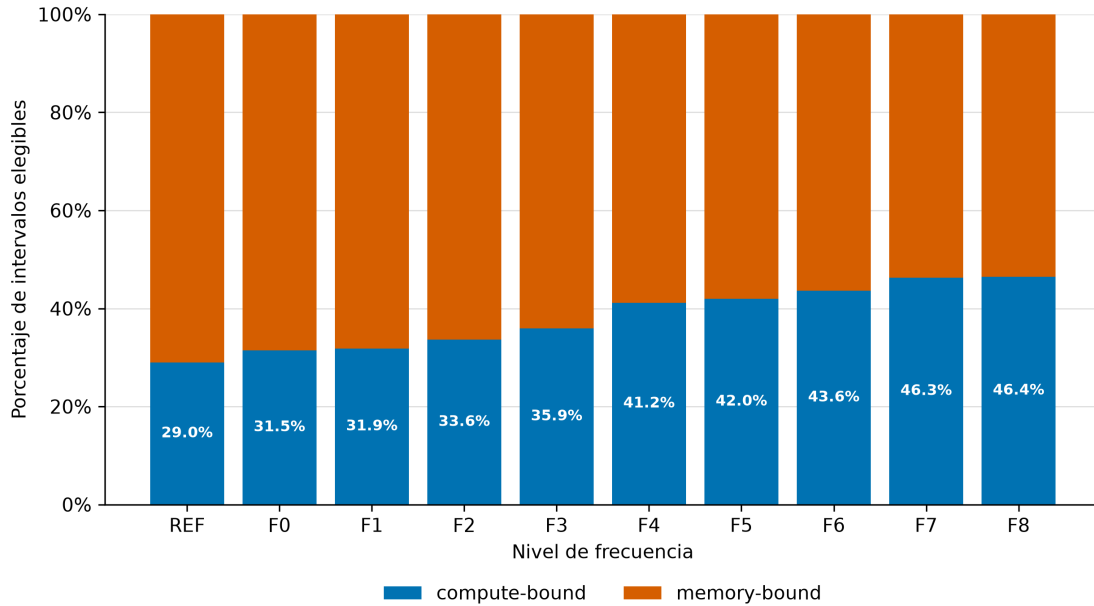


Figura 3.2: Porcentaje de intervalos elegibles clasificados *compute_bound* y *memory_bound* en cada nivel de frecuencia de la campaña CPU. Fuente: elaboración propia.

3.1.3. Techos Roofline y *ridge*

Los techos del modelo Roofline se calibraron de forma empírica sobre la propia plataforma (sección 1.1.2). El punto de inflexión desciende con el reloj, de 8.69 FLOP/byte en la referencia nativa a 2.92 FLOP/byte en F8 (Figura 3.3), porque baja el techo de cómputo mientras el ancho de banda sigue limitado por memoria.

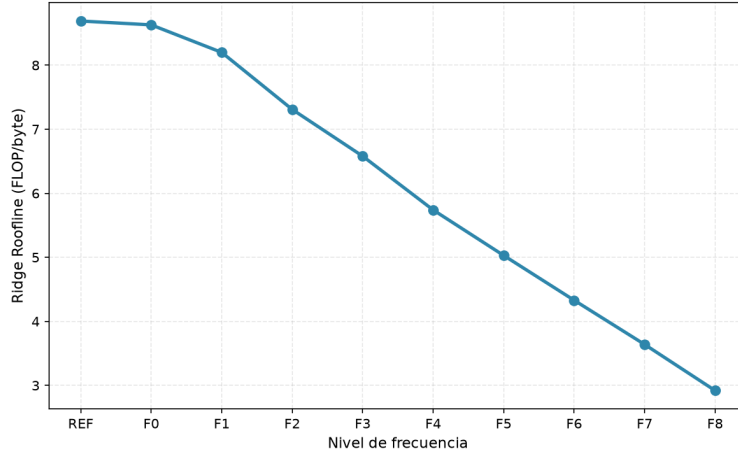


Figura 3.3: *Ridge* Roofline según el nivel de frecuencia CPU. El punto de inflexión desciende de 8.69 FLOP/byte en referencia nativa a 2.92 FLOP/byte en F8. Fuente: elaboración propia.

Validación de los techos. Para verificar la calibración con un mecanismo independiente se empleó Intel Advisor, que contabiliza FLOPs y bytes mediante instrumentación binaria y no mediante contadores de PMU [49]. El ancho de banda calibrado coincide con el de Advisor dentro de 12 a 15 % (58.4 a 58.8 GB/s frente a 64.0 a 66.8 GB/s) y el techo de cómputo (417.8 a 555 GFLOP/s) es consistente con la referencia de Advisor, que midió 577.7 GFLOP/s sobre los mismos seis núcleos. El punto de inflexión del nivel nativo queda entre 7.0 y 9.3 FLOP/byte, compatible con los 8.69 FLOP/byte usados en la campaña.

3.1.4. Clasificador de fase en CPU

Los resultados del clasificador siguen el protocolo LOFO y la exactitud balanceada por celda definidos en la sección 2.3.

Muestra de entrenamiento

El modelo CPU se ajustó con 42 458 intervalos de 30 familias (Tabla B.1). La sensibilidad al tamaño de la muestra mantuvo la exactitud entre 0.720 y 0.739, sin tendencia creciente (Tabla B.2); la unidad de generalización sigue siendo la familia, no cada intervalo.

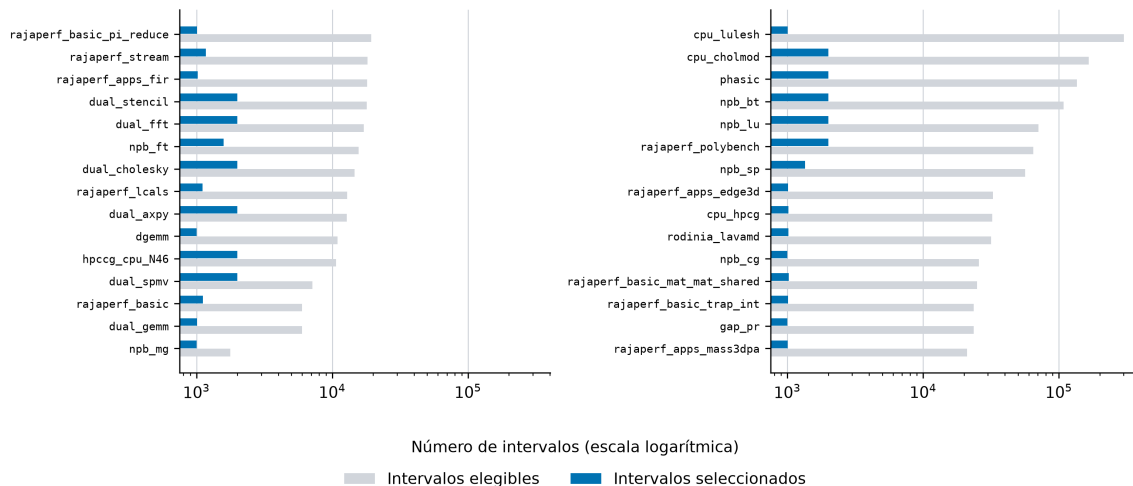


Figura 3.4: Intervalos elegibles y seleccionados por familia algorítmica. El eje logarítmico permite comparar familias cuyo volumen original difiere en varios órdenes de magnitud. Fuente: elaboración propia.

Variables de entrada

El candidato CPU usa las seis variables definidas en la sección 2.3. La auditoría de correlación (sección A.3.1) se calculó sobre las doce columnas candidatas, antes de descartar las seis redundantes (Figura 3.5): el par más alto es 0.89 (Pearson), entre MPKI y las referencias de caché por millar de instrucciones, dos normalizaciones distintas de la misma señal de fallos de caché, y de ese par se retuvo MPKI. Sobre el vector final de seis variables, ningún par supera ya la correlación de 0.85: la correlación absoluta más alta es 0.74 (de Spearman), negativa, entre IPC y la fracción de ciclos detenidos por memoria, y la siguiente es 0.72 entre IPC e IPS. El factor de inflación de la varianza máximo es 5.5 con la correlación de Pearson y 7.3 con la de rangos, ambos de IPC, por debajo del umbral de alerta de 10, de modo que no fue necesario retirar ninguna variable adicional.

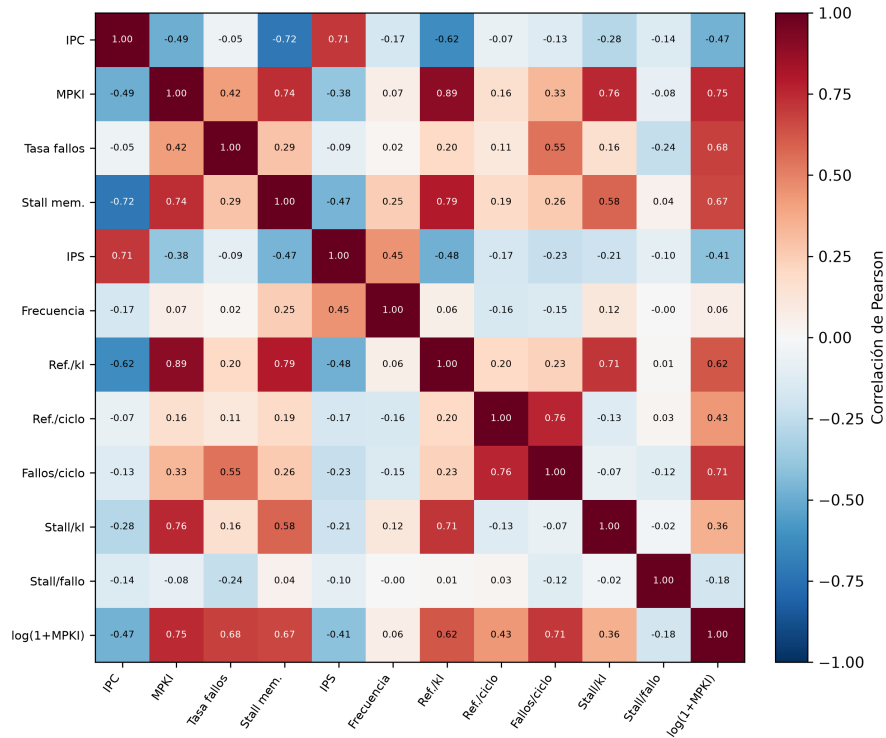


Figura 3.5: Correlación de Pearson entre las doce columnas candidatas, sobre los 1 309 755 intervalos elegibles, calculada antes de la selección del vector final de seis variables. Fuente: elaboración propia.

Ampliar el vector de seis a doce entradas redundantes cambió la exactitud balanceada de 0.728 a 0.724 (IC95 de la diferencia, $-0,026$ a $+0,011$; Tabla B.3). Por ello se conservaron seis variables. La redundancia perjudicó más a la regresión logística, sin ofrecer a XGBoost una mejora medible de inferencia o precisión.

Selección de modelo e hiperparámetros

La Figura 3.6 compara los candidatos con hiperparámetros fijos, cinco semillas y el mismo protocolo LOFO. XGBoost, Extra Trees y Random Forest quedan dentro del mismo intervalo de incertidumbre; la regresión logística rinde menos. La puntuación F_1 macro agrupada de los modelos de seis variables es de 0.205 para la regla mayoritaria, 0.713 para el árbol de profundidad 1, 0.765 para la regresión logística, 0.717 para Extra Trees, 0.763 para Random Forest y 0.760 para XGBoost. Se seleccionó XGBoost por su menor latencia entre los modelos de árboles. La latencia de una fila y un hilo, medida en el nodo con el proceso fijado a cuatro núcleos, es de 243 μ s de mediana (404 μ s en el percentil 99) para XGBoost, frente a 5.3 ms (5.5 ms) de los dos bosques, más de trece veces mayor; el árbol de profundidad 1 y la regresión logística toman 105 μ s (109) y 450 μ s (475).

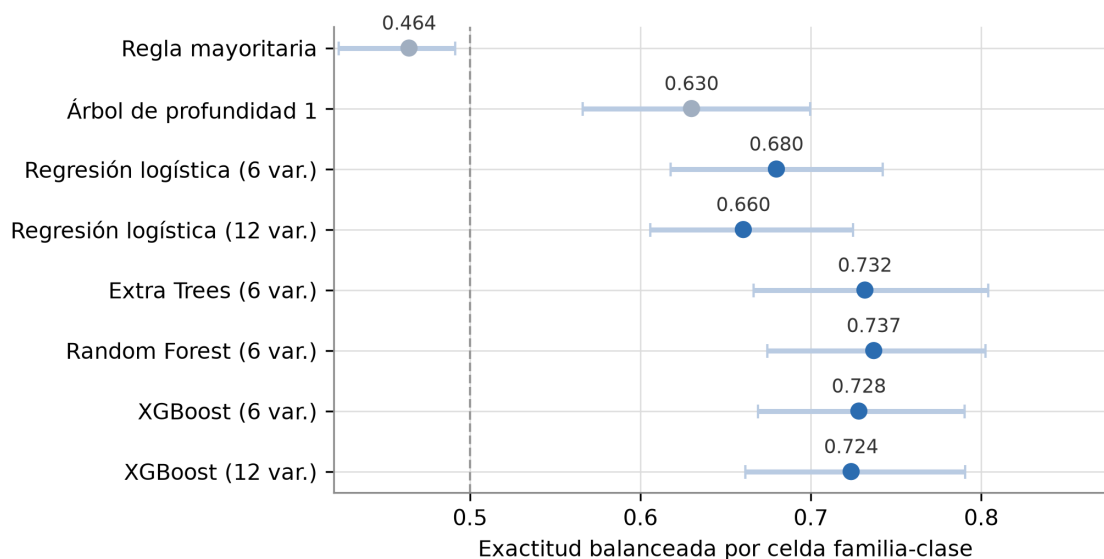


Figura 3.6: Exactitud balanceada por celda de familia y clase de cada modelo, con intervalo de confianza del 95 % obtenido por remuestreo de las 30 familias, con hiperparámetros fijos. Las variantes de doce variables se detallan en la Tabla B.3 del Anexo B. Los intervalos se solapan entre todos los modelos con capacidad suficiente. Fuente: elaboración propia.

Búsqueda de hiperparámetros. La búsqueda anidada con Optuna [29] no produjo una mejora operativamente útil. Para XGBoost, la diferencia pareada por familia frente a la configuración fija es de +0,019 (IC95 de $-0,007$ a $+0,050$, que incluye el cero; 14 de 30 familias mejoran y 8 empeoran). Para la regresión logística es distinguible de cero (IC95 de $+0,0004$ a $+0,0042$; 5 familias mejoran, 2 empeoran), pero de magnitud irrelevante ($+0,002$). El puntaje interno de cada pliegue varía poco (de 0.734 a 0.761) mientras el resultado externo va de 0.487 a 1.000, de modo que el criterio interno no distingue configuraciones buenas de malas para una familia nueva. Se conservó la configuración fija de XGBoost, la más simple de reproducir. Junto con la comparación de modelos y de vectores de entrada, esto indica que el desempeño no está limitado por la capacidad del modelo ni por sus hiperparámetros.

La configuración congelada es la de la Tabla 3.1.

Tabla 3.1: Configuración final del clasificador CPU.

Elemento	Valor
Modelo	XGBoost, 100 árboles, profundidad máxima 6, objetivo <code>binary:logistic</code>
Entradas	6 variables PMU definidas en la sección 2.3
Ponderación	Peso total igual por celda de familia y clase, ecuación (A.3), con <code>scale_pos_weight = 1</code>
Salida	Probabilidad de <i>memory-bound</i> ; clase con confianza $\geq 0,85$ y <i>revisar</i> por debajo
Semilla	20260918

Evaluación del modelo final

La métrica principal, la exactitud balanceada por celda, fue 0.728 sin abstención (IC95 de 0.669 a 0.790); la variación entre las 30 familias limita la distinción entre configuraciones cercanas. Esa cifra depende de la unidad de separación: mezclar intervalos al azar da 0.874 y dejar fuera un kernel, 0.733 (Tabla B.4 del Anexo B). Los valores de las demás métricas están en la Tabla B.5 del mismo anexo. Sobre los 1309755 intervalos elegibles, el modelo alcanzó una F1 macro agrupada de 0.760 y una exactitud global de 0.769; la sensibilidad fue 0.714 para *compute-bound* y 0.806 para *memory-bound*. La Figura 3.7 muestra las matrices de confusión.

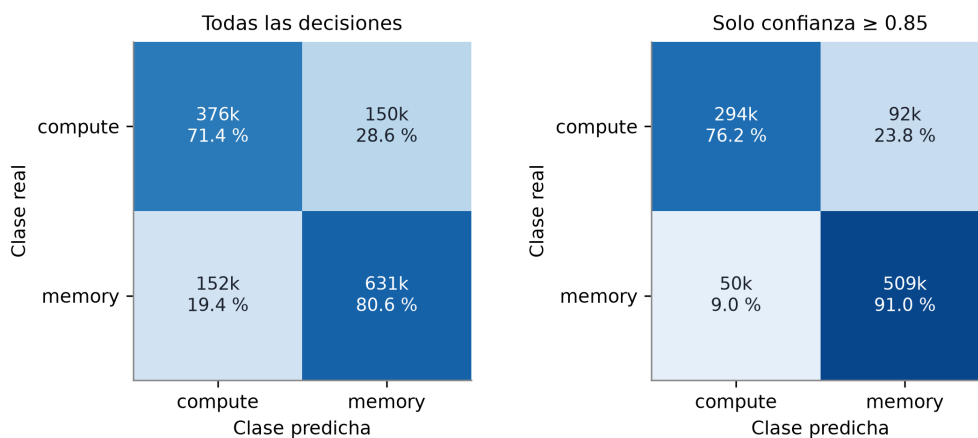


Figura 3.7: Matrices de confusión del modelo final. A la izquierda, cuando se obliga a emitir una clase en todos los intervalos; a la derecha, solo sobre los intervalos con confianza mayor o igual a 0.85. Fuente: elaboración propia.

El resultado no es uniforme entre familias (Figura 3.8). Catorce familias superan 0.9 de

exactitud y cinco quedan por debajo de 0.55, con *npb_ft* en 0.314. Las mejores son las de comportamiento extremo y estable; las peores, aquellas cuyo perfil de contadores coincide con el de familias de la clase contraria.

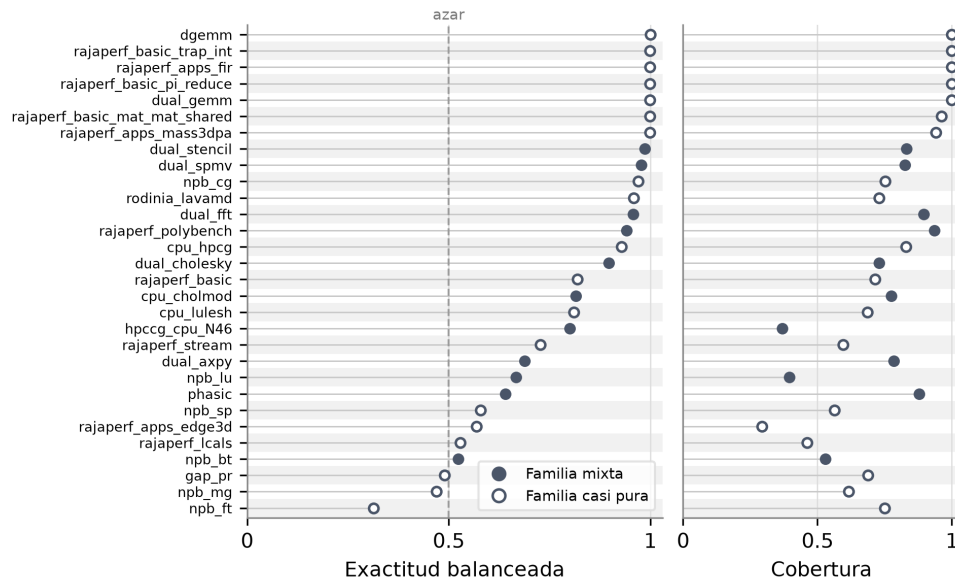


Figura 3.8: Exactitud balanceada (izquierda) y cobertura de la decisión automática (derecha) para cada familia retirada, ordenadas por exactitud. El marcador lleno distingue las familias mixtas, entre 10 y 90 % de intervalos *memory-bound*, y el hueco las casi puras; la línea discontinua marca el azar. Fuente: elaboración propia.

Al bajar el reloj, la exactitud balanceada por celda y la cobertura de la decisión automática caen de forma conjunta (Figura 3.9). Es consistente con el desplazamiento del *ridge*: en los niveles bajos más intervalos pasan a la clase de cómputo y quedan cerca de la frontera entre clases, donde el modelo se abstiene más.

La exactitud global repunta en F8 (0.757, frente a 0.710 en F7), pero es un efecto de composición: F8 aporta más intervalos (256 531, frente a 85 447–186 309 en los demás), mayoritariamente *compute-bound*, que el modelo reconoce mejor a esa frecuencia; la exactitud balanceada por celda no rebota.

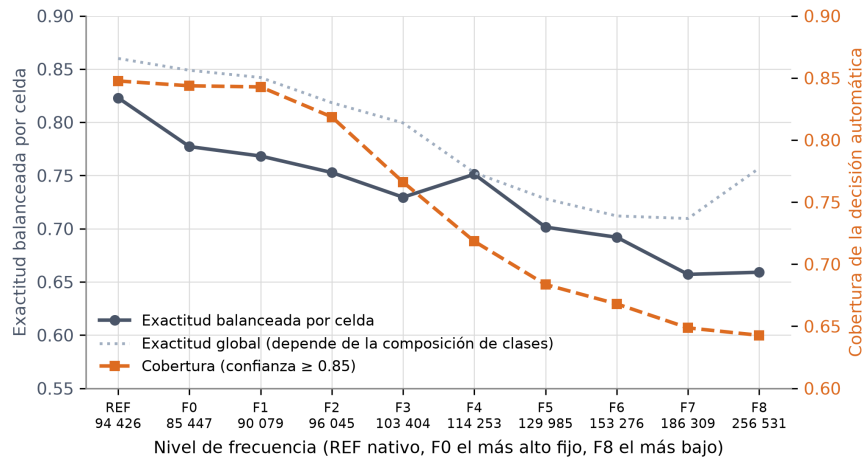


Figura 3.9: Exactitud balanceada por celda y cobertura de la decisión automática (confianza $\geq 0,85$) en función del nivel de frecuencia, con el número de intervalos evaluados de cada nivel bajo su rótulo; la línea punteada es la exactitud global, que depende de la composición de clases de cada nivel. Ambas caen de forma conjunta al bajar el reloj. Fuente: elaboración propia.

Decisión selectiva y costo de inferencia

Se congeló el umbral de abstención CPU en 0.85 (Figura 3.10, derecha); elevarlo a 0.90 aumentaba la exactitud de selección solo 0.003 y reducía la cobertura en 6.5 puntos. Es una decisión de diseño posterior al resultado; elegir el umbral por pliegue, dentro de cada familia retirada, da prácticamente lo mismo (0.734 sobre lo decidido), de modo que congelar un único valor no infla el resultado. Con ese umbral, sobre el modelo final, el agente decide el 72.2 % de los intervalos (75.1 % promediando por familia y 29.3 % en la familia peor cubierta) y el resto recibe *revisar*. Sobre lo que decide, la exactitud balanceada por celda sube de 0.728 a 0.738 y la F1 macro agrupada de 0.760 a 0.842, y la sensibilidad de las clases *compute-bound* y *memory-bound* de 0.714 y 0.806 a 0.762 y 0.910 (Figura 3.7); la ganancia por celda es modesta y la mayor parte de la mejora se ve en las métricas agrupadas. Abstenerse al azar en una proporción equivalente de intervalos no mejora nada (0.716 y 0.752), de modo que la ganancia proviene de que la confianza identifica casos difíciles y no de decidir menos (Figura 3.10, derecha).

El límite de la abstención es que la confianza está mal calibrada fuera de la familia de entrenamiento (Figura 3.10, izquierda): el modelo declara una confianza media de 0.890 y acierta el 0.764 (error de calibración esperado de 0.126; una sola semilla, frente a 0.769 de exactitud global promediando cinco). Además, hay familias que fallan *con* confianza alta, a las que el umbral no protege: entre los intervalos con confianza mayor o igual a 0.9, *npb_ft* acierta el 32 % y *npb_mg* el 35 %.

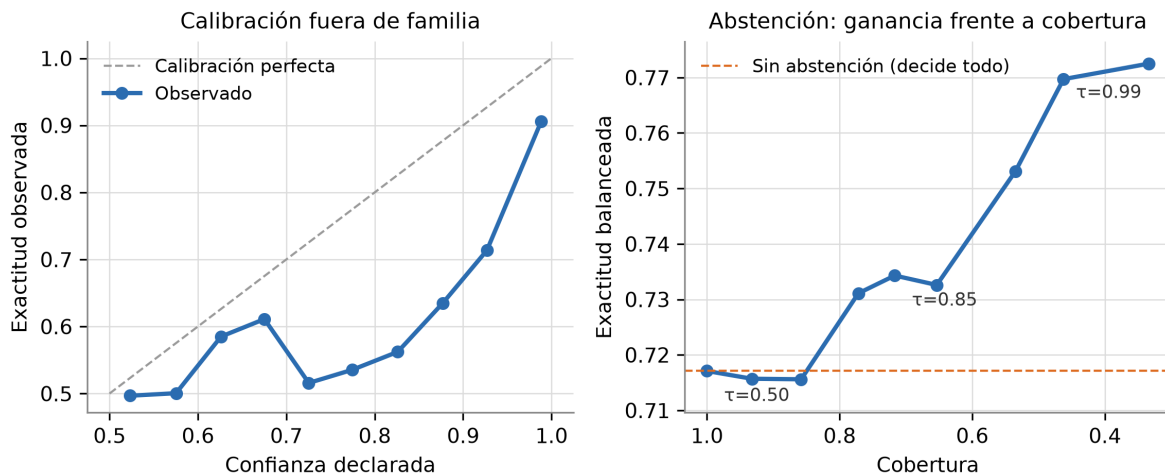


Figura 3.10: A la izquierda, confianza declarada frente a exactitud observada sobre familias no vistas; la curva por debajo de la diagonal indica sobreconfianza. A la derecha, exactitud balanceada sobre lo decidido en función de la cobertura al variar el umbral τ (el eje de cobertura está invertido: a la derecha, el umbral es más estricto); la línea discontinua naranja es la exactitud sin abstención; ambas curvas usan una sola semilla de muestreo, por lo que sus valores difieren ligeramente de los promedios de cinco semillas que se dan en el texto. Fuente: elaboración propia.

Ante la salida *revisar*, el agente conserva la última frecuencia aplicada. Las latencias de inferencia de la selección de modelo incluyen la sobrecarga del intérprete de Python y son una cota superior para la implementación integrada en el agente (sección 3.3.2).

Dónde está el límite del resultado

Cuatro análisis indican que el límite proviene de la información contenida en los contadores y no del modelo.

El primero es que un clasificador sin ajuste, de 25 vecinos más cercanos sobre cuatro tasas estandarizadas, alcanza 0.684 con el mismo protocolo, y 0.699 si recibe además la frecuencia: queda a menos de 0.045 del modelo final. El segundo es la existencia de perfiles gemelos entre familias de clase opuesta (Figura 3.11). El caso más simétrico enfrenta a *gap_pr*, *memory-bound*, con la fase *compute-bound* de *phasic*: la primera tiene MPKI de 0.82, IPC de 1.11 y tasa de fallos de 0.76; la segunda, MPKI de 0.94, IPC de 0.97 y tasa de fallos de 0.72. El par más cercano en distancia estandarizada es otro: la celda rara *memory-bound* de *rajaperf_basic_mat_mat_shared* (0.1% de sus intervalos; la familia es *compute-bound*) queda a la mitad de distancia de la celda *compute-bound* típica de *rajaperf_basic_pi_reduce*, de la misma suite. Ningún modelo que reciba solo esas señales puede separar estos pares, y

las tres familias involucradas están entre las de peor resultado.

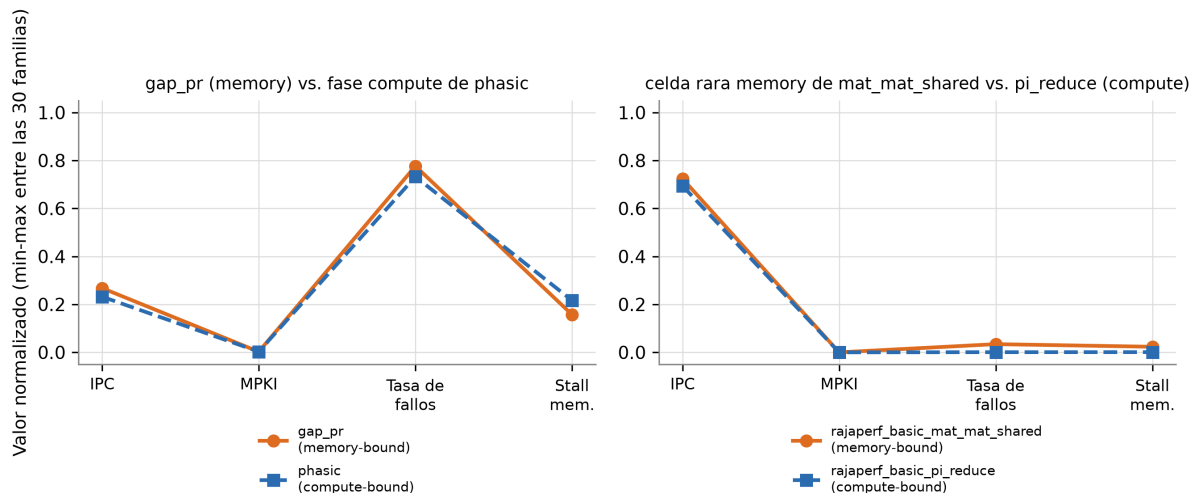


Figura 3.11: Cuatro variables de entrada, normalizadas al rango de las 30 familias, para los dos pares de celdas más confusables entre clases opuestas. Fuente: elaboración propia.

El tercero es la posición respecto del ridge (Figura 3.12). Los intervalos a menos de un factor dos del punto de inflexión son el 12.5 % del total y se clasifican con 0.521 de exactitud, frente a 0.799 en el resto, pero concentran solo el 25.3 % de los errores: tres cuartas partes ocurren lejos del ridge, donde la etiqueta no es ambigua. Además, el error no es simétrico alrededor de la frontera: en la franja compute-bound inmediata al ridge la exactitud queda por debajo del azar, mientras que en la franja simétrica memory-bound es bastante mayor. Cerca de la frontera el modelo empuja hacia memory-bound, lo que es consistente con el déficit de sensibilidad de compute-bound (sección 3.1.4) y con los perfiles gemelos.

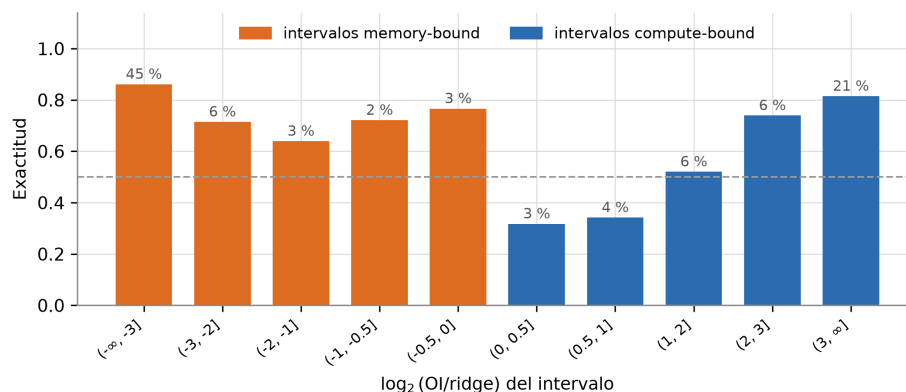


Figura 3.12: Exactitud del modelo final por franja de $\log_2(\text{OI}/\text{ridge})$. El porcentaje sobre cada barra indica qué fracción de los intervalos cae en esa franja. Las franjas negativas corresponden a intervalos *memory-bound* y las positivas a *compute-bound*. Fuente: elaboración propia.

El cuarto es la curva de aprendizaje por número de familias (Figura 3.13). Al ajustar con subconjuntos de 3 a 23 familias, el resultado sobre la familia retirada sube de 0.615 a 0.718, lo que implica aproximadamente 0.037 de ganancia cada vez que se duplica el catálogo. El resultado es consistente con que la diversidad algorítmica lo gobierne, aunque el diseño no separa el número de familias del tamaño de la muestra; la mejora disponible por esa vía, dentro del alcance de este trabajo, es pequeña.

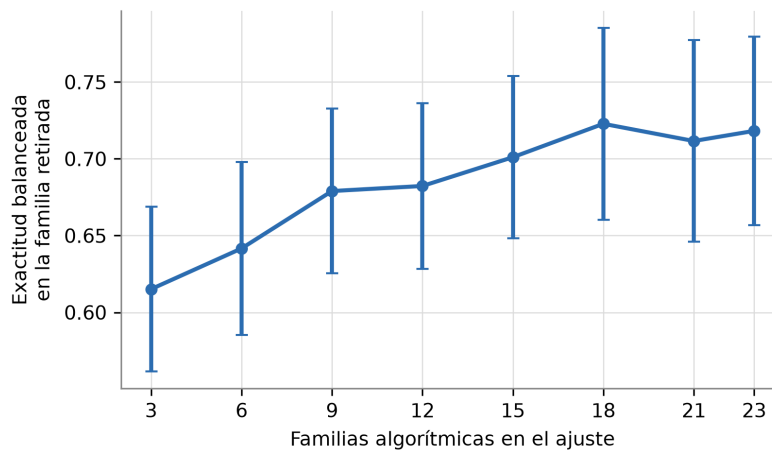


Figura 3.13: Exactitud balanceada sobre la familia retirada en función del número de familias empleadas en el ajuste. Cada punto promedia las 30 familias retiradas y tres repeticiones de la selección; las barras indican el intervalo de confianza del 95 % entre familias. Fuente: elaboración propia.

Como referencia de límite superior, el mismo modelo con la intensidad operacional medida como entrada adicional alcanza 0.9989, frente a 0.728 sin ella. No es una mejora del modelo, porque la intensidad operacional es la cantidad con la que se define la etiqueta; por eso queda fuera del vector de entrada y solo muestra que la brecha entre 0.728 y 1 es información que los contadores de rendimiento no transportan.

3.1.5. Invarianza del clasificador de CPU frente a la acción del agente

La frecuencia observada es una de las seis variables del clasificador de CPU y el agente la modifica, de modo que se le aplicó la prueba contrafactual de la sección 2.3.4: se fuerza la frecuencia a un valor fijo y se mide cuántas predicciones cambian. Sobre el modelo final y sus 1.31 millones de intervalos de 49 kernels, forzar la frecuencia a 3.2, 2.9, 2.0 y 0.8 GHz cambia, respectivamente, el 10.6, 10.1, 4.5 y 8.8 % de las predicciones, de modo que existe una dependencia moderada. Pero el cambio que el agente ejecuta es acotado, de F0 (3.2 GHz) a

F1 (2.9 GHz): sobre las 179 774 filas observadas cerca de 3.2 GHz solo cambia el 0.42 % de las predicciones. Con ese espacio de acción la retroalimentación por esta variable es despreciable, y el modelo de CPU se despliega sin modificaciones.

3.1.6. Política de frecuencia por clase en CPU

Se comparó el EDP de cada nivel fijo frente a la base conforme al procedimiento de la sección 2.3. La política resultante es *no actuar* en ambas clases (Figura 3.14). La Figura 3.15 separa la energía y el tiempo que producen ese resultado. En cómputo, F0 mejora 0.56 % frente a la base, menos que el efecto mínimo relevante de 1 %; todos los niveles inferiores empeoran el EDP. En memoria tampoco hay ganancia agregada a menor reloj: F1 empeora 5.4 % (IC95 $[-6,8; -3,4]$) y los niveles inferiores empeoran más. Algunos kernels STREAM presentan ahorros puntuales entre 4 y 11 % en su mejor nivel, pero el signo y la magnitud varían entre niveles y repeticiones; no justifican una regla común.

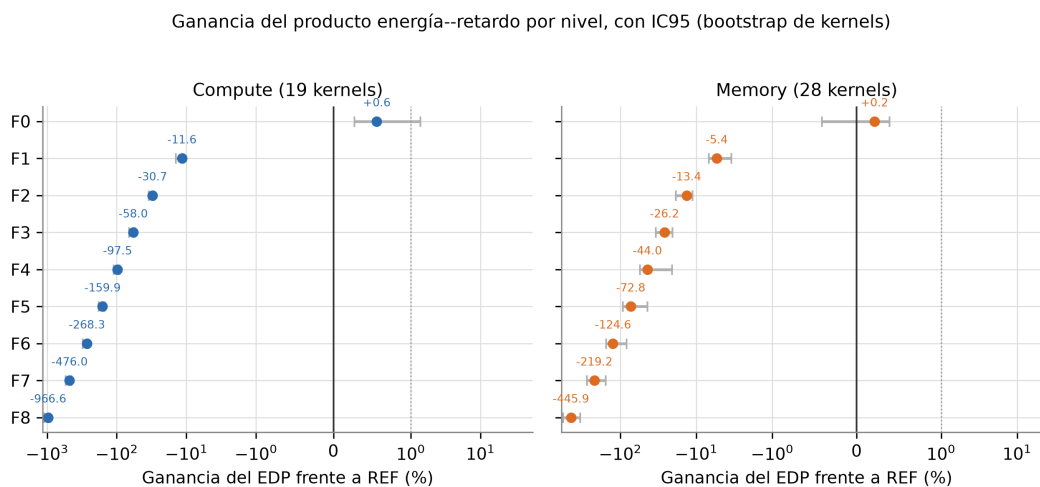


Figura 3.14: Ganancia del EDP frente a la base por nivel de frecuencia y clase (cómputo: 19 kernels; memoria: 28), con intervalo de confianza del 95 % por *bootstrap* de kernels; un valor negativo indica empeoramiento y cada punto lleva su valor en %. Los límites numéricos de los intervalos están en el repositorio. Valor p de Wilcoxon pareado: en cómputo, 0.008 en F0 y menor que 0.001 de F1 a F8; en memoria, 0.938 (F0), 0.120 (F1), 0.073 (F2), 0.018 (F3), 0.009 (F4) y menor que 0.001 de F5 a F8. El eje horizontal es simétrico-logarítmico para mostrar a la vez F0, cercano a cero, y F8, con una pérdida de casi un orden de magnitud. Ningún nivel deja el intervalo de confianza por encima de cero de forma útil. Fuente: elaboración propia.

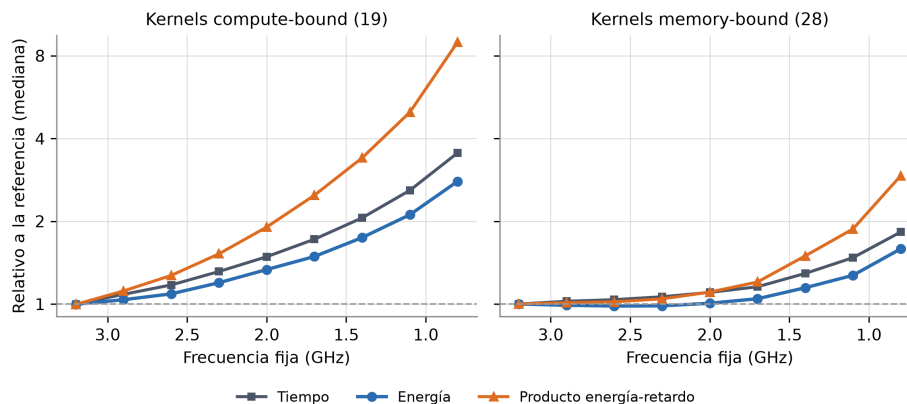


Figura 3.15: Mediana entre kernels de la energía (paquete y DRAM), el tiempo y el producto energía-retardo de cada nivel fijo, relativos a la referencia nativa, para las dos clases. El eje vertical es logarítmico. Fuente: elaboración propia.

La energía sube al bajar la frecuencia porque el tiempo crece más de lo que cae la potencia (Figura 3.15); en memoria se mantiene cerca de la referencia hasta unos 2 GHz. La razón es que unos 85 W, el 84 % de la potencia a 3.2 GHz, se consumen sea cual sea la frecuencia (Figura 3.16). Reducir la frecuencia a 0.8 GHz ahorra solo 15 % de potencia mientras alarga la ejecución. Incluso un oráculo que eligiera el mejor nivel medido por kernel ganaría 2.2 % en promedio; 22 de 47 kernels quedarían por debajo del efecto mínimo de 1 % (los 49 kernels del clasificador, sin los tres *phasic* de duración fija, y con *ptrchase*, que solo se midió para esta política). Esta cota es optimista porque el mejor nivel también captura ruido de medición.

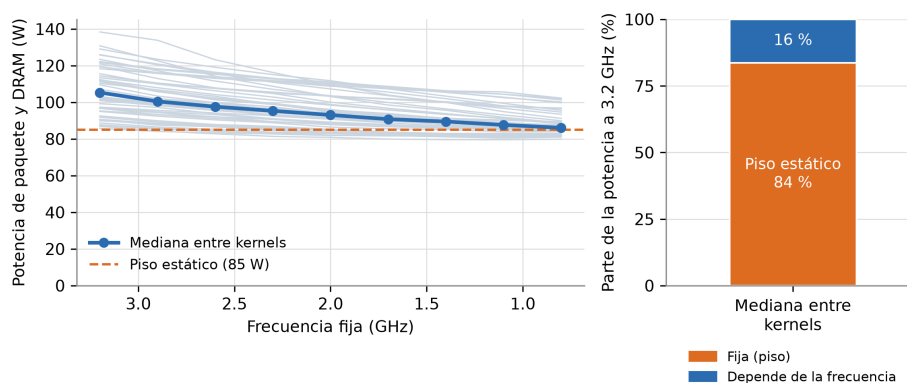


Figura 3.16: A la izquierda, potencia media de paquete y DRAM de cada kernel (líneas grises) y su mediana en cada nivel fijo, con el piso estático ajustado. A la derecha, parte de la potencia a 3.2 GHz que corresponde al piso: la barra muestra la mediana entre kernels (84 %), y según el kernel va de 62 a 94 %. Fuente: elaboración propia.

Qué tamaño de efecto podía detectar este diseño

Un resultado de *no actuar* solo es informativo si se sabe qué efecto podía detectar el experimento; de lo contrario no distingue entre que bajar la frecuencia no ayude y que la medición fuera demasiado imprecisa para verlo. La variabilidad de los datos tiene dos fuentes. La primera es el error de medición: repetir el mismo kernel en el mismo nivel da un producto energía-retardo que oscila con una desviación estándar del logaritmo de 1.05 % en mediana. La segunda es la diferencia entre familias: cada una responde de forma distinta a bajar la frecuencia, y esa dispersión entre las 19 familias *memory_bound* es de 5.35 % en F1 y 9.70 % en F2. La incertidumbre de la ganancia promedio proviene sobre todo de la segunda fuente, entre cinco y diez veces mayor que la primera. Por eso repetir más las mediciones no la reduciría, y solo lo haría un catálogo con más familias.

Un modelo de potencia $P(f) = P_0 + c f^k$, ajustado a cada kernel con un error mediano de 0.34 W y un piso casi idéntico entre kernels (80 a 100 W), explica por qué. Bajar la frecuencia mejora el EDP únicamente si la pendiente α de $\log T$ frente a $\log f$ es menor que un valor crítico, cuya mediana es 0.14 y que en ningún kernel supera 0.33, mientras los kernels de cómputo denso tienen un α cercano a 1. El oráculo que elige el mejor nivel de cada kernel gana como máximo 12.5 %. Por tanto, decidir actuar exige identificar la región de α bajo, que la clase Roofline por sí sola no determina.

3.2. Resultados de GPU

3.2.1. Validación de control y etiquetado

El actuador de GPU sostuvo el bloqueo del reloj de *streaming multiprocessors* bajo carga: con objetivos de 600 y 1200 MHz se registraron 305 y 174 lecturas activas coincidentes con el valor solicitado, y la restauración se verificó al finalizar las campañas dedicadas (la premisa física de la política se comprueba en la sección 3.3.1). En el nivel de referencia, los puntos de inflexión medidos fueron 14.97 FLOP/byte para FP32 y 6.90 FLOP/byte para FP64. La caracterización solicitó ambas precisiones a la vez y mostró que la precisión declarada no siempre es la ejecutada: *rodinia_backprop* y *rodinia_myocyte*, declarados FP32, ejecutaron 82.3 y 71.3 % de sus FLOPs en FP64 (se conservaron con precisión FP64 porque su clasificación *memory-bound* no cambia con ninguno de los dos techos), y *rodinia_hotspot*, con 30.4 % en FP64 y una intensidad de 7.21 FLOP/byte en el borde del *ridge* FP32 (7.28 a 7.98), se retiró del catálogo por no admitir un único techo.

3.2.2. Conjunto de datos y clasificador de fase en GPU

El conjunto de entrenamiento del clasificador GPU reúne 483 corridas aceptadas de 19 kernels y 16 familias algorítmicas, en los diez niveles de frecuencia (referencia nativa y F0 a F8). Proceden de la campaña GPU final y de tres campañas adicionales de *kmeans*, *miniBUDE* y *conv2d*. Cada corrida tiene una etiqueta Roofline obtenida fuera de línea y señales NVML agregadas por corrida. Los tres núcleos CUDA de RAJAPerf (GEMM, *stencil* de calor y Jacobi 2D) forman familias separadas, porque sus perfiles Roofline difieren entre sí. La composición es de 171 corridas *compute_bound* y 312 *memory_bound*, y 13 de las 16 familias son homogéneas respecto de su etiqueta.

Los candidatos reciben cinco señales agregadas por corrida: las medianas de utilización GPU, utilización de memoria, potencia y reloj SM, más la desviación estándar de la utilización de memoria, que evita que la mediana diluya cargas de lanzamientos cortos y repetidos como *kmeans*, de intensidad operacional alta pero con una firma NVML agregada similar a la de una carga poco ocupada. Las cinco no presentan colinealidad relevante: el factor de inflación de varianza (VIF) fue de 1.89 para la utilización GPU, 1.76 para la de memoria, 2.26 para la potencia, 1.37 para el reloj SM y 1.32 para la desviación estándar de la utilización de memoria, todos muy por debajo del umbral de alerta de 10 del contrato de variables.

Con el protocolo de validación definido en la sección 2.3, ninguno de los siete modelos candidatos domina de forma concluyente (Figura 3.17). *Random forest* (0.807), *extra trees* (0.806) y la regresión logística (0.792) resultaron estadísticamente indistinguibles entre sí por remuestreo pareado de familias, con probabilidades de superioridad (fracción de réplicas *bootstrap* en que un modelo supera al otro) entre 0.51 y 0.59. *Random forest* sí superó con evidencia razonable a XGBoost, con probabilidad de superioridad de 0.984. La búsqueda anidada de hiperparámetros empeoró cuatro de los cinco modelos ajustables (por ejemplo, *random forest* pasó de 0.807 a 0.778) y solo mejoró el árbol de profundidad 6, de 0.727 a 0.783; por eso se descartó.

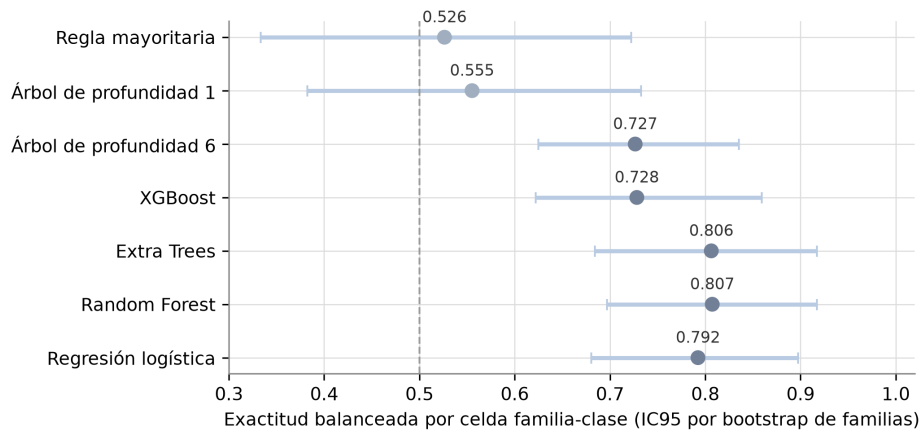


Figura 3.17: Exactitud balanceada por celda familia-clase de los siete modelos candidatos con las cinco señales, con intervalo de confianza del 95 % por *bootstrap* de familias. Fuente: elaboración propia.

El núcleo GEMM de RAJAPerf y *LUD* de Rodinia no se resuelven con ninguna variable admisible por una razón física: cambian de régimen Roofline según el nivel de frecuencia, porque el punto de inflexión se desplaza con el reloj mientras la intensidad operacional del kernel es fija (*LUD* es *memory_bound* en los relojes más altos y *compute_bound* en los más bajos). Distinguirlos exigiría una variable de distancia al *ridge*, que es fuga de la propia etiqueta.

3.2.3. Candidato operativo de GPU: invarianza frente a la acción del agente

El agente fija el reloj de la GPU, y el reloj y la potencia entre las señales candidatas dependen de esa acción, por lo que la selección exige entradas invariantes (sección 2.3.4). La auditoría, sobre las mismas 483 corridas y sin mediciones nuevas, rompe el empate entre los tres primeros modelos (Figura 3.18). Al fijar de forma contrafactual el reloj de cada observación en 1410 MHz, la regresión logística (0.792 con las cinco señales, ECE 0.0569) cambia de clase el 26.7 % de sus predicciones, y su exactitud baja a 0.654 al excluir el reloj. El *random forest* conserva el desempeño al retirar las entradas afectadas por la actuación, con 0.807 usando cinco variables, 0.812 sin reloj y 0.819 sin reloj ni potencia, sin diferencia distinguible. El modelo desplegado es, por tanto, el *random forest* de tres señales invariantes, elegido por invarianza y no por exactitud (las tres variantes son indistinguibles) (utilización de GPU, utilización de memoria y desviación estándar de esta última), y la regresión logística de cinco señales queda como referencia estática.

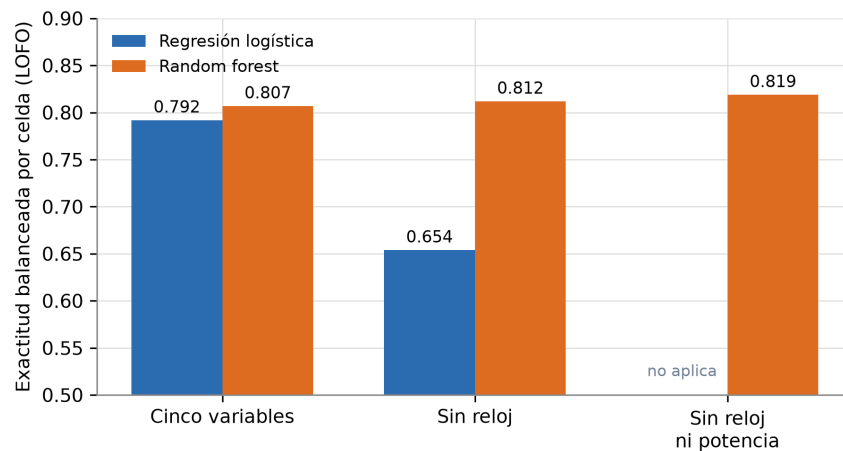


Figura 3.18: Exactitud balanceada por celda (LOFO, 483 corridas, 16 familias) al excluir las variables que dependen de la acción del agente. Fuente: elaboración propia.

Al evaluar cada familia con un modelo entrenado sin ella, el *random forest* clasifica correctamente el 89.2% de las corridas (0.819 de exactitud balanceada por celda; Tabla B.5): reconoce el 82.5% de las *compute_bound* y el 92.9% de las *memory_bound* (Figura 3.21, izquierda). El resultado no es parejo entre familias (Figura 3.19): diez de las dieciséis se clasifican sin error, mientras *rodinia_gaussian* (0.50, la familia más cercana al *ridge* del catálogo), *dual_cholesky* (0.60) y *rodinia_lud* (0.60, una de las que cambian de régimen con la frecuencia) quedan en el nivel del azar o por debajo.

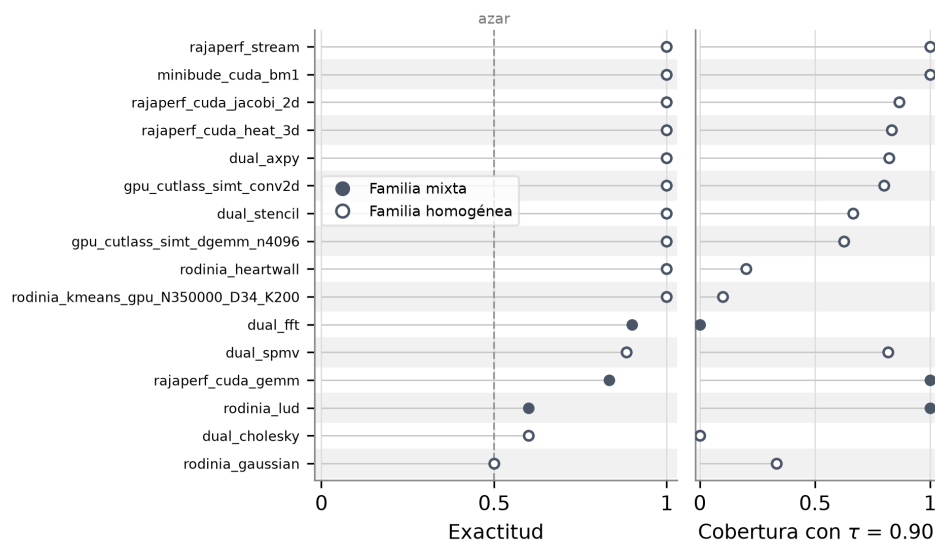


Figura 3.19: Exactitud y cobertura de la decisión automática (confianza ≥ 0.90) del *random forest* desplegado para cada familia retirada, ordenadas por exactitud. El marcador lleno indica las familias con ambas clases y el hueco las homogéneas; la línea discontinua marca el azar. Fuente: elaboración propia.

Su ECE con validación por familia es 0.0868, calculado sobre diez rangos de confianza de igual amplitud. La confianza es fiable donde más pesa: las 263 corridas con confianza superior a 0.95 se aciertan el 95.4 %, mientras que los rangos intermedios agrupan pocas corridas de una o dos familias y la exactitud oscila; el mínimo, en 0.75 a 0.80, son sobre todo corridas de *rodinia_gaussian* (Figura 3.20, izquierda).

Con el umbral de 0.90 alcanza 0.900 de exactitud balanceada por celda, con una cobertura media por celda de 65.7 % (62.1 % de las corridas; Figura 3.20, derecha). Sobre las 296 corridas que decide no comete errores en *memory_bound* (224 aciertos) y sus 15 errores son *compute_bound* clasificadas como memoria (Figura 3.21, derecha). La abstención recae sobre todo en las familias de cómputo: decide el 42 % de las *compute_bound* y el 72 % de las *memory_bound*. El modelo se ajustó sobre las 483 corridas después de cerrar la evaluación LOFO. La inferencia de una fila tarda 4.7 ms en mediana y 5.0 ms en el percentil 99, medida en Python con *scikit-learn* y no dentro del agente, frente a los segundos necesarios para detectar y actuar sobre una fase; el agente decide una sola vez por fase, así que una abstención no se reevalúa.

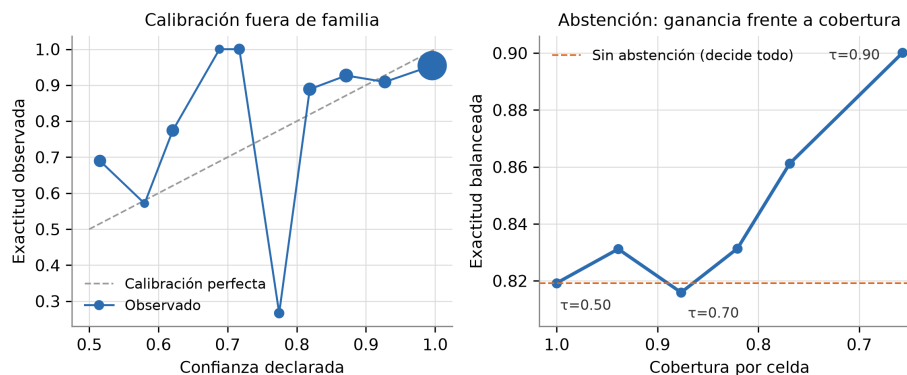


Figura 3.20: Izquierda, confianza declarada contra exactitud observada, agrupada en diez rangos de igual amplitud; el tamaño de cada punto es proporcional al número de corridas del rango. Derecha, exactitud balanceada por celda contra cobertura para distintos umbrales de decisión τ . Fuente: elaboración propia.

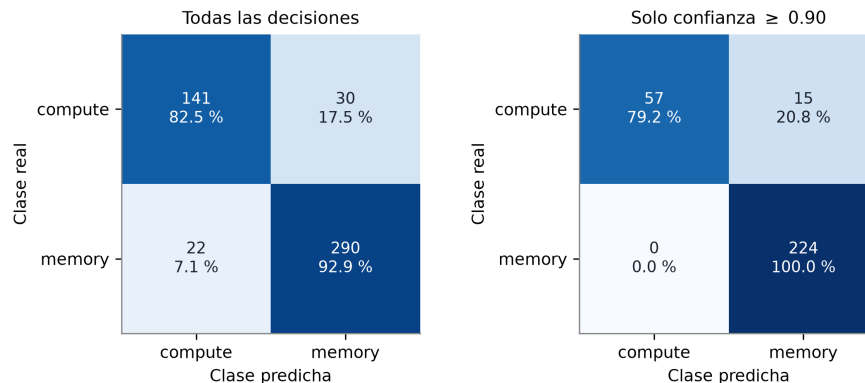


Figura 3.21: Matrices de confusión del modelo desplegado bajo validación por familia (483 corridas, una semilla; las métricas del texto por celda son promedios de cinco semillas). A la izquierda, todas las decisiones; a la derecha, solo las de confianza mayor o igual a 0.90. Fuente: elaboración propia.

La Figura 3.22 muestra que la utilización de memoria y su variabilidad concentran la dependencia predictiva del *random forest* desplegado. La utilización global de GPU aporta una señal menor.

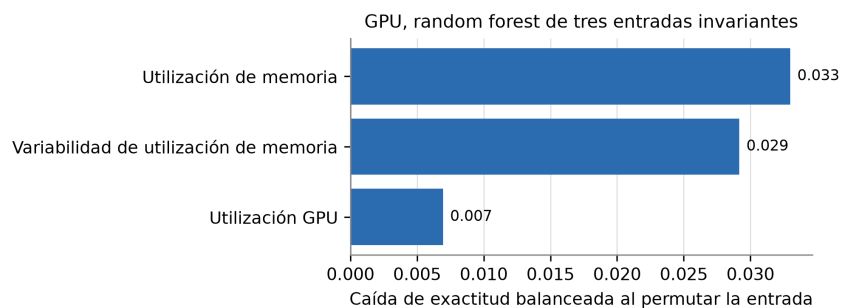


Figura 3.22: Caída media de exactitud balanceada por celda al permutar cada entrada del *random forest* en la familia retenida. Un valor positivo indica que la entrada contribuye a la predicción fuera de familia. Fuente: elaboración propia.

3.2.4. Política de frecuencia por clase en GPU

La política GPU se derivó sobre las mismas 483 corridas, conforme al procedimiento de la sección 2.3. El análisis usa la familia como unidad de agregación para evitar que las variantes de tamaño de un mismo algoritmo pesen varias veces en la decisión. La comparación es pareada con la base, así que cinco de los 19 kernels no entran: `minibude_cuda_bm1` solo tiene corridas válidas en la base (en los demás niveles el reloj observado no coincidió con el solicitado), y las variantes pequeñas de `dual_axpy` y `dual_stencil`, el GEMM y el *stencil* de calor de RAJAPerf no tienen corridas aceptadas en la base (más cortas a relojes altos, no

alcanzaron la fracción mínima de muestras NVML posteriores al arranque). Quedan 5 kernels *compute_bound* y 9 *memory_bound*, que se agrupan en 5 y 8 familias (los dos tamaños de *dual_spmv* forman una sola).

Tabla 3.2: Síntesis de la política GPU. Las ganancias son de EDP frente a la base; la rejilla completa está en la Figura 3.23.

Clase	Evidencia decisiva	Política
Cómputo	F1: +2,1 % (IC95 $[-12,7; 14,2]$, $p = 0,438$); sin mejora confiable	No actuar
Memoria	F1: +8,9 % (IC95 $[3,0; 13,3]$, $p = 0,020$); siete de ocho familias mejoran	Fijar F1

El resultado es **no actuar** en *compute_bound* y **actuar en F1 para *memory_bound*** (Tabla 3.2 y Figura 3.23). En *memory_bound*, la validación por familia retirada conserva la dirección en siete de ocho pliegues, con ganancias de 4.2 a 14.8 %, y arroja una ganancia fuera de muestra de 6.5 % (IC95 de $-4,0$ a 13.1 %). La excepción es *rodinia_gaussian*, que pierde 27.3 %: es la familia más cercana al *ridge* del catálogo, y por eso su tiempo responde al reloj como el de una carga de cómputo pese a su etiqueta de memoria. Se dejó en el análisis, porque quitarla tras ver la pérdida sesgaría la política a su favor. F2 alcanza una ganancia de 7.4 %, pero no es significativa al usar la familia como unidad.

El mecanismo es el mismo que en CPU. Sobre los 18 kernels con al menos tres niveles de reloj medidos (se excluye *minibude_cuda_bm1*, que solo tiene corridas en la base), los *compute_bound* tienen α mediano de 0.837 (rango 0.344–1.216, $n = 6$) y los *memory_bound* de 0.189 (rango 0.093–0.941, $n = 12$): su tiempo apenas responde al reloj SM porque el cuello de botella, el ancho de banda de la memoria del dispositivo, pertenece a un dominio de reloj que la matriz no varía. La excepción es *rodinia_gaussian*, con $\alpha = 0,941$ pese a ser *memory_bound* sin ambigüedad, y por eso la única familia que pierde en la validación por familia retirada; *rodinia_lud*, con el menor margen de clase (0.60), gana 13.9 %. Al clasificador lo confunde un kernel cuya etiqueta cambia con el nivel, y a la política, uno con etiqueta estable pero respuesta al reloj parecida a la de la otra clase. Ninguna de las dos familias se excluyó, porque hacerlo seleccionaría datos por el resultado que producen.

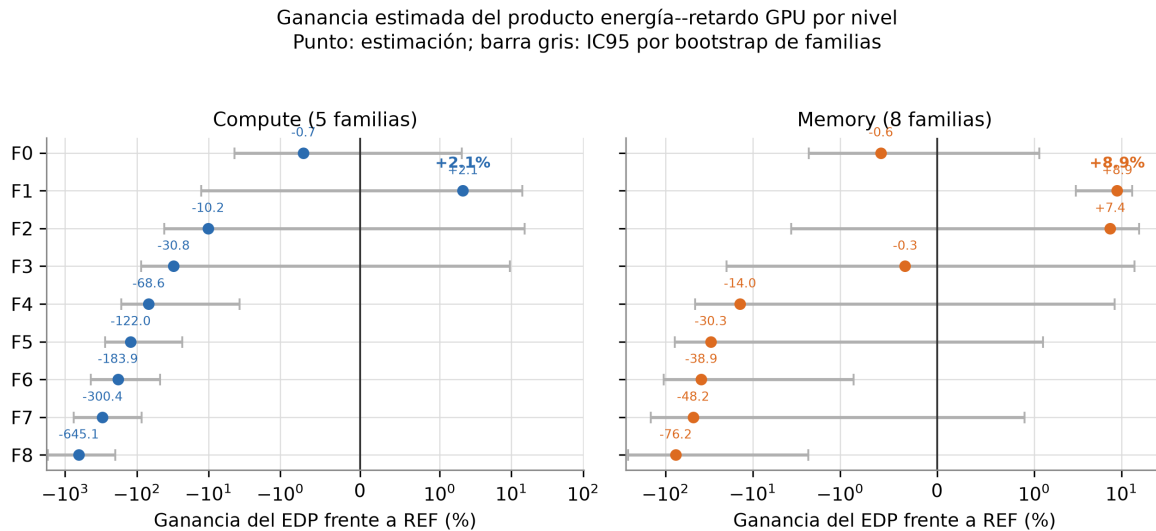


Figura 3.23: Ganancia del EDP GPU frente a la base por nivel y clase, con la familia como unidad (cómputo: 5 familias; memoria: 8). Cada punto es la ganancia estimada (en %, con su valor rotulado) y la barra gris, su IC95 por *bootstrap*, cuyos límites numéricos están en el repositorio; el rótulo en negrita señala F1. Valor p de la prueba pareada: en cómputo entre 0.438 y 1.000; en memoria, 0.020 en F1, 0.156 en F2 y entre 0.422 y 0.926 en los demás niveles. El eje horizontal es simétrico-logarítmico para mostrar a la vez F1, cercano a cero, y F8, con pérdidas de más de un orden de magnitud en *compute_bound*. Fuente: elaboración propia.

En *compute_bound*, el promedio descriptivo de F1 (+2.1 %) no es una mejora confiable (Tabla 3.2), y la validación LOFO de la regla que escoge F1 produce una ganancia realizada de -4,8 % (IC95 de -13,4 a 0 %): aplicada a una familia que no participó en elegir el nivel, empeora en promedio. Por ello el daemon conserva la base en *compute_bound*; la conclusión coincide con la de CPU.

La Figura 3.24 hace visible el contraste entre clases. En *compute_bound* el tiempo y el EDP crecen juntos desde F1, porque la energía apenas cae. En *memory_bound* el tiempo crece más despacio y la energía cae de forma sostenida hasta F3, de modo que el EDP se mantiene por debajo de 1 en F1–F2; después la caída de energía se agota y, a partir de F5, el tiempo domina.

La diferencia viene del piso estático de potencia. Se ajustó $P(f) = P_0 + c f^k$ a la potencia de cada kernel frente al reloj SM y se conservaron los 12 kernels donde el ajuste es confiable; los otros seis se descartaron por tener menos de siete niveles con datos o $R^2 < 0,8$. El piso P_0 resulta de 38.5 W en mediana y es el 55.8 % de la potencia al reloj más alto (Figura 3.25). Es una fracción menor que el 84 % de CPU, así que queda más potencia dinámica por recortar, y eso es consistente con que *memory_bound* encuentre una región de mejora en GPU que

CPU no encontró en ninguna clase.

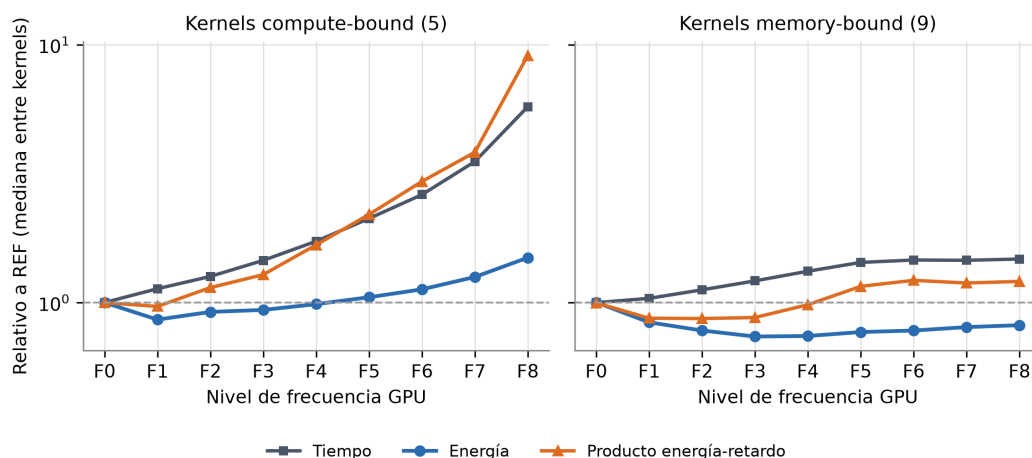


Figura 3.24: Mediana entre kernels del tiempo, la energía y el producto energía-retardo de cada nivel, relativos a la base, para las dos clases (5 y 9 kernels con corridas en la base, kernel como unidad, descriptivo; la significancia se evalúa por familia en la Tabla 3.2). El eje vertical es logarítmico. Fuente: elaboración propia.

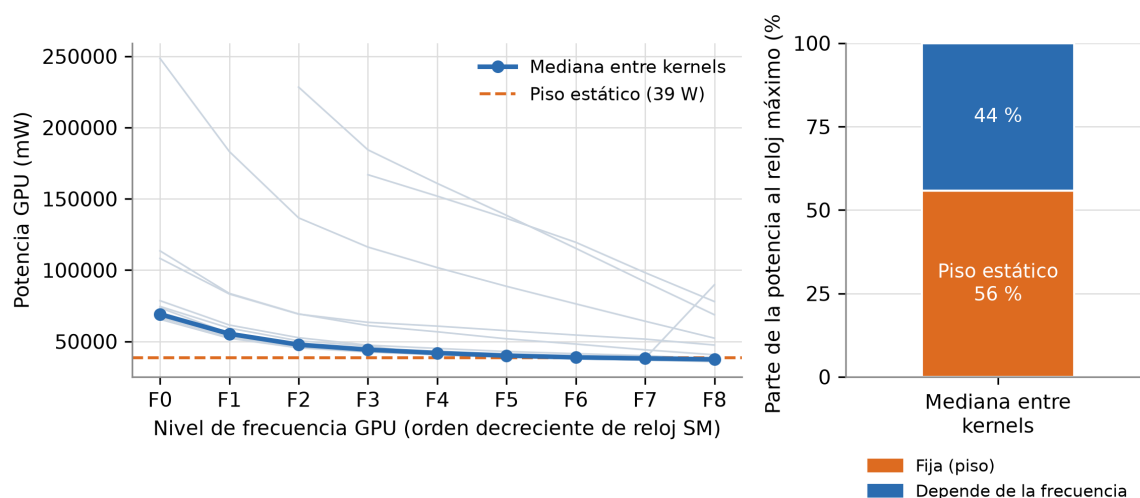


Figura 3.25: Izquierda: potencia media de GPU de cada kernel (líneas grises) y su mediana en cada nivel, con el piso estático ajustado (12 kernels con ajuste identificable). Derecha: mediana entre esos kernels de la parte de la potencia al reloj más alto que es fija (piso) y la que depende de la frecuencia. Fuente: elaboración propia.

3.3. Resultados del agente de control

Antes de esa verificación se comprobó que la exportación a ONNX no altera los modelos. En CPU, el XGBoost exportado se comparó con el original sobre los 1 309 755 intervalos

elegibles: la mayor diferencia de probabilidad fue de 3×10^{-7} y ninguna etiqueta cambió. En GPU, el *random forest* exportado se comparó sobre las 483 corridas con una diferencia máxima de 9×10^{-8} y sin cambios de etiqueta. Ambas están muy por debajo de la tolerancia de 10^{-4} , de modo que lo que decide el agente en C++ coincide con lo que se validó en Python.

3.3.1. Candado de reloj de GPU y premisa física de la política

Con una carga sintética de fases controladas se comprobó que, al fijar el reloj con `nvidia-smi -lgc`, el reloj observado coincide con el solicitado en los seis valores probados y que el rendimiento responde con la forma que supone la política (Figura 3.26): el de una carga *compute_bound* escala aproximadamente con el reloj, mientras que el de una carga *memory_bound* no cae hasta llegar a 510 MHz. Es la premisa física de bajar el reloj en la clase *memory_bound* sin costo de tiempo, medida sobre el mismo instrumento que aplicará el agente.

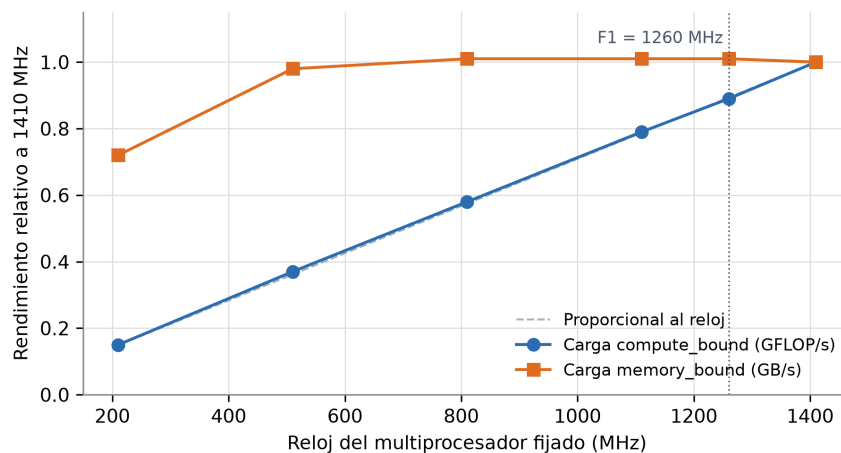


Figura 3.26: Rendimiento relativo al reloj de 1410 MHz con el reloj de GPU fijado, para una carga *compute_bound* y una *memory_bound*; la línea discontinua es la proporcionalidad exacta al reloj. Fuente: elaboración propia.

3.3.2. Latencia, restauración y detección del agente

Latencias de conmutación y sobrecarga. Fijar el reloj de GPU tarda unos 50 ms y el reloj observado alcanza el valor pedido a los 70 a 80 ms; un cambio completo, con el asentamiento de las lecturas de NVML, cuesta entre 0.35 y 0.37 s. Con la GPU en reposo, el proceso usa entre 0.0 y 0.1 % de un núcleo. En CPU, escribir los límites en los núcleos delegados toma 27.9 ms en secuencia y 2.85 ms en paralelo, y la inferencia del clasificador, 16.4 μ s en la mediana y 19.3 μ s en el percentil 99. La permanencia mínima de GPU resultante,

diez veces esa latencia, fue de 3.7 s. La sobrecarga energética del brazo *sombra* se cuantifica en la Fase 4.

Restauración y coordinación. La Tabla B.6 resume las pruebas de restauración y de coordinación. En cada prueba de terminación anómala inducida (**SIGTERM** o **SIGINT**) el agente atendió la señal y devolvió el dispositivo a su estado inicial, con código de salida 0. La prueba de coexistencia ejecutó ambos procesos en brazo activo sobre la aplicación compuesta A de GPU con el piso de CPU en 3.2 GHz.

Clasificación de fase de GPU sobre aplicaciones compuestas. Con el umbral de 0.90, en la aplicación A (tres ciclos) cada brazo tuvo tres decisiones correctas, ninguna errónea y dos abstenciones, las correctas con confianza 1.00 y las abstenciones entre 0.57 y 0.67; en la B (dos ciclos, brazo sombra) hubo dos correctas y dos abstenciones, y *LavaMD*, una familia inédita, se clasificó correctamente con confianza 1.00. Con conteos tan pequeños, la exactitud se mide en la Fase 4.

3.4. Evaluación del agente frente a la base

Las razones se expresan frente a la base (matriz de la sección 2.5); un valor menor que 1 indica una mejora.

3.4.1. Aplicaciones compuestas

La Figura 3.27 presenta las aplicaciones compuestas de familias vistas e inéditas. La fracción de tiempo en fases *memory_bound* varía entre 15 % en la aplicación B de CPU y 71 % en la A, lo que acota el ahorro posible de una política que actúa solo en esa clase.

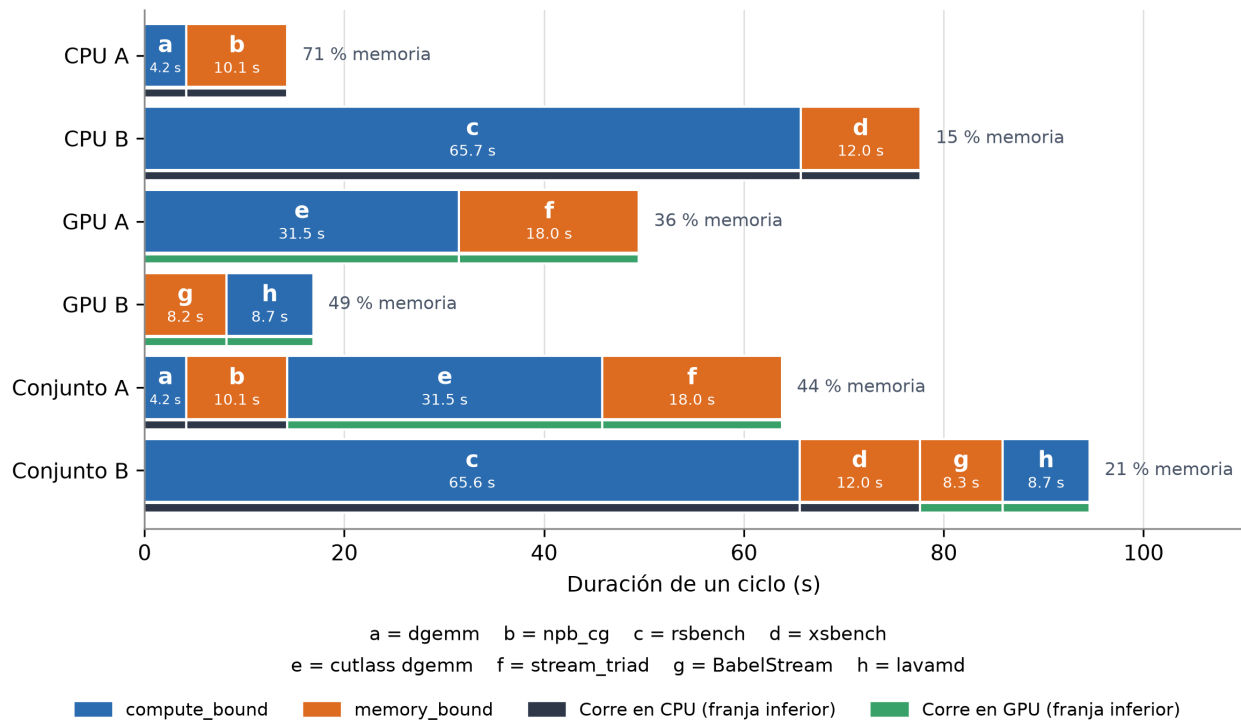


Figura 3.27: Un ciclo de cada aplicación. Cada bloque es una fase, rotulada con la letra de su kernel (clave debajo del gráfico) y su duración media en segundos, y coloreada por su clase. La franja inferior marca el dispositivo donde corre la fase, de modo que en las aplicaciones conjuntas se ve el tramo de CPU y el de GPU. A la derecha de cada barra, la fracción del ciclo en fases *memory_bound*. Fuente: elaboración propia.

3.4.2. Clasificación de fase con el agente en el lazo

Cada decisión del agente se puntúa contra la fase real correspondiente. En CPU, el agente acierta entre 96.7 y 97.5 % de las decisiones sobre las familias vistas y entre 99.6 y 99.8 % sobre las inéditas (dos kernels por aplicación, ponderadas por ventanas, de modo que no son comparables con la exactitud LOFO). La diferencia favorable a las inéditas no debe leerse como mejor generalización: en la aplicación B una sola fase de cómputo ocupa el 85 % del tiempo y ponderar por ventanas la hace dominante, de modo que el resultado depende de cómo se separan esos dos kernels y no de familias arbitrarias. En GPU no hubo ninguna decisión errónea entre las que el agente tomó, pero se abstuvo con frecuencia: dos de doce fases en A y entre cuatro y cinco de doce en B (en el alcance conjunto, tres o cuatro de veinte en A y nueve de veinte en B). Toda fase, en cualquier alcance, recibió al menos una decisión. La exactitud del brazo activo y la del brazo sombra son indistinguibles, como corresponde a un clasificador cuya entrada no depende de la acción del agente.

3.4.3. Tiempo, energía, EDP y costo del agente frente a la base

La Figura 3.28 reporta la duración, las energías de CPU y de GPU por separado y las razones frente a la base de la matriz inicial, con el agente en su configuración final. Tres hechos separan los alcances:

- En **GPU** el agente activo no cambia el resultado: el EDP del nodo queda en 1.00 frente a la base en A y en B (y el brazo sombra, en 1.01 y 1.00). El agente de GPU no impone sobrecarga medible.
- En **CPU** el brazo activo queda en 1.15 (A) y 1.06 (B) veces el EDP de la base, con un aumento de 5.1 y 1.6 % en tiempo y de 11 y 5 % en energía de CPU.
- En el alcance **conjunto** el EDP queda en 1.04 (A) y 1.06 (B) veces el de la base. Con cinco repeticiones por brazo, la prueba de Mann–Whitney da $p = 0,0079$ para sombra, activo y activo sin piso frente a la base, en A y en B: las cinco corridas de cada brazo quedaron por encima de las de la base.

El brazo sombra, que decide pero no escribe ninguna frecuencia, ya presenta la mayor parte del exceso de CPU (1.10 y 1.05 de EDP en A y B): el agente de CPU añade entre 0.4 y 1.4 % de tiempo y entre 5 y 9 % de energía de CPU (matriz inicial), un costo de mantenerlo que no depende de la frecuencia que se escriba, mientras que el de GPU no presentó costo medible (sombra entre 1.002 y 1.010 del EDP de la base). El agente de CPU usa por defecto un único hilo de inferencia: con el conjunto de hilos por defecto de ONNX Runtime, cada hilo espera de forma activa entre inferencias y compite con la aplicación por los núcleos. La acción de frecuencia añade entre +0,8 % y +5,1 % (Figura 3.28; razón activo/sombra de 1.051 en CPU con kernels vistos, 1.008 en CPU con inéditos y 1.009 en el alcance conjunto, en ambos): nunca mejora sobre solo observar, y el máximo es en CPU con kernels vistos, donde bajar el reloj en memoria empeora el EDP 5 %. Usar F0 también en *memory* no cambia el resultado de forma apreciable, y quitar el piso de coordinación lo empeora (escenario C, abajo). Una segunda medición de la base (*base sin turbo*, el mismo estado) quedó entre 0.995 y 1.001 de la primera en EDP en las cinco combinaciones compuestas comparables de C y E (se excluye E-B, cuya mediana de tres repeticiones no es comparable por la bimodalidad de *myocyte*, sección 3.4.5), lo que acota la repetibilidad de la medición.

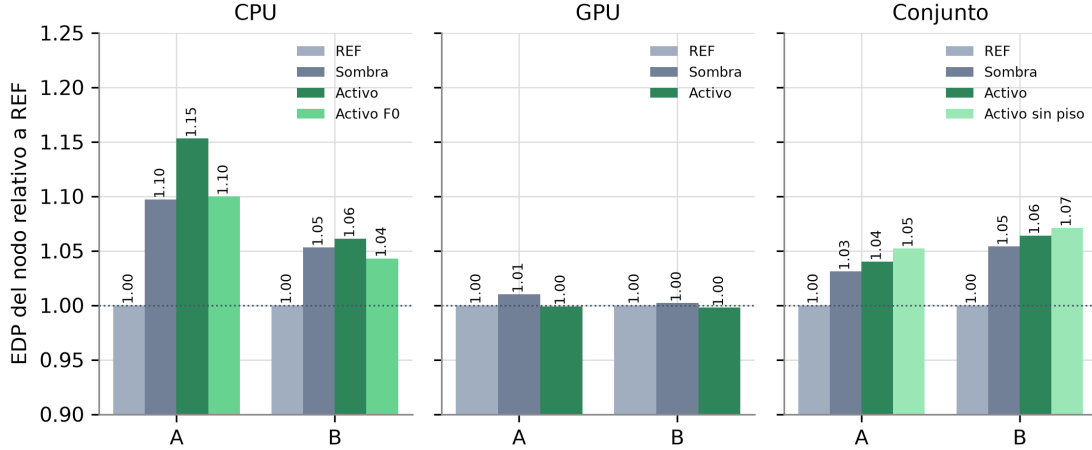


Figura 3.28: EDP del nodo, $(E_{\text{CPU}} + E_{\text{GPU}})T$, relativo a la base, por alcance, aplicación (A: vistos, B: inéditos) y brazo, en la matriz inicial. La línea punteada marca la paridad con la base.

Una repetición independiente de los alcances de CPU y conjunto (Escenario C), con el mismo agente y tres repeticiones por brazo, reprodujo estas razones con diferencias menores de 0.013, con la misma exactitud de clasificación (97.7% en A y 99.8% en B en CPU, sin errores en las decisiones de GPU). En ella, bajar a F1 en memoria en CPU con kernels vistos dio 1.154 de EDP frente a 1.085 al mantener F0, y quitar el piso de coordinación en el alcance conjunto lo empeoró entre 0.5 y 1.6 puntos.

3.4.4. Potencial de la GPU

La ganancia máxima que la política de GPU puede aportar está acotada por la Fase 2: fijar el reloj en F1 (1260 MHz) durante fases *memory_bound* mejora el EDP en 8.9% (IC95 3.0 a 13.3%), y los niveles más bajos no mejoran o empeoran (Figura 3.29, izquierda). El efecto sobre una aplicación completa se reduce por la fracción de su tiempo en memoria. Con 36% (A) y 49% (B) de fases *memory_bound*, la ganancia esperada de EDP de GPU es de 3.2% y 4.3%. La ganancia medida con el agente activo fue de 0.5% en A y de 3.0% en B en EDP de GPU, y de 0.1% y 0.2% en el EDP del nodo, que incluye la energía de CPU que el agente de GPU no modifica (Figura 3.29, derecha). Con tres repeticiones ninguna se separa de cero: en A una de las tres repeticiones empeoró (1.5%) y las otras dos mejoraron 1.4%; en B las tres mejoraron (de 1.5 a 4.2%), pero el brazo sombra, que no escribe frecuencia, ya queda 0.8% por debajo de la base, del orden de la dispersión de la base entre repeticiones (1.3% en energía de GPU).

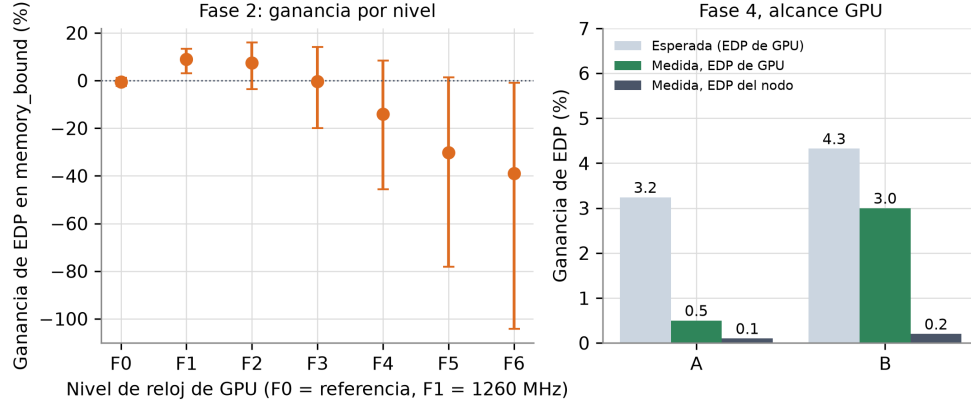


Figura 3.29: Izquierda: ganancia de EDP por nivel de reloj de GPU en fases *memory_bound* (Fase 2, IC95 sobre familias). Derecha: ganancia esperada de EDP de GPU (fracción de tiempo en memoria por la ganancia de F1) frente a la medida con el agente activo en el alcance GPU, en EDP de GPU y en EDP del nodo (medianas de tres repeticiones).

En A, el agente clasificó las fases de *triad* como memoria y aplicó F1 en cinco de las seis (en una repetición se abstuvo en ambas), y aun así la ganancia medida queda por debajo de la esperada: con tres repeticiones no puede separarse de cero y su causa no está aislada. En B, las fases de BabelStream, de 8 s, se resolvieron con una sola decisión cada una; en cuatro de las seis esa decisión fue una abstención, así que el reloj no se bajó; con esa cobertura de decisión (dos de seis fases), la ganancia esperada es del orden de 1.4%, no de 4.3%. La ganancia posible de la GPU depende entonces de dos condiciones: que la aplicación pase una parte apreciable del tiempo en fases largas *memory_bound*, y que el agente no se abstenga en ellas. El escenario E de la sección 3.4.5 evalúa ambas.

3.4.5. Escenario E: GPU dominada por fases de memoria

El escenario E evalúa la política de GPU en aplicaciones dominadas por fases *memory_bound* largas, que es donde bajar el reloj tiene más tiempo para rendir. La primera decisión de GPU llegó entre 13 y 16 s después del inicio de cada fase de memoria (la ventana de observación sostenida es de 3 s), de modo que las fases de 46 a 60 s pudieron pasar entre 65 y 78% de su duración en F1.

La Tabla 3.3 resume las dos aplicaciones declaradas antes de medir. En B, *myocyte* recibe una etiqueta de memoria por kernel pero casi no ocupa la GPU; solo el 42% del ciclo corresponde a memoria con actividad GPU. Esta diferencia entre etiqueta y oportunidad de actuación condiciona la interpretación del resultado.

Tabla 3.3: Composición del escenario E. Duraciones aproximadas por fase medidas antes de la matriz.

Aplicación	Fases de memoria	Fase de cómputo	Ciclo en memoria
A (vistas)	Stencil (58–60 s), SpMV (46 s), AXPY (47–48 s)	DGEMM (31 s)	84 %
B (inéditas)	BabelStream (36 s), Myocyte (40–41 s)	LavaMD (9 s)	89 % por etiqueta; 42 % con actividad de GPU

Familias vistas (A). Un confirmatorio declarado antes de medir, con cinco bloques aleatorizados y cinco repeticiones válidas por brazo, dio para el agente activo 0.9257 de energía de GPU, 0.9883 de duración y 0.9558 de EDP del nodo frente a la base, una reducción de 4.42 %. El brazo sombra quedó en 0.9970, así que el efecto no se debe al costo de observar. Las cinco diferencias activo–base fueron favorables; con bloques como unidad, la prueba de signos da $p = 0,0625$. Una medición exploratoria previa de tres repeticiones señaló en el mismo sentido (0.935 de energía de GPU y 0.990 de EDP del nodo; Figura 3.30).

El ahorro del nodo es modesto porque la CPU consume el 64 % de la energía y no se modifica. En la medición exploratoria, con el agente de CPU en observación, su costo (3.4 % más de energía de CPU y 0.4 % más de tiempo) iguala el ahorro de GPU y el EDP del nodo vuelve a 1.003. Conviene, por tanto, un agente de GPU sin agente de CPU, coherente con la política de no actuar sobre la CPU.

El brazo F1 fijo, previsto en el protocolo, no produjo celdas válidas: el reloj observado bajo carga quedó fuera del intervalo de 1230–1290 MHz exigido en cerca del 21 % de las muestras de cada bloque. Por ello el confirmatorio reproduce la mejora frente a la base, pero no permite establecer la ventaja de cambiar por fase frente a mantener F1 toda la aplicación. La premedición diagnóstica de F1 fijo anticipó el problema (en DGEMM el reloj observado con F1 fue indistinguible de la base) y no se usa para estimar el efecto del agente.

Familias inéditas (B). La fase de *myocyte* dura unos 41 s o unos 50 s sin relación con el brazo (con valores intermedios de 42 a 44.5 s), de modo que una mediana de tres repeticiones no permite comparar: razones como 0.84 de EDP reflejan qué repeticiones lentas cayeron en cada brazo. Por eso se agruparon las repeticiones de las tres corridas de esta aplicación con la misma configuración (la etapa E, una repetición de ocho bloques y una corrida de diagnóstico), con 21 repeticiones de la base y de sombra y 11 del brazo activo (Figura 3.30).

Sombra y activo quedan en 1.028 y 1.016 del EDP de la base y en 1.017 y 1.012 de su duración, sin diferencia distinguible ($p = 0,21$ y $0,55$ en EDP, Mann–Whitney); la energía de GPU queda en 1.006 y 1.000. La fracción de repeticiones lentas es parecida en los tres brazos (5 de 21 en la base, 4 de 21 en sombra y 3 de 11 en el activo), y la corrida de diagnóstico, con telemetría de GPU, de la frecuencia y del núcleo donde corre la aplicación y de los demás hilos del nodo, no encontró diferencia entre repeticiones lentas y rápidas; fijar el proceso del agente de GPU a un núcleo fuera de los núcleos 0 a 5 tampoco las elimina de forma distinguible. La variabilidad es entonces de la aplicación y su causa no está identificada. BabelStream recibió una decisión de memoria en 2 de las 11 repeticiones del brazo activo (confianza 1.00), con 11 a 13% menos de energía de GPU que una repetición rápida de la base; en las otras nueve se abstuvo (confianza 0.56 a 0.67). *Myocyte* no superó el umbral de actividad GPU y no recibió decisión. El límite observado es la cobertura de decisión sobre familias inéditas, no un empeoramiento por F1.

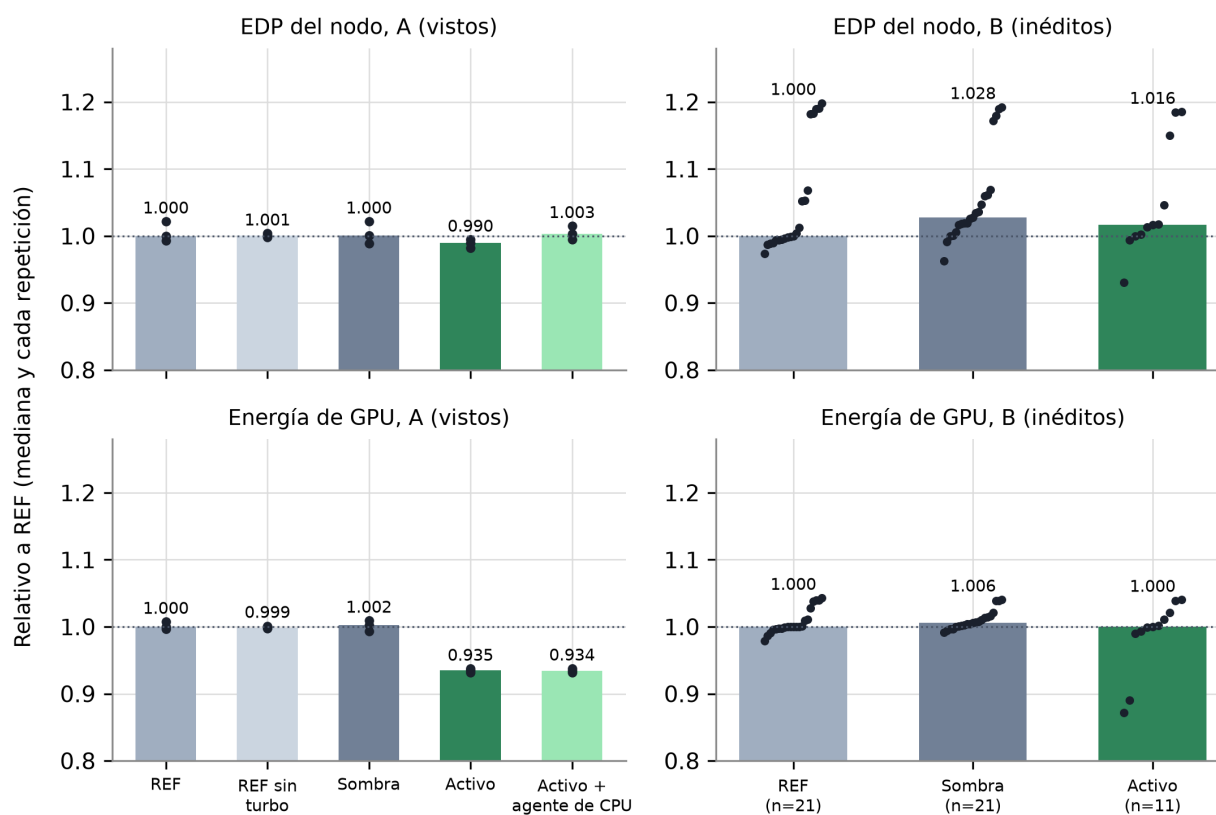


Figura 3.30: Escenario E. EDP del nodo (arriba) y energía de GPU (abajo) relativos a REF, con la mediana (barra) y cada repetición (punto), para las aplicaciones de familias vistas (A) e inéditas (B). En B se agrupan las repeticiones de tres corridas con la misma configuración (el número de repeticiones va en cada barra).

3.4.6. Escenario D: LAMMPS como aplicación de terceros

El escenario D evalúa LAMMPS 2023 con el paquete GPU del clúster, sin imponer fronteras de fase al programa. Se ejecutaron tres bloques de las entradas `chain`, `eam`, `lj` y `rhodo`, con la base, sombra, agente de GPU activo y una variante que mantiene el agente de CPU en observación. Todas las 48 celdas terminaron con código de aplicación y de daemon válido, y restauraron el estado de frecuencia. La Figura 3.31 muestra que el brazo activo de GPU queda entre 1.003 y 1.009 de la base en EDP, según la entrada. No se observa una mejora persistente y la variante con el agente de CPU en observación incrementa el EDP entre 2.9 y 3.4%. En `chain` y `rhodo` el agente no emitió decisiones y en `lj` y `eam` solo emitió `compute_bound`, por lo que no aplicó F1. Este resultado extiende la evaluación a una aplicación externa, pero no mide el efecto de la política GPU sobre una fase actuada.

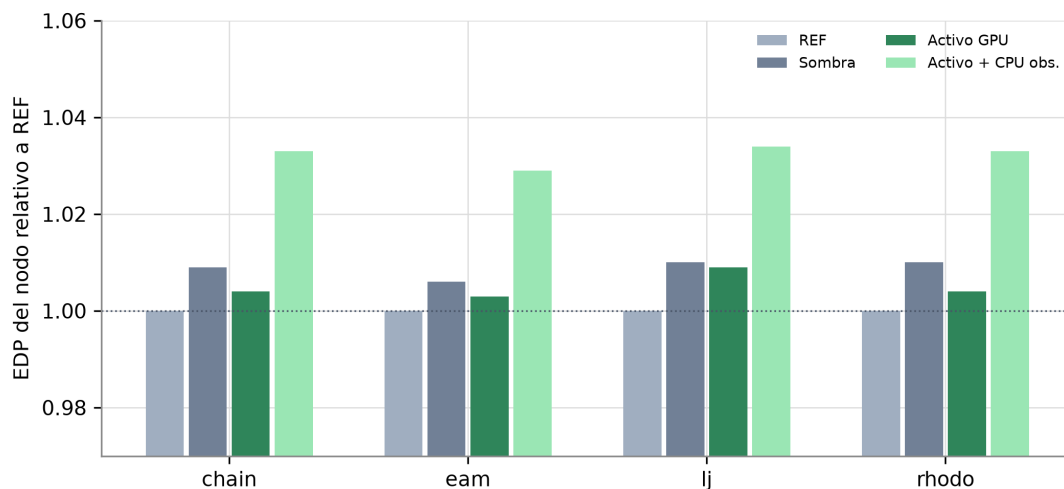


Figura 3.31: EDP mediano de LAMMPS por entrada y brazo, relativo a la base. Fuente: elaboración propia.

3.4.7. CloverLeaf CUDA Fortran como aplicación continua

CloverLeaf CUDA Fortran se evaluó como una aplicación continua de dinámica de fluidos, sin fronteras de fase impuestas al programa. El confirmatorio ejecutó cinco bloques aleatorizados con la base, sombra y activo. Las quince corridas terminaron con código de aplicación válido, restauración correcta del reloj y solución numérica reproducible: la energía cinética del fluido simulado, que CloverLeaf informa al terminar, coincidió en las quince corridas dentro de una tolerancia relativa de 10^{-8} . En cada corrida activa el agente emitió una decisión `memory_bound` con confianza 1.00 tras aproximadamente 3 s y solicitó 1260 MHz. Tras esa decisión, las 433 o 434 muestras NVML bajo carga de cada bloque permanecieron

en 1260 MHz. CloverLeaf no aporta una etiqueta Roofline independiente por fase, de modo que este experimento verifica la actuación y su efecto energético, pero no puntúa la exactitud de esa decisión.

El brazo activo mantuvo la duración en 1.0015 de la base, redujo la energía de GPU a 0.9393 y dejó el EDP del nodo en 0.9625, una reducción de 3.75 %. El brazo sombra quedó en 0.9958 de EDP, por lo que no se observa un costo apreciable de mantener el agente sin actuar. Las cinco diferencias de EDP activo–base fueron favorables; la prueba de signos por bloques da $p = 0,0625$, que no alcanza el umbral de 0.05. La Figura 3.32 separa el resultado entre CPU y GPU: la actuación reduce la energía del dispositivo sin modificar de forma apreciable el tiempo de ejecución.

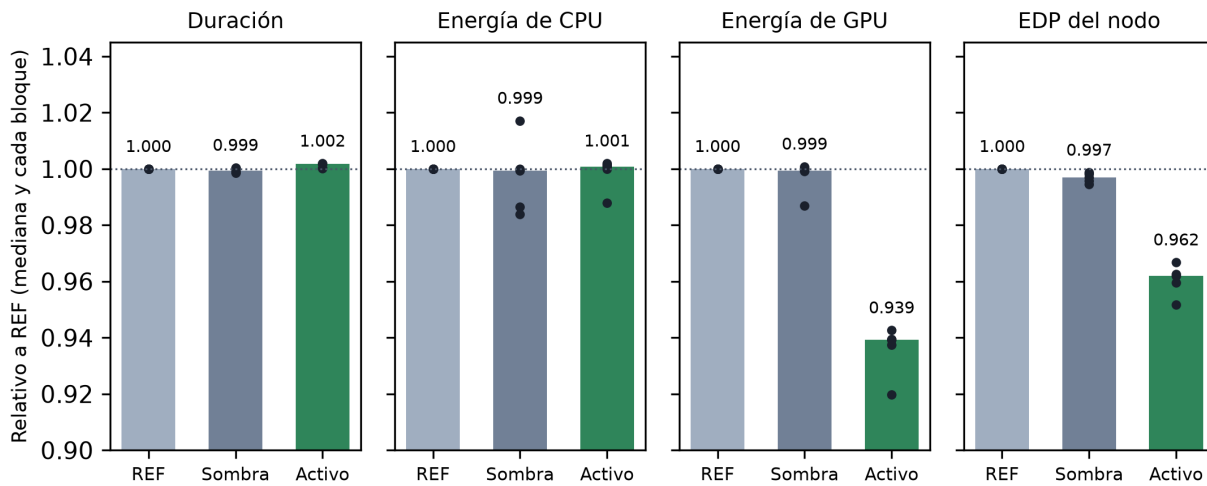


Figura 3.32: CloverLeaf CUDA Fortran: duración, energía de CPU, energía de GPU y EDP del nodo de cada brazo relativos a la base. La barra es la mediana de los cinco bloques, los puntos son los bloques individuales y la línea punteada marca la paridad con la base. Fuente: elaboración propia.

Capítulo 4

Discusión

4.1. Validez de la medición

Los enfoques de selección de frecuencia revisados [5, 10, 32] centran su evaluación en la decisión. En este trabajo también se comprobó su premisa física: que el hardware ejecuta la frecuencia solicitada. Restringir solo los procesadores lógicos delegados de CPU no alteró el rendimiento; fue necesario restringir sus hermanos de *SMT*. En GPU, el candado sostuvo los relojes solicitados durante la prueba de actuación, aunque el brazo F1 fijo del confirmatorio posterior falló la verificación bajo carga. Estas observaciones justifican aceptar una corrida por el reloj medido y no por el comando de actuación. La verificación independiente de los techos Roofline y del presupuesto de contadores cumple la misma función de protección de la etiqueta.

El catálogo GPU mostró, además, que el tipo de dato declarado por un kernel no siempre describe la precisión que ejecuta el hardware (sección 3.2.1). Para un kernel cercano al punto de inflexión, como *rodinia_hotspot*, esa diferencia puede cambiar la etiqueta Roofline. La caracterización de precisión efectiva debe preceder al entrenamiento cuando la etiqueta depende del número de operaciones por precisión.

4.2. Generalización de los clasificadores

El resultado de CPU cambia al modificar la unidad de separación: mezclar intervalos al azar da 0.874 de exactitud balanceada; dejar fuera un kernel, 0.733; dejar fuera una familia, 0.728. Los 1.31 millones de intervalos proceden de 30 familias, por lo que el último protocolo mide mejor la llegada de un algoritmo nuevo. Cambiar el modelo, ampliar las variables o

buscar hiperparámetros no produjo una mejora distinguible bajo ese protocolo. La diversidad de familias sí elevó el resultado. Es consistente con que la etiqueta se define por FLOPs sobre bytes de DRAM, mientras que el modelo observa fallos de caché, ciclos detenidos e instrucciones retiradas, que no guardan una relación fija con esa razón cuando cambia el algoritmo. La abstención reduce algunos errores, pero no corrige los de alta confianza en familias inéditas.

En GPU, la selección cambió al considerar el lazo de control. La regresión logística de cinco señales tenía buena exactitud y calibración estática, pero leía reloj y potencia, dos consecuencias de la acción que debía decidir. El bosque desplegado usa tres entradas invariantes y evita esa retroalimentación; su umbral de 0.90 sacrifica cobertura para sostener la precisión de las decisiones emitidas. La latencia de inferencia es pequeña frente a los segundos que consume detectar una fase y actuar. Por ello, mejorar solo el tiempo de predicción tendría poco efecto sobre fases breves; la abstención, que el agente no reevalúa en la misma fase, sigue siendo la limitación principal ante memoria inédita.

4.3. Alcance de la ganancia energética

La diferencia entre CPU y GPU procede de la respuesta física de cada dispositivo. En CPU, el piso estático de potencia domina: reducir el reloj deja poco margen energético y prolonga la ejecución, de modo que la política por clase es *no actuar*. En GPU, el piso pesa menos, queda más potencia dinámica por recortar y las fases limitadas por memoria responden poco al reloj SM, de modo que F1 mejoró su EDP agregado en la campaña de kernels (sección 3.2.4). La etiqueta Roofline identifica una oportunidad, pero no garantiza que todos los kernels de la clase respondan igual: `rodinia_gaussian` perdió en la validación por familia retirada.

La Fase 4 delimitó dónde se traduce esa oportunidad en una aplicación. El brazo sombra separó el costo de observar del efecto de actuar: el agente GPU no mostró costo medible, mientras que el de CPU añadió entre 5 y 9% de energía de CPU en la matriz inicial. E-A y CloverLeaf redujeron el EDP del nodo frente a la base, y el ahorro del dispositivo se diluyó porque la energía CPU apenas cambió. Los cinco bloques de cada caso favorecieron al agente, sin alcanzar significancia al nivel 0.05 (cifras en las secciones 3.4.5 y 3.4.7). La invalidez del brazo F1 fijo de E-A deja abierta la ventaja de conmutar frente a mantener ese nivel. LAMMPS y las familias inéditas quedaron cerca de paridad porque el agente no aplicó F1 de forma sostenida. La mejora observada corresponde a fases de memoria largas que reciben una decisión confiable, no a cualquier aplicación GPU.

Capítulo 5

Conclusiones

El instrumento CPU–GPU quedó validado mediante comprobaciones de contadores, frecuencia efectiva y calibración Roofline antes de usar sus medidas para entrenar. La campaña CPU produjo 1470 corridas aceptadas y 1 309 755 intervalos elegibles; la campaña GPU reunió 483 corridas de 16 familias. Esta base permitió separar la calidad de las etiquetas, la generalización de los modelos y el efecto energético de la actuación, tres propiedades que no se deducen unas de otras.

En CPU, el clasificador XGBoost obtuvo 0.728 de exactitud balanceada por celda sobre intervalos, al dejar una familia algorítmica fuera y 0.738 sobre el 72.2 % de los intervalos aceptados con el umbral de confianza de 0.85. La diversidad de familias influyó más que cambiar el algoritmo o ajustar sus hiperparámetros. La política energética resultó *no actuar* en las dos clases Roofline: unos 85 W de potencia de paquete, equivalentes al 84 % de la potencia al mayor nivel fijo, permanecen al reducir el reloj. Incluso la elección ideal del mejor nivel para cada kernel ofrece solo 2.2 % de ganancia media. La Fase 4 confirmó que, en las aplicaciones compuestas medidas, actuar sobre la frecuencia CPU no mejoró el EDP frente a la base.

En GPU, la regresión logística de cinco señales fue una referencia estática competitiva (exactitud balanceada 0.792; ECE 0.0569), pero el reloj y la potencia entre sus entradas cambian cuando actúa el agente. El modelo desplegado es un *random forest* de tres señales invariantes, con exactitud balanceada de 0.819 sobre corridas completas y ECE de 0.0868. Con el umbral operativo de 0.90, su exactitud sobre decisiones aceptadas llegó a 0.900, con cobertura media por celda de 65.7 %. La política asociada seleccionó F1 para *memory_bound*, con una mejora de EDP de GPU de 8.9 % (IC95 de 3.0 a 13.3 %) en la campaña de kernels y de 6.5 % (IC95 de −4.0 a 13.1 %, que incluye el cero) al elegir el nivel sin la familia evaluada;

para *compute_bound* conservó el *governor* nativo (la base). Este contraste entre dispositivos responde al menor peso del piso de potencia GPU y a la escasa sensibilidad al reloj de cómputo de las cargas limitadas por memoria.

El agente quedó implementado y evaluado en los brazos base, sombra y activo. En la Fase 4, el confirmatorio prospectivo de E-A redujo la energía GPU 7.43 % y el EDP del nodo 4.42 % frente a la base. CloverLeaf CUDA Fortran, una aplicación externa continua, redujo esas magnitudes 6.07 y 3.75 %, respectivamente. Los cinco bloques de cada experimento favorecieron al agente, pero la prueba de signos dio $p = 0,0625$; por ello no se afirma significancia al nivel 0.05. En CloverLeaf se verificaron la actuación y la salida numérica, pero no existe una etiqueta física independiente para puntuar su clasificación de fase. El brazo F1 fijo del confirmatorio E-A no sostuvo el reloj exigido bajo carga, de modo que la ventaja del cambio por fase frente a ese nivel fijo quedó sin establecer. LAMMPS y las aplicaciones con familias GPU inéditas mostraron resultados cercanos a paridad; en estas últimas, la abstención limitó la oportunidad de ahorro. La mejora observada es, por tanto, condicional a fases de memoria suficientemente largas y a decisiones de alta confianza.

5.1. Limitaciones

Los intervalos CPU proceden de 30 familias y no son unidades independientes de generalización. Su etiqueta tiene la granularidad de *uncore*, cercana a 10 ms, y la abstención no elimina errores de alta confianza sobre familias nuevas. El modelo GPU clasifica corridas completas, no intervalos, y se evaluó solo con validación por familia retirada sobre 16 familias; NVML no observa los contadores que definen directamente el régimen Roofline, y algunos kernels cambian de régimen al variar la frecuencia. La política F1 se validó con ocho familias *memory_bound*, cuatro de ellas núcleos de elaboración propia. Los dos experimentos favorables de aplicación tuvieron cinco bloques cada uno y el contraste contra F1 fijo quedó inválido. Todas las mediciones proceden de un único nodo, y las aplicaciones de familias vistas reutilizan kernels del entrenamiento. Estas condiciones acotan la extrapolación a otras aplicaciones y plataformas.

5.2. Trabajo futuro

La siguiente validación debería ampliar las familias GPU externas, especialmente fases *memory_bound* que hoy provocan abstención, y evaluar una sola vez el modelo y el umbral congelados. También conviene repetir el contraste entre la base, agente y F1 fijo con veri-

ficación continua del reloj bajo carga. Para mejorar la respuesta en fases breves, se puede medir por separado la latencia de detección, recolección de señales y actuación, y estudiar una reevaluación controlada de las abstenciones. CUPTI Activity podría aportar fronteras de fase para construir etiquetas independientes, previa medición de su intrusión, mientras NVML permanece como entrada ligera del agente. En CPU, la salida *revisar* podría tener contenido si el agente midiera la etiqueta física de una fracción mínima de intervalos de la carga en ejecución, con su propio método y protocolo. También conviene ampliar el catálogo con familias elegidas por su distancia a los perfiles ya cubiertos, reservar algunas para una prueba externa sellada (las seis familias de cómputo denso de RAJAPerf provienen de una misma suite) y medir la latencia de inferencia de GPU dentro del agente y no en el intérprete de Python. Además, cualquier revisión de la política requiere primero reducir el costo de observación y demostrar un margen de ahorro superior al piso de potencia medido.

Apéndice A

Detalle técnico de la metodología

A.1. Diseño de la medición

A.1.1. Muestreo y dominio *uncore*

Los eventos de núcleo se atribuyen a la carga medida. Los comandos CAS del controlador de memoria integrado pertenecen al dominio *uncore*, compartido en el zócalo [50], por lo que la campaña requiere asignación exclusiva del nodo.

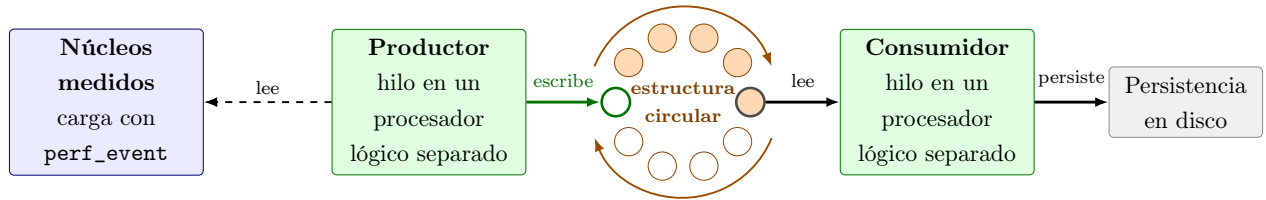


Figura A.1: Ruta de una muestra de telemetría desde la lectura periódica de los contadores hasta su persistencia. Las flechas curvas indican el sentido de avance de las posiciones de la estructura circular. Los hilos de medición y persistencia se fijan a procesadores lógicos distintos de los que ejecutan la carga. Fuente: elaboración propia.

A.1.2. Detección del transitorio de arranque

El transitorio se estima después de adquirir la traza completa, sin condicionar la captura. Sobre IPC en CPU y utilización en GPU, primero se busca el primer instante en que el coeficiente de variación no supera 5% en dos ventanas móviles consecutivas. Si una corrida contiene n muestras, cada ventana móvil tiene $w = \text{máx}(3, \text{mín}(15, \lfloor n/4 \rfloor))$ muestras. Así, el análisis usa aproximadamente un cuarto de la traza cuando esta es corta, nunca menos

de tres muestras ni más de quince. El instante detectado recibe un margen de seguridad de 20 %. En GPU se exige además que la media de ambas ventanas alcance al menos 5 % de utilización, para que el reposo inicial no se interprete como estabilidad.

Si ese criterio no encuentra un estado estable, se aplica detección recursiva de puntos de cambio sobre la misma señal. Cada partición debe reducir al menos 10 % la suma de errores cuadráticos, conserva segmentos de $\max(5, \lfloor n/50 \rfloor)$ muestras y alcanza como máximo seis niveles de recursión. Entre los segmentos obtenidos se selecciona el primero cuya media sea al menos 80 % de la meseta de mayor carga de la corrida. Durante la calibración de GPU, el intervalo se contrasta además con el ascenso de potencia hacia el régimen sostenido.

A.2. Fase 1. Recolección y caracterización

Tabla A.1: Verificaciones previas agrupadas por familia.

Familia	Verificación bloqueante
Entorno	Turbo, afinidad NUMA, dominio real de frecuencia, política SMT y presupuesto de contadores.
Catálogo	Binario, suma de verificación, criterio de éxito y memoria suficiente.
Instrumentación	Dominios RAPL, NVML, manejo de desbordamiento y almacenamiento disponible.
Actuación	Permiso de escritura y rango soportado de frecuencia cuando la campaña lo requiere.

A.2.1. Catálogo

Tabla A.2: Catálogo de kernels y procedencia de las suites.

Dispositivo	Kernel o familia	Suite u origen	Variante	Papel y referencia
CPU	STREAM	STREAM	64M elementos	Calibración de ancho de banda; McCalpin [51].
CPU	ERT	ERT	Probe propio	Calibración de cómputo; Lo et al. [52].
CPU	BT, CG, FT, LU, MG, SP	NPB-OMP	Clases B y C	Conjunto de datos; NAS Parallel Benchmarks [53].
CPU	DGEMM	DGEMM-OpenBLAS	$N = 2048$	Conjunto de datos; BLAS optimizada [8].
CPU	LavaMD	Rodinia-OpenMP	boxes1d24	Conjunto de datos; Rodinia [54].

Continúa en la siguiente página

Tabla A.2 – continuación

Dispositivo	Kernel o familia	Suite u origen	Variante	Papel y referencia
CPU	RAJAPerf-stream	RAJAPerf-OpenMP	10 veces la LLC	Conjunto de datos; RAJA Performance Suite [55].
CPU	RAJAPerf-lcals	RAJAPerf-OpenMP	10 veces la LLC	Conjunto de datos; RAJA Performance Suite [55].
CPU	RAJAPerf-polybench	RAJAPerf-OpenMP	10 veces la LLC	Conjunto de datos; RAJA Performance Suite [55].
CPU	RAJAPerf-basic	RAJAPerf-OpenMP	10 veces la LLC	Conjunto de datos; RAJA Performance Suite [55].
CPU	RAJAPerf-apps	RAJAPerf-OpenMP	Rep. controladas	Conjunto de datos; RAJA Performance Suite [55].
CPU	CHOLMOD	SuiteSparse-CHOLMOD	Laplacian3D	Conjunto de datos; SuiteSparse [56].
CPU	LULESH	LULESH	120 ³ , 300 iter.	Conjunto de datos; LLNL [57].
CPU	HPCG	HPCG	80 ³ , 5 s	Conjunto de datos; HPCG [58].
CPU	HPCCG	HPCCG-CG	$N = 46$	Conjunto de datos; Mantevo [59].
CPU	PageRank	GAP-Benchmark	Kronecker, escala 22	Conjunto de datos; GAP Benchmark Suite [60].
CPU	GEMM, Cholesky, FFT, SpMV, AXPY, stencil	Elaboración propia	Varios tamaños	Conjunto de datos; familias duales CPU-GPU.
CPU	<i>phasic</i>	HYPERION-PHASE	P010-P1000	Control sintético; elaboración propia.
GPU	BabelStream	BabelStream	Calibración BW	Calibración de ancho de banda; referencia del catálogo.
GPU	DGEMM	cuBLAS-DGEMM	$N = 4096$	Calibración y control de cómputo; NVIDIA cuBLAS.
GPU	ERT FP32, ERT FP64	ERT-GPU	FP32/FP64	Calibración de cómputo; elaboración propia.
GPU	DGEMM y Conv2D	CUTLASS	$N = 4096$ / configuración fija	Conjunto de datos; CUTLASS.
GPU	miniBUDE	miniBUDE-CUDA	BM1	Conjunto de datos; suite externa del catálogo.
GPU	RAJAPerf-stream, -jacobi, -heat, -gemm	RAJAPerf-CUDA	Variantes escaladas	Conjunto de datos; RAJA Performance Suite [55].
GPU	Gaussian, Heartwall, K-means, LUD	Rodinia	Tamaños escalados	Conjunto de datos; Rodinia [54].
GPU	AXPY, Cholesky, FFT, SpMV, stencil	Elaboración propia	Varios tamaños	Conjunto de datos; familias duales CPU-GPU.

Los tamaños, repeticiones y sumas de verificación de cada entrada están en el catálogo versionado del repositorio. Las cargas de elaboración propia no se presentan como suites externas y los microbenchmarks de calibración no forman parte del entrenamiento del clasificador.

A.2.2. Conteo de FLOPs e intensidad de GPU

En CPU, los FLOPs de cada ventana suman los sub-eventos escalares y vectoriales de 128, 256 y 512 bits de FP_ARITH_INST_RETIRED, ponderados por el número de operandos de doble precisión de cada instrucción (ecuación (A.1)). En GPU, la intensidad de un kernel suma sobre sus L lanzamientos perfilados las instrucciones de suma, multiplicación y FMA (esta última con dos FLOPs) y las divide por los bytes B_ℓ movidos a la memoria del dispositivo (ecuación (A.2)).

$$\text{FLOPs}_i^{\text{HW}} = \Delta N_{\text{esc},i} + 2\Delta N_{128,i} + 4\Delta N_{256,i} + 8\Delta N_{512,i}. \quad (\text{A.1})$$

$$I^{\text{GPU}} = \frac{\sum_{\ell=1}^L (N_\ell^{\text{add}} + N_\ell^{\text{mul}} + 2N_\ell^{\text{fma}})}{\sum_{\ell=1}^L B_\ell}. \quad (\text{A.2})$$

A.3. Fase 2. Desarrollo y validación de los modelos

A.3.1. Variables y prevención de fuga

La intensidad operacional, los FLOPs, los bytes, el punto de inflexión y la etiqueta están prohibidos como entradas. La selección de variables se congela en un contrato versionado por dispositivo: las columnas ausentes, constantes o no finitas se tratan explícitamente y la presencia de una variable prohibida aborta el entrenamiento.

Antes de entrenar se calculan las correlaciones de Pearson y de Spearman y el factor de inflación de la varianza. Cuando dos columnas describen la misma señal se conserva la medición físicamente más directa.

A.3.2. Ponderación y diagnóstico del límite

El peso de cada observación en el ajuste, inverso al tamaño de su celda de familia y clase y con media uno, es

$$w_i = \frac{1/n_{fc}}{(1/N) \sum_{k=1}^N 1/n_{f_k c_k}}. \quad (\text{A.3})$$

El candidato se congela antes de examinar errores por familia o frecuencia.

El límite del resultado se diagnostica con cuatro pruebas: una referencia de vecinos cercanos sin ajuste, los perfiles de celdas de clases opuestas con señales similares, el desempeño según la distancia al *ridge* y la curva de aprendizaje por número de familias. En conjunto

separan la falta de capacidad del modelo de la falta de información en las señales.

A.3.3. Procedimiento estadístico de la política

Para cada kernel se compara la mediana de sus tres repeticiones en cada nivel fijo con la de su propia base, lo que forma diferencias pareadas por kernel. Con menos de ocho pares se aplica Wilcoxon de rangos con signo [48]; con ocho o más, la prueba de Shapiro–Wilk sobre las diferencias decide entre la prueba t pareada, si no se rechaza la normalidad, y Wilcoxon; en CPU todos los niveles se evaluaron con Wilcoxon. El intervalo de confianza del 95 % de la ganancia agregada se estima por *bootstrap* de kernels con 4000 réplicas. En CPU se excluyen las tres variantes de *phasic*, cuya duración es fija (20.6 s en todos los niveles) y cuyo trabajo, por tanto, cambia entre niveles. En GPU, la clase de un kernel es la mayoritaria entre sus corridas y se marca ambigua, sin excluirla, si su margen es menor que 0.75; los tamaños de un mismo algoritmo se agregan por familia y la ganancia se contrasta con Wilcoxon unilateral hacia la mejora sobre las familias.

A.4. Señales y unidad de observación

Tabla A.3: Señales de telemetría del instrumento.

Dominio	Señales
Núcleo (CPU)	Instrucciones, ciclos, referencias y fallos de caché, ciclos detenidos por memoria, sub-eventos de punto flotante de doble precisión (para FLOPs), frecuencia observada (<code>scaling_cur_freq</code>)
Zócalo (CPU)	Comandos CAS de lectura y escritura del controlador de memoria (bytes a DRAM), energía RAPL de paquete y DRAM
GPU	NVML: utilización de cómputo y de memoria, potencia, reloj del multiprocesador, temperatura y energía acumulada

Tabla A.4: Unidad de observación del conjunto de entrenamiento.

Dispositivo	Unidad de observación	Origen de I	Tamaño
CPU	Intervalo del controlador de memoria	En vivo, por intervalo	1 309 755 intervalos, 30 familias
GPU	Corrida completa	Fuera de línea, por kernel	483 corridas, 16 familias

A.5. Amenazas a la validez

Tabla A.6: Amenazas a la validez y medidas de control.

Amenaza	Medida de control
Interferencia de procesos ajenos en RAPL y en la GPU	Asignación exclusiva del nodo; ninguna carga adicional durante una medición
Atribución incorrecta de contadores	Proceso hijo detenido hasta abrir los contadores, con herencia
Multiplexación de eventos	Nueve eventos dentro del presupuesto verificado; las ventanas degradadas se excluyen
Frecuencia efectiva distinta de la pedida	Turbo apagado, rango por núcleo y hermanos <i>SMT</i> , verificación por ventana
Fuga de la etiqueta hacia las variables	Lista de columnas prohibidas comprobada por código
Pseudo-replicación entre tamaños y repeticiones	La familia como unidad de validación (LOFO) y de comparación de la política
Ajuste sobre datos de prueba	Congelación del candidato antes de auditar errores por familia o frecuencia
Retroalimentación del agente sobre el modelo	Auditoría de invarianza de las variables frente a la acción del agente
Sesgo temporal o de orden	Orden aleatorizado con semilla registrada y repeticiones independientes
Deriva del estado del nodo	Snapshot y restauración del estado de frecuencia por celda
Muestra intencional en un solo nodo	Se declara como limitación; no se generaliza a otras plataformas

A.6. Protocolos de la Fase 4

Tabla A.7: Protocolos complementarios ejecutados en la Fase 4.

Escenario	Qué cambia	Motivo
Matriz inicial	Ninguno	Referencia, tal como se diseñó
C: repetición de CPU y conjunto	Repite la matriz inicial en CPU y en el alcance conjunto con el mismo agente	Comprueba la reproducibilidad de la matriz y separa el efecto de actuar del costo de mantener el agente
E: GPU dominada por memoria	Aplicaciones de GPU con tres o dos fases <i>memory_bound</i> largas (45 a 60 s) y una de cómputo, de familias vistas (E-A) e inéditas (E-B); la CPU sin agente o con el agente en observación	El techo medido en la Fase 2 es 8.9% de EDP en GPU y el efecto escala con la fracción de tiempo en memoria; es un escenario de aplicabilidad
Base sin turbo	Sin agente, con turbo de CPU apagado, en las combinaciones compuestas de C y E	Segunda medición de la base, como control de repetibilidad; no sustituye un brazo de GPU a F1 fijo
D: aplicación real de terceros	LAMMPS 2023 con paquete GPU, cuatro entradas de referencia y fases naturales	Validez externa; tres bloques por entrada y brazo
F: premedición de F1 fijo	Base frente a la solicitud de F1 en DGEMM de CUTLASS, Conv2D y LavaMD, cinco repeticiones por nivel	Comprobar que una comparación posterior contra un nivel fijo tendría un reloj efectivo distinto de la base
Confirmatorio E-A	Cinco bloques aleatorizados de la base, sombra, agente GPU activo y F1 fijo para la aplicación E-A	Confirmar la mejora frente a la base y contrastarla con F1 fijo; el brazo fijo se declara inválido si no sostiene el reloj bajo carga
CloverLeaf CUDA Fortran	Aplicación externa continua, cinco bloques aleatorizados con la base, sombra y agente GPU activo; una decisión inicial del agente por corrida	Verificar la política sobre una carga de terceros sostenida, con energía de nodo, salida numérica y reloj auditados

Apéndice B

Resultados complementarios

B.1. Conjunto de datos y clasificador de CPU

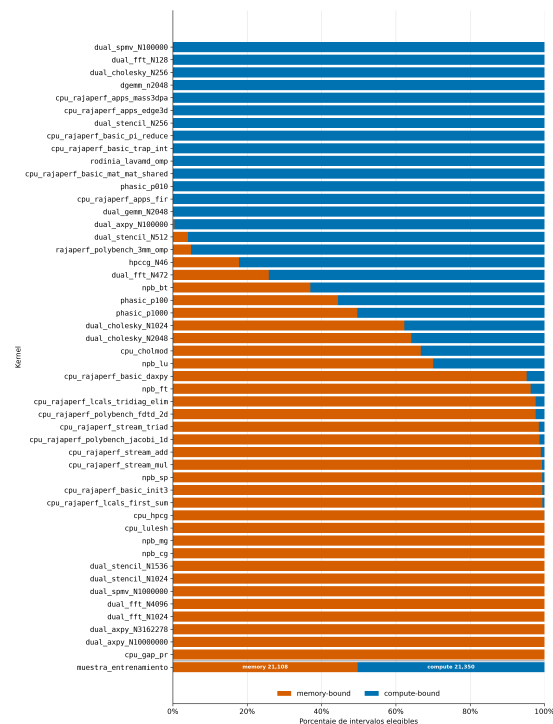


Figura B.1: Porcentaje de intervalos elegibles clasificados *compute_bound* y *memory_bound* para cada configuración de kernel de la campaña CPU. La barra final resume la muestra usada por el clasificador, con 21 350 intervalos *compute_bound* y 21 108 intervalos *memory_bound*. Fuente: elaboración propia.

Tabla B.1: Flujo de intervalos de la campaña CPU, desde los procesados hasta la muestra de ajuste. Campaña de 49 configuraciones de kernel, diez niveles de frecuencia y 1470 corridas aceptadas de 1470. El modelo se evalúa sobre todos los intervalos elegibles, en los que las seis variables de entrada están definidas.

Etapas	Intervalos	Proporción
Procesados	1 424 531	100.0 %
Rechazados por frecuencia no utilizable	71 658	5.0 %
Rechazados por ventana de origen no válida	43 118	3.0 %
Elegibles	1 309 755	91.9 %
<i>compute_bound</i>	526 279	40.2 % de los elegibles
<i>memory_bound</i>	783 476	59.8 % de los elegibles
Muestra de ajuste, $\sum \min(n_{fc}, 1000)$ sobre las celdas observadas	42 458	
<i>compute_bound</i>	21 350	
<i>memory_bound</i>	21 108	

B.1.1. Tamaño de la muestra

Tabla B.2: Sensibilidad del resultado al tope de intervalos por celda de familia y clase. El valor con tope de 1000 proviene de la corrida de sensibilidad y no de la evaluación final (0.728).

Tope por celda	Intervalos de ajuste	Exactitud balanceada
100	4 728	0.732
300	13 430	0.729
1000	42 458	0.723
3000	122 135	0.739
10 000	355 228	0.737
30 000	730 974	0.720

B.1.2. Vector de entrada y protocolo de partición

Tabla B.3: Variantes del vector de entrada bajo el mismo modelo y el mismo protocolo. La diferencia se calcula frente al vector de 6 variables.

Vector de entrada	Exact. bal.	F1 macro	Diferencia (IC95)
6 variables (configuración final)	0.728	0.760	referencia
12 variables (seis transformaciones adicionales)	0.724	0.783	-0,004 (-0,026 a +0,011)
6 variables sin frecuencia	0.720	0.752	-0,008 (-0,018 a +0,001)
Solo frecuencia observada	0.592	0.504	-0,136 (-0,208 a -0,065)

Tabla B.4: Efecto del esquema de partición sobre el resultado, con el mismo modelo y los mismos datos. El $\pm$ es la desviación estándar entre las cinco semillas de muestreo.

Esquema de partición	Qué mide	Exactitud balanceada
Intervalos repartidos al azar (80/20)	Memorización de la carga	0.874 ± 0.024
Dejar un kernel fuera (49 pliegues)	Otro tamaño o variante	0.733 ± 0.016
Dejar una familia fuera (30 pliegues)	Algoritmo nunca visto	0.728 ± 0.015

B.1.3. Métricas de los modelos finales

Tabla B.5: Valores de todas las métricas de los modelos finales de CPU y de GPU, sin abstención [61, 62]. CPU: media de cinco semillas de muestreo, desviación entre semillas e intervalo de confianza del 95 % por remuestreo de las 30 familias (el de la exactitud balanceada por celda, sobre las cinco semillas; el de la exactitud balanceada clásica, con una sola semilla). GPU: validación por familia retirada sobre 483 corridas y 16 familias, con la probabilidad promediada entre cinco semillas y el mismo remuestreo por familia; la exactitud balanceada por celda es la media de las semillas.

Métrica	CPU			GPU	
	Valor	Desv.	IC95 familias	Valor	IC95 familias
Exactitud	0.769	0.011	0.678 a 0.840	0.892	0.803 a 0.967
Precisión <i>compute-bound</i>	0.712	0.016	–	0.865	–
Sensibilidad <i>compute-bound</i>	0.714	0.026	–	0.825	–
F1 <i>compute-bound</i>	0.713	0.015	–	0.844	–
Precisión <i>memory-bound</i>	0.808	0.013	–	0.906	–
Sensibilidad <i>memory-bound</i>	0.806	0.017	–	0.929	–
F1 <i>memory-bound</i>	0.807	0.010	–	0.918	–
F1 macro agrupada	0.760	0.012	–	0.881	–
Exactitud balanceada clásica	0.760	0.012	0.670 a 0.846	0.877	0.750 a 0.965
AUC-ROC agrupada	0.842	0.008	–	0.913	–
Coeficiente de Matthews	0.520	0.023	–	0.763	–
Exactitud balanceada por celda	0.728	0.015	0.669 a 0.790	0.819	0.710 a 0.938

B.1.4. Auditoría de frecuencia de la campaña CPU

Toda corrida se ejecutó con turbo desactivado y con la frecuencia solicitada verificada en cada ventana, con una tolerancia de $\pm 5\%$; una ventana que no cumple ese criterio no recibe etiqueta. Tras cada cambio de nivel se descartan unos 10 ms, porque un procesador delegado tardó entre 10 y 11 ms en engancharse al límite justo después de arrancar el kernel, y se descartan 0.15 s al inicio y al final de cada corrida, porque en los kernels de una sola región paralela larga un hilo que termina antes queda ocioso y diluye el promedio de frecuencia. Al restringir solo los procesadores lógicos delegados, la razón de rendimiento entre el nivel más alto y el más bajo se mantuvo en 0.995; al restringir también sus hermanos de *SMT*, subió a

3.78, cercana a la razón de cuatro esperada entre esos niveles.

B.2. Agente y Fase 4

Tabla B.6: Restauración bajo terminación anómala y coordinación entre procesos.

Prueba	Resultado
Proceso de GPU, candado puesto	El reloj vuelve al valor nativo; código de salida 0
Proceso de CPU, frecuencia modificada	Turbo y rango de frecuencia idénticos al estado inicial; código de salida 0
Ambos procesos a la vez	Ambos salen con código 0; estado de CPU idéntico al inicial y reloj de GPU liberado
Piso de CPU con GPU activa (3.2 GHz)	Respetado en el 99.2 % de las muestras; las restantes (cuatro, de ≈ 0.24 s) están en los flancos de subida

Bibliografía

- [1] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba, “Increasing energy efficiency of astrophysics simulations through gpu frequency scaling,” in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1826–1834.
- [2] M. T. Costa *et al.*, “Characterizing the impact of gpu power management on an exascale system,” in *Proc. SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops ’25)*, New York, NY, USA, 2025, pp. 1524–1533.
- [3] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama, “Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs,” in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, Atlanta, GA, USA, 2024, pp. 1–15.
- [4] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccion, “Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications,” *Concurrency and Computation: Practice and Experience*, vol. 29, no. 12, p. e4143, 2017.
- [5] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen, “An automated and portable method for selecting an optimal gpu frequency,” *Future Generation Computer Systems*, vol. 149, pp. 71–88, 2023.
- [6] R. Gonzalez, B. M. Gordon, and M. A. Horowitz, “Supply and threshold voltage scaling for low power CMOS,” *IEEE Journal of Solid-State Circuits*, vol. 32, no. 8, pp. 1210–1216, 1997.
- [7] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

- [8] K. Goto and R. A. van de Geijn, “Anatomy of high-performance matrix multiplication,” *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008, article 12.
- [9] O. Antepara, L. Oliker, and S. Williams, “Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200,” in *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 1205–1216.
- [10] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, “DVFS-aware application classification to improve GPGPUs energy efficiency,” *Parallel Computing*, vol. 83, pp. 93–117, 2019.
- [11] National Energy Research Scientific Computing Center (NERSC), “Roofline performance model,” NERSC Documentation, 2026, accessed: Aug. 7, 2026. [Online]. Available: <https://docs.nersc.gov/tools/performance/roofline/>
- [12] V. M. Weaver and S. A. McKee, “Can hardware performance counters be trusted?” in *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, Seattle, WA, USA, 2008, pp. 141–150.
- [13] V. M. Weaver, “Self-monitoring overhead of the linux perf event performance counter interface,” in *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 102–111.
- [14] Intel Corporation, “Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters),” 2026, accessed: Aug. 12, 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [15] T.-Y. Liu, J. Guo, and B. Huang, “Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping,” *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 1, pp. 1–26, 2024.
- [16] Linux man-pages project, “perf_event_open(2) — linux manual page,” 2026, accessed: Aug. 29, 2026. [Online]. Available: https://man7.org/linux/man-pages/man2/perf_event_open.2.html
- [17] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.

- [18] S. McCarty, “Architecting containers part 1: Why understanding user space vs. kernel space matters,” Red Hat Blog, 2015, accessed: Apr. 3, 2025. [Online]. Available: <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>
- [19] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le, “RAPL: Memory power estimation and capping,” in *Proc. 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, 2010, pp. 189–194.
- [20] P. Thamm and U. Leser, “Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters,” *arXiv preprint arXiv:2505.09375*, 2025.
- [21] R. Hebbar and A. Milenković, “PMU-events-driven DVFS techniques for improving energy efficiency of modern processors,” *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 7, no. 1, pp. 1–31, 2022.
- [22] Linux Kernel Documentation, “CPU Performance Scaling,” 2026, accessed: Sep. 5, 2026. [Online]. Available: <https://docs.kernel.org/admin-guide/pm/cpufreq.html>
- [23] D. Velicka, O. Vysocky, and L. Riha, “Methodology for GPU frequency switching latency measurement,” in *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 2025, pp. 830–839.
- [24] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2020.
- [25] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*, 2nd ed., ser. Springer Texts in Statistics. Springer, 2021.
- [26] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [27] P. Geurts, D. Ernst, and L. Wehenkel, “Extremely randomized trees,” *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.
- [28] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [29] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.

- [30] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems*, vol. 24, 2011, pp. 2546–2554.
- [31] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan, “Energy delay product,” in *Energy-Efficient High Performance Computing: Measurement and Tuning*, ser. SpringerBriefs in Computer Science. London, UK: Springer, 2013, ch. 6, pp. 51–55.
- [32] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang, “DRLCap: Runtime GPU frequency capping with deep reinforcement learning,” *IEEE Transactions on Sustainable Computing*, vol. 9, no. 5, pp. 712–726, 2024.
- [33] NVIDIA, “Nvidia management library (nvml),” NVIDIA Developer Documentation, 2026, accessed: Apr. 3, 2026. [Online]. Available: <https://docs.nvidia.com/deploy/nvml-api/index.html>
- [34] Linux Kernel Documentation, “intel_pstate cpu performance scaling driver,” 2026, accessed: Sep. 5, 2026. [Online]. Available: https://docs.kernel.org/admin-guide/pm/intel_pstate.html
- [35] Linux perf project, “perf-stat(1) measurement overhead documentation,” 2026, accessed: Sep. 20, 2026. Man pages explicitly warn: “The overhead percentage could be high in some cases, for instance with small, sub 100ms intervals. Use with caution.” Intervals below 10ms incur syscall overhead that dominates measurement time in typical deployments. [Online]. Available: <https://man7.org/linux/man-pages/man1/perf-stat.1.html>
- [36] Intel Corporation, “perfmon: Performance monitoring events for intel architecture processors (event to fp_arith_inst_retired, familia ice lake),” 2026, accessed: Aug. 10, 2026. [Online]. Available: <https://github.com/intel/perfmon>
- [37] NVIDIA, “Nsight compute,” NVIDIA Developer Documentation, 2026, accessed: Aug. 8, 2026. [Online]. Available: <https://docs.nvidia.com/nsight-compute/>
- [38] —, “CUDA profiling tools interface (CUPTI),” NVIDIA Developer Documentation, 2026, accessed: Sep. 18, 2026. [Online]. Available: <https://docs.nvidia.com/cupti/>
- [39] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. Wiley, 2017.

- [40] D. R. Roberts, V. Bahn, S. Ciuti, M. S. Boyce, J. Elith, G. Guillera-Arroita, S. Hauens-
tein, J. J. Lahoz-Monfort, B. Schröder, W. Thuiller, D. I. Warton, B. A. Wintle, F. Har-
tig, and C. F. Dormann, “Cross-validation strategies for data with temporal, spatial,
hierarchical, or phylogenetic structure,” *Ecography*, vol. 40, no. 8, pp. 913–929, 2017.
- [41] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formula-
tion, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*,
vol. 6, no. 4, pp. 1–21, 2012.
- [42] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, “The balanced accu-
racy and its posterior distribution,” in *2010 20th International Conference on Pattern
Recognition*, 2010, pp. 3121–3124.
- [43] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman
& Hall, 1993.
- [44] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of
Machine Learning Research*, vol. 7, pp. 1–30, 2006.
- [45] G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent
selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11,
pp. 2079–2107, 2010.
- [46] S. Plimpton, “Fast parallel algorithms for short-range molecular dynamics,” *Journal of
Computational Physics*, vol. 117, no. 1, pp. 1–19, 1995.
- [47] A. C. Mallinson, D. A. Beckingsale, W. P. Gaudin, J. A. Herdman, J. M. Levesque, and
S. A. Jarvis, “CloverLeaf: Preparing hydrodynamics codes for exascale,” in *Proceedings
of the Cray User Group 2013*, Napa, CA, USA, May 2013. [Online]. Available:
https://cug.org/proceedings/cug2013_proceedings/includes/files/pap130.pdf
- [48] M. Hollander, D. A. Wolfe, and E. Chicken, *Nonparametric Statistical Methods*, 3rd ed.
John Wiley & Sons, 2014.
- [49] Intel Corporation, “Roofline analysis with intel advisor,” Intel Advisor User Guide,
2026, accessed: Aug. 12, 2026. [Online]. Available: [https://www.intel.com/content/
www/us/en/docs/advisor/user-guide/2026-0/roofline-perspective-in-intel-advisor.html](https://www.intel.com/content/www/us/en/docs/advisor/user-guide/2026-0/roofline-perspective-in-intel-advisor.html)
- [50] —, *3rd Gen Intel Xeon Processor Scalable Family, Codename Ice Lake, Uncore
Performance Monitoring Reference Manual*, May 2021, revision 1.0. [Online]. Available:
<https://cdrdv2-public.intel.com/679093/639778%20ICX%20UPG%20v1.pdf>

- [51] J. D. McCalpin, “Memory bandwidth and machine balance in current high performance computers,” IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, Tech. Rep., Dec. 1995.
- [52] Y. J. Lo, S. Williams, B. Van Straalen, T. J. Ligocki, M. J. Cordery, N. J. Wright, M. W. Hall, and L. Oliker, “Roofline model toolkit: A practical tool for architectural and program analysis,” in *High Performance Computing Systems. Performance Modeling, Benchmarking, and Simulation (PMBS 2014)*, ser. Lecture Notes in Computer Science, vol. 8966. Springer, 2015, pp. 129–148.
- [53] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrishnan, and S. K. Weeratunga, “The NAS parallel benchmarks,” *The International Journal of Supercomputer Applications*, vol. 5, no. 3, pp. 63–73, 1991.
- [54] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, “Rodinia: A benchmark suite for heterogeneous computing,” in *2009 IEEE International Symposium on Workload Characterization (IISWC)*, 2009, pp. 44–54.
- [55] D. A. Beckingsale, J. Burmark, R. Hornung, H. Jones, W. Killian, A. J. Kunen, O. Pearce, P. Robinson, B. S. Ryujin, and T. R. W. Scogland, “RAJA: Portable performance for large-scale scientific applications,” in *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 2019, pp. 71–81.
- [56] Y. Chen, T. A. Davis, W. W. Hager, and S. Rajamanickam, “Algorithm 887: CHOLMOD, supernodal sparse cholesky factorization and update/downdate,” *ACM Transactions on Mathematical Software*, vol. 35, no. 3, pp. 22:1–22:14, 2008.
- [57] I. Karlin, J. Keasler, and R. Neely, “LULESH 2.0 updates and changes,” Lawrence Livermore National Laboratory, Tech. Rep. LLNL-TR-641973, 2013.
- [58] J. Dongarra, M. A. Heroux, and P. Luszczek, “High-performance conjugate-gradient benchmark: A new metric for ranking high-performance computing systems,” *The International Journal of High Performance Computing Applications*, vol. 30, no. 1, pp. 3–10, 2016.
- [59] M. A. Heroux, D. W. Doerfler, P. S. Crozier, J. M. Willenbring, H. C. Edwards, A. Williams, M. Rajan, E. R. Keiter, H. K. Thornquist, and R. W. Numrich, “Improving performance via mini-applications,” Sandia National Laboratories, Tech. Rep. SAND2009-5574, 2009.

- [60] S. Beamer, K. Asanović, and D. Patterson, “The GAP benchmark suite,” 2015.
- [61] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.
- [62] D. Chicco and G. Jurman, “The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation,” *BMC Genomics*, vol. 21, no. 1, p. 6, 2020.

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>

El repositorio incluye las rejillas numéricas completas de la política de frecuencia (ganancia, límites de los intervalos y valores p por nivel y clase), la del umbral de confianza, los resultados del clasificador por nivel de frecuencia y las tablas por escenario de la Fase 4.